

城市垃圾分类运输的路径优化与调度

摘 要

随着城市化进程的加速我国城市生活垃圾产量呈爆发式增长，2023 年清运量已达 4 亿吨，几乎逼近城市处理能力极限，垃圾分类运输成为破解垃圾管理难题的关键环节。该环节需统筹多类型垃圾收集要求、车辆载重与容积限制、中转站时空约束等多维度挑战。在此背景下，构建合适的“中转站选址-路径优化-碳排放控制”综合模型，通过合理规划中转站布局、优化多阶段运输路径实现优化目标对提升城市垃圾运输效率、降低综合管理成本具有重要现实意义。

针对问题一，首先进行了数据可视化处理，进而我们通过设置决策变量，并以最小化总行驶距离为目标，选择路径闭约束，节点覆盖约束，载重可行约束，流量守恒约束为约束条件建立了**带容量约束车辆路径优化（CVRP）模型**，接着我们利用 **OR-Tools 求解器**（通过贪心策略快速生成初始解，再利用元启发式算法在邻域内搜索更优解），得到**总行驶距离为 1108.20km**，同时我们与蚁群算法，节约算法等五个模型进行比较，并进行模型**敏感性分析**，发现 OR-Tools 具有便捷，精确的特点。最后我们给出了模型的 11 个理论的时间复杂度及引入时间依赖的行驶时间和考虑不同车辆速度差异两个改进方向。

针对问题二，我们以最小化每日总运输成本为目标函数，以路径闭约束，任务完整性约束，载重可行约束，容积限制约束为约束条件建立了**带容量约束的异构车队车辆路径问题（HFVRP）**和引入时间约束下的**带时间窗口的异构车队车辆路径问题（HFVRPTW）**，利用 OR-Tools 求解器求解得到**总成本为 2812.67 元**，**总行驶距离为 1065.10km**，同时我们给出了 HFVRPTW 的在不同的时间约束下的求解结果，并通过详细的路径图的变化给出分析。最后我们进行多模型比较和敏感性分析证明 OR-Tools 具有高精度，高稳健性的特点。

针对问题三，我们以最小化总成本和最小化碳排放量作为目标函数，以中转站选址和容量约束，垃圾分配完整性约束，车辆载重与容积约束，时间窗口约束，路径连续性约束作为约束条件建立城市垃圾分类运输中含中转站选址与时间窗口的多目标综合优化调度模型，通过对 31 种站点组合进行穷举，并利用 DEAP 库结合 OR-Tools 求解器来寻找多目标优化的寻找**帕累托最优解**得到仅开启中转站 31 时为最优解，**成本为每日 3617.67 元（十年为 1320.45 万元），每日碳排放量为 1065.65kg（十年为 388.96 万元）**。引入非对称约束后，求解得到**成本为每日 3681.59 元（十年为 1343.78 万元），每日碳排放量为 1086.13kg（十年为 396.44 万元）**，比无约束情况下有所增加。

关键词：OR-Tools；帕累托前沿解；VRP 优化；多模型对比；敏感度分析

一、问题重述

1.1 问题背景

随着城市化进程加快，城市生活垃圾的产生与处理已成为全球可持续发展的重要挑战。我国垃圾产量自 2004 年跃居全球首位后持续增长，2023 年垃圾清运量已达 4 亿吨，逼近城市处理能力极限，垃圾管理问题日益凸显。而垃圾分类运输作为城市环境治理的关键环节，涉及多维度的约束：不同垃圾类型（如厨余垃圾、可回收物、有害垃圾、其他垃圾）的收集要求、运输车辆的载重与容积限制、中转站的处理能力及时空约束（如时间窗口）等，还需平衡运输成本与碳排放控制。在此背景下，通过建立合理的数学模型优化垃圾分类运输的路径规划与调度方案，提升效率并降低综合成本，成为城市管理领域的重要研究课题。

1.2 问题要求

附件 1、2、3、4、5 分别提供了收集点坐标及垃圾产生量、车辆类型参数（载重限制、容积限制）、单位距离运输成本、碳排放系数、多类型垃圾产生量数据、中转站候选位置及时空参数（最大存储量）、时间窗口、建设成本、城市路网矩阵（含对称/非对称距离约束）等信息。此外，垃圾处理厂工作时间设定为 6:00-18:00，车辆行驶速度统一为 40km/h，车型在单一类型问题中载重固定，多类型问题中按类别区分专用车辆，并且允许车辆多次往返运输。为优化城市垃圾分类运输的路径规划与调度，现需结合实际约束与数据建立数学模型分析以下问题：

问题一：针对单一类型垃圾运输场景，1.要求以最小化每日总行驶距离为目标，建立含车辆载重约束的路径优化模型，确定车辆数量、收集点任务分配及往返路径；2.以 30 个收集点（载重限制 5 吨）为例，采用合适算法求解最优调度方案，并分析模型时间复杂度与问题规模的关联性；3.讨论模型局限性（如未考虑动态交通因素）并至少提出一种改进方向增强适应性。

问题二：面向 4 类垃圾的多车辆协同调度场景，1.要求以最小化总运输成本为目标，构建含车辆类型（载重、容积、单位成本）及垃圾量分配约束的数学模型；2.基于问题一单车型算法扩展车辆类型约束与独立调度机制，并分析容积-载重双重限制对路径规划的影响；3.增加车辆每日最大行驶时间约束改进模型并举例说明时间约束对路径规划的影响。

问题三：1.以运输成本与中转站建设成本之和最小化为目标，纳入时间窗口、存储容量及时序约束构建“中转站选址-路径优化-碳排放控制”综合模型；2.根据附件所给数据设计两阶段算法：第一阶段确定中转站选址及收集点分配，第二阶段基于选址结果优化各类型车辆路径，并说明两阶段之间的关联与协同机制；3.针对非对称路网场景，修改距离矩阵并调整已有模型，同时对比对称与非对称路网下路径优化的复杂度差异。

二、问题分析

2.1 问题一的分析

针对问题一，这是一个单一类型垃圾运输场景的优化类问题，我们需要以最小化每日总行驶距离为目标择路径闭约束，节点覆盖约束，载重可行约束，流量守恒约束为约束条件建立了带容量约束车辆路径优化（CVRP）模型，并选择合适的模型对问题进

行求解并分析模型的时间复杂度，针对该模型的局限性给出相应的改进方向，最后进行模型比较分析模型的选择优劣性，并进行模型的灵敏度分析。

2.2 问题二的分析

针对问题二，这是一个面向 4 类垃圾的多车辆协同调度的优化类问题，我们需要以最小化每日总运输成本为目标函数，以路径闭约束，任务完整性约束，载重可行约束，容积限制约束为约束条件建立了带容量约束的异构车队车辆路径问题（HFVRP）和引入时间约束下的带时间窗口的异构车队车辆路径问题（HFVRPTW），并选择合适的模型进行求解，并说明时间约束对路径规划的影响。

2.3 问题三的分析

针对问题三，这是一个聚焦“中转站选址-路径优化-碳排放控制”综合场景的优化类问题，我们以最小化总成本和最小化碳排放量作为目标函数，以中转站选址和容量约束，垃圾分配完整性约束，车辆载重与容积约束，时间窗口约束，路径连续性约束作为约束条件建立城市垃圾分类运输中含中转站选址与时间窗口的多目标综合优化调度模型，并采用合适的策略对中转站进行选址，并对多目标优化问题进行求解。

三、模型假设

1. 假设各收集点每日各类垃圾产生量固定且已知，不考虑随机波动或临时变化。
2. 假设中转站每日运营结束后清空所有垃圾，不考虑跨日存储或累积容量约束。
3. 假设不同类型垃圾的运输任务相互独立，车辆仅运输单一类型垃圾，无混装情况。
4. 假设运输过程中车辆无故障、无延迟，严格按规划路径执行任务。

四、符号说明

符号	说明
$d(i, j)$	节点 i 与节点 j 之间的实际行驶距离
$R_{t,k}$	车辆 t 服务的收集点集合
$w_{i,k}$	收集点 i 的类型 k 垃圾重量
$y_{i,k,m}$	收集点 i 的 k 类垃圾运往设施 m 的量
C_k	类型 k 车辆的单位距离成本
u_{jk}	车辆到达节点 j 时的载重

五、数据可视化

为了更好的分析和理解数据，我们利用 Python 分别绘制了附件一中的坐标和垃圾量的 3D 散点图和附件 3 中不同垃圾的坐标和垃圾量的 3D 曲面图，如图 1 和图 2 所示：

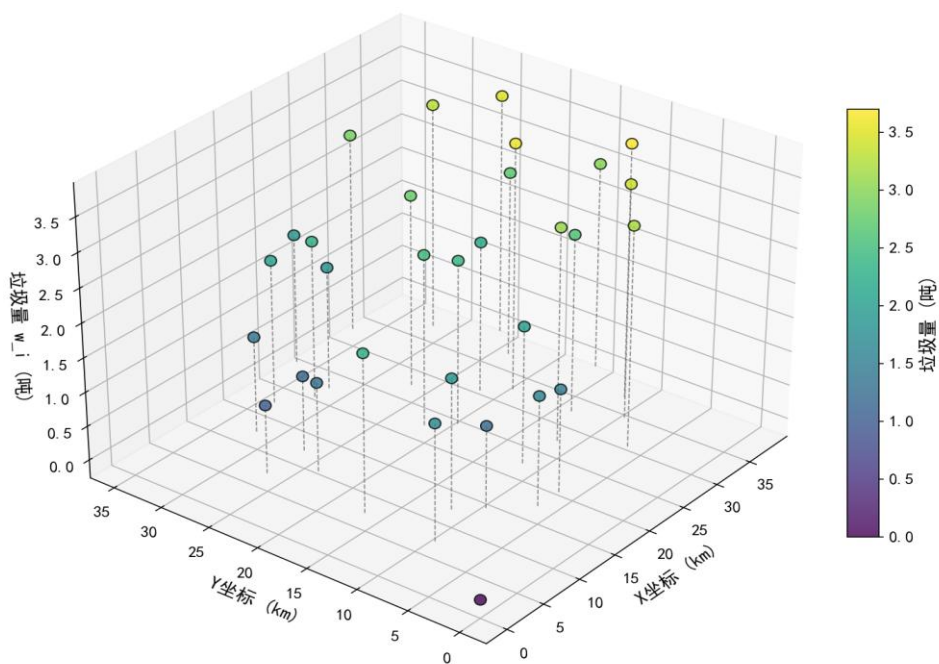


图 1 坐标和垃圾量的 3D 散点分布图

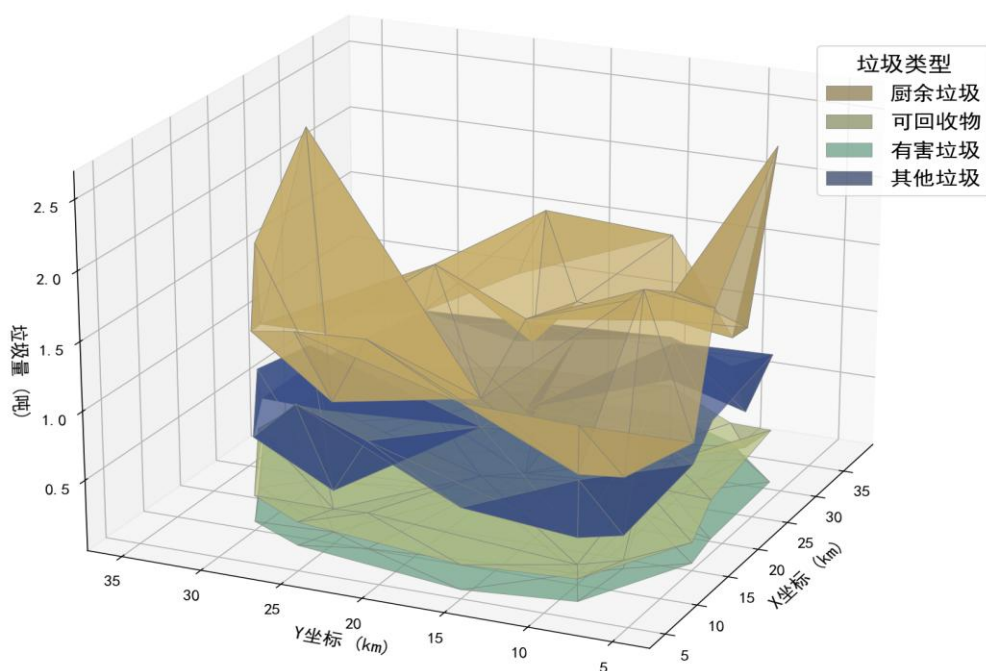


图 2 不同垃圾和垃圾量的 3D 曲面分布图

通过观察图 1 我们发现，附件一中的数据不同点大垃圾数据量有高有低，观察图 2 我们发现厨余垃圾的数量最多，有害垃圾的数量最少，且不同类型的垃圾在不同的坐标点的数量仍有差别，这对我们后面的路径规划的方法选择及其与结果的交叉验证提供了一定的依据。

六、问题一模型建立与求解

问题分析：问题一是一个单一类型垃圾运输场景的优化类问题，我们需要明确以最

小化每日行驶总距离作为约束条件，并确定好对应的约束条件选择合适的算法进行求解，并分析算法的时间复杂度。

6.1 模型建立

● 变量准备

1. 决策变量

在复杂的车辆路径规划场景中，路径的选择直接影响运输效率与成本。为精准刻画车辆在节点间的行驶关系，我们引入 0-1 变量 x_{ijk} 作为路径选择的核心标识，具体定义如下：

$$x_{ijk} = \begin{cases} 1, & \text{车辆 } k \text{ 从节点 } i \text{ 直接行驶至节点 } j \\ 0, & \text{否则} \end{cases} \quad (1)$$

而在车辆载重管理环节，实时跟踪载重状态是确保运输安全与效率的关键。因此我们设置连续变量 u_{ik} 动态记录载货量变化，表示车辆 k 到达节点 i 时的累计载重。

● 优化目标函数设立

在运输调度系统中，行驶距离是影响运营成本的核心驱动要素。从成本结构来看，车辆的燃油消耗、轮胎磨损、司机工时费用以及因行驶产生的碳排放成本等关键支出项均与行驶里程呈显著正相关关系。因此，我们构建最小化总行驶距离目标函数如下：

$$\min \sum_{k \in K} \sum_{i \in N} \sum_{j \in N} d(i, j) \cdot x_{ijk}, \quad (2)$$

其中， K 表示车辆集合； N 为节点集合（包含起点、终点及客户点）； $d(i, j)$ 表示节点 i 与节点 j 之间的实际行驶距离。

● 约束条件

1. 路径闭约束

为确保每辆车从处理厂出发 $\sum_{j \in N} x_{0jk} = 1$ 并返回处理厂 $\sum_{i \in N} x_{i0k} = 1$ ，形成完整的运输循环避免车辆滞留或路径中断得到如下约束条件：

$$\sum_{j \in N} x_{0jk} = 1, \quad \sum_{i \in N} x_{i0k} = 1, \quad \forall k \in K \quad (3)$$

2. 节点覆盖约束

为确保所有垃圾被收集，且无遗漏或重复访问，要求每个收集点（ $j \neq 0$ ）必须恰好被一辆车访问一次 $\sum x_{ijk} = 1$ ：

$$\sum_{k \in K} \sum_{i \in N} x_{ijk} = 1, \quad \forall j \in N \setminus \{0\} \quad (4)$$

3. 载重可行约束

为使车辆到达节点 j 时的载重 u_{jk} 等于到达节点 i 时的载重 u_{ik} 加上节点 j 的需求量 $w_j x_{ijk}$ ，且始终不超过最大容量 Q ，即：

$$u_{jk} = u_{ik} + w_j x_{ijk}, \quad 0 \leq u_{jk} \leq Q, \quad \forall i, j \in N, \forall k \in K \quad (5)$$

4. 流量守恒约束

为确保路径的逻辑一致性（无有去无回或者重复进入的无效路径）我们得到如下流量约束：

$$\sum_{i \in N} x_{ijk} = \sum_{i \in N} x_{jik}, \quad \forall j \in N, \forall k \in K \quad (6)$$

➤ 综上，我们建立了单一车辆类型下的带容量约束车辆路径优化（CVRP）调度模型如下：

$$\left\{ \begin{array}{l} \min \sum_{k \in K} \sum_{i \in N} \sum_{j \in N} d(i, j) \cdot x_{ijk} \\ s.t. \left\{ \begin{array}{l} \sum_{j \in N} x_{0jk} = 1, \quad \sum_{i \in N} x_{i0k} = 1, \quad \forall k \in K \quad (\text{路径闭约束}) \\ \sum_{k \in K} \sum_{i \in N} x_{ijk} = 1, \quad \forall j \in N \setminus \{0\} \quad (\text{节点覆盖约束}) \\ u_{jk} = u_{ik} + w_j x_{ijk}, \quad 0 \leq u_{jk} \leq Q, \quad \forall i, j \in N, \forall k \in K \quad (\text{载重可行约束}) \\ \sum_{i \in N} x_{ijk} = \sum_{i \in N} x_{jik}, \quad \forall j \in N, \forall k \in K \quad (\text{流量守恒约束}) \end{array} \right. \end{array} \right. \quad (7)$$

6.2 模型求解

在下文中的求解中我们主要采用 OR-Tools 求解器对问题进行求解，OR-Tools 是谷歌开发的一款开源优化工具库，可高效解决线性规划、整数规划、约束规划、车辆路径问题（VRP）等多种优化问题。其中对于 VRP[1]问题提供专门的路由求解器，支持容量约束（CVRP）、时间窗口（VRPTW）、取送货（VRPPD）等多种场景。

考虑到该问题提出的 30 个收集点和 1 个处理厂的带容量约束车辆路径优化（CVRP）需求，我们采用 Google 开发的开源组合优化工具库 OR-Tools 中的路由求解器，OR-Tools 将 CVRP 转化为带容量约束的最短路径覆盖问题，通过贪心策略快速生成初始解，再利用元启发式算法在邻域内搜索更优解，最终输出满足所有约束的最小总距离方案求解中小规模 CVRP，同时平衡效率和精度。本文求解流程图如下图 3 所示：

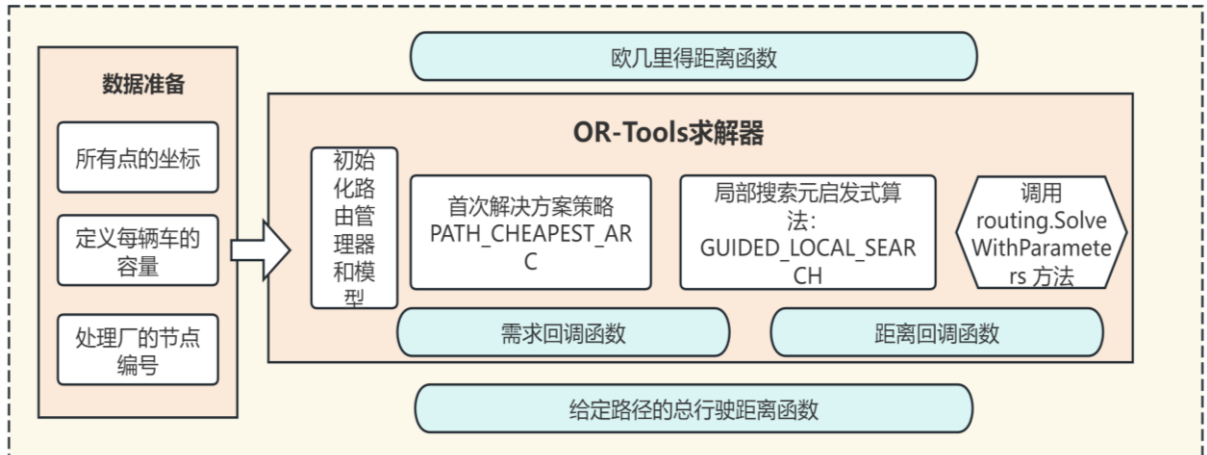


图 3 问题一求解流程图

最终利用 Python 求解得到以下结果：

表 1 问题一路径规划结果表

总行驶距离	1108.20km
车辆数量	15 辆
求解耗时	5.0254 秒

表 2 问题一路径规划具体调度表

车辆编号	具体路径	载重/吨	路径距离/km
车辆 1	0->15->4->0	5.00	56.07
车辆 2	0->5->22->0	4.90	83.03
车辆 3	0->29->0	3.70	72.80
车辆 4	0->13->6->0	4.70	57.12
车辆 5	0->16->28->0	4.80	73.47
车辆 6	0->2->11->0	4.00	32.25
车辆 7	0->12->24->27->0	4.50	73.29
车辆 8	0->7->30->0	4.80	95.44
车辆 9	0->26->1->0	4.80	70.00
车辆 10	0->18->23->0	4.80	99.05
车辆 11	0->9->14->0	5.00	70.74
车辆 12	0->8->17->0	4.40	98.52
车辆 13	0->10->25->0	5.00	81.90
车辆 14	0->20->21->0	5.00	66.06
车辆 15	0->3->19->0	4.60	78.48

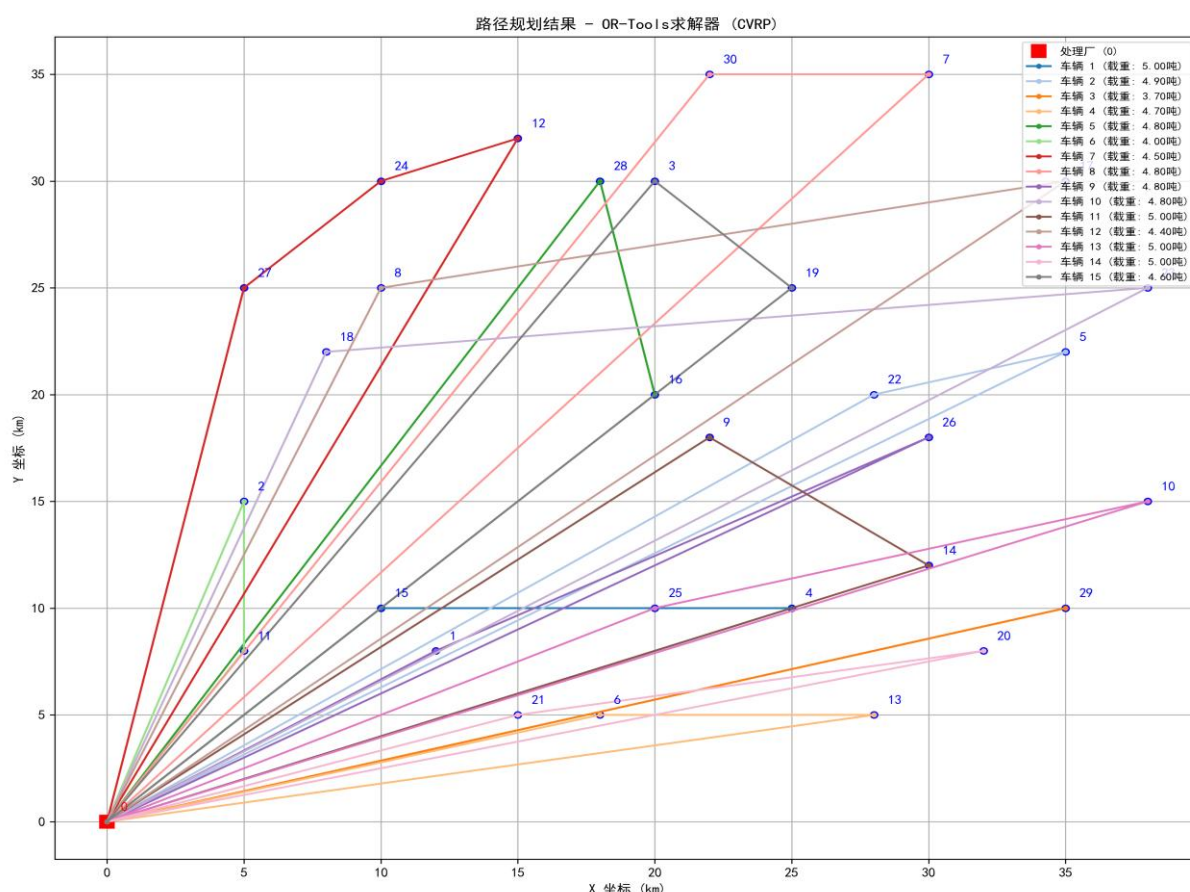


图 4 问题一路径规划结果图

观察上述规划结果我们可以发现，在满足车辆最大载重 5 吨约束条件下，有效地将 30 个收集点合理分配至 15 辆运输车辆，且各车辆的载重分布均接近于 5.00 吨，车辆空载距离较小，本模型所求结果较为准确。最终我们得到总行驶距离为 1108.20km，最终

调度车辆数为 15 辆。

同时实际上在本问题中，我们假设车辆返回处理厂之后不再进行使用，是因为在仅要求最小化每日总行驶距离且对车辆无时间限制的情况下，我们完全可以使用一辆车去复制所有车辆的行驶轨迹也可以多辆车往返，因此在本问题中对于使用车辆的范围为 0~15 辆均可。

6.3 模型评价与灵敏度分析

6.3.1 多模型比较

在此部分我们通过将 OR-Tools 求解器（时间限制为 5 秒；距离和需求分别放大 10 0 倍，10 倍）与节约算法[2]（Clarke-Wright），蚁群算法[4]（ACO）（num_ants=30；迭代次数为 200；alpha=1.0；beta=3.0；每轮迭代后信息素衰减比例为 0.5；信息素更新强度为 1.0），聚类+TSP 错误!未找到引用源。启发式（KMeans 簇数由总需求/车辆容量估计；设置随机种子为 42；自动选择初始质心数量），贪心算法（最近邻）这几种算法进行比较阐释我们选择该算法的原因。

表 3 不同算法求解效果比较

算法	总行驶距离/km	车辆数量	求解耗时/秒
ACO	1131.38	15	3.1821
Clarke-Wright	1157.23	16	0.0009
OR-Tools	1108.20	15	5.0141
KMeans	1424.20	21	0.1840
最近邻	1350.81	18	0.0015

观察上表我们可以看出，OR-Tools 求解器（CVRP）所求得的总行驶距离为最短路径，但实际上我们观察到它的求解耗时是比其他的模型要长的，实际上这是因为我们对于 OR-Tools 的求解时间的限制设置为 5s 所导致的，在下面的参数敏感度分析中我们将给出即使将模型的求解限制减小我们依然可以得到比其他模型更优的解，但并非为本题的最优解。

6.3.2 OR-Tools 求解器求解时间限制灵敏度分析

在上述的求解过程中我们对于 OR-Tools 的最大限制时间设置为 5s，我们在此处对该参数设置进行模型的灵敏度分析，将求解时间限制从 1 秒到 10 秒（步长 1 秒）绘制得到总行驶距离与参数的关系如下图 5 所示：

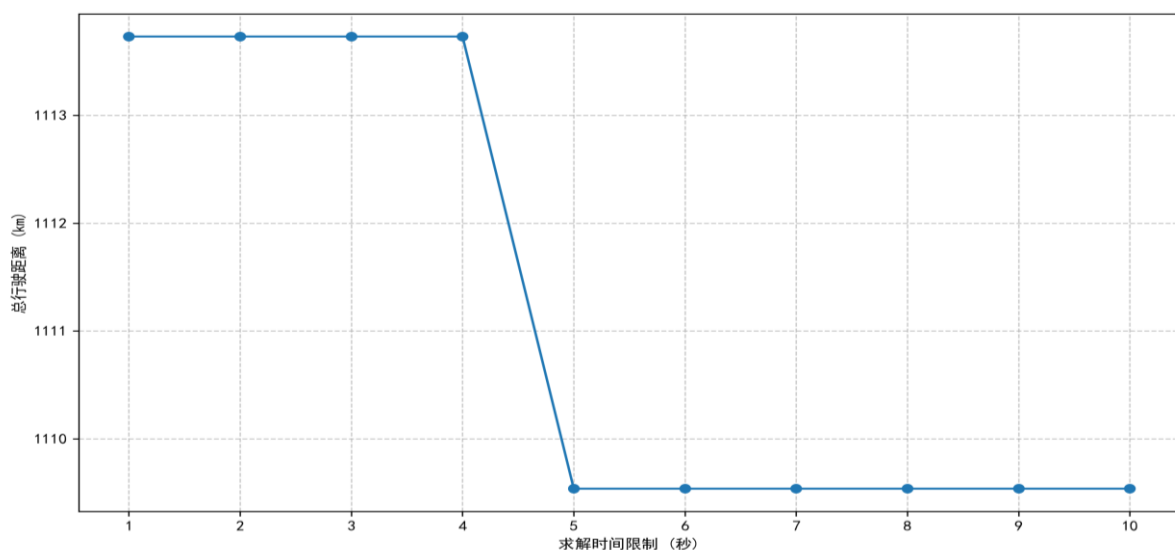


图 5 求解时间限制对总行驶距离影响的灵敏度分析

观察上图我们可以看出在求解时间为 1s 时我们算得的结果为 1113.73km 这依然比上述的所有模型都优秀，同时再将求解时间限制放大之后进步并没有很大的差异，因此我们认为 OR-Tools 求解器在本文中是一个稳健性非常强的模型，这得益于它的首次解决方案策略和局部搜索元启发式算法，在求解器中这些均可以自动选择较好的算法进行求解。

6.4 模型的时间复杂度分析

- 数据定义和常量：仅是常量和字典初始化，时间复杂度为 $O(n)$ （构造字典需要遍历所有点）。
- 欧几里得距离函数：只做常数次的坐标访问和算术运算，时间复杂度为 O 。
- 路线距离累加函数：遍历给定路径上每条边，最坏路径长度可达 $O(n)$ ，每次调用又是 $O(1)$ ，故总体 $O(m) \leq O(n)$ 。
- 可视化函数：遍历所有点与所有路径节点，总操作数为

$$O(n + \sum_i |\text{route}_i|) = O(n + k) \approx O(n) \quad (8)$$

- 构建 OR-Tools 求解器（常数开销）：仅是内部结构初始化，时间复杂度可视为 $O(1)$ 。
- 注册回调函数：注册本身为常数操作 $O(1)$ 。
- 设置容量约束：内部需要为每个车辆初始化容量维度，复杂度约为 $O(k)$ 。
- 首次可行解策略（*PATH_CHEAPEST_ARC*）：这一策略需要在所有点之间选取“最便宜的下一跳”构造解，每一步要在剩余节点中找最小代价边，最坏需检查 $O(n)$ 条边；总共要插入 n 次节点，故时间约 $O(n^2)$ 。
- 引导式局部搜索（*GUIDED_LOCAL_SEARCH*）：每次邻域移动（如 2-opt、节点交换等）评估代价通常需 $O(n^2)$ ；搜索次数受时间限制或预设迭代次数 N 限制；若以 N 次迭代计，则整体约为 $O(Nn^2)$ ；当将时间限制看作常数，则可近似为 $O(n^2)$ 。
- 提取解并计算总距离：总遍历次数不超过所有车辆路径节点之和 $\leq n + k$ ，故为 $O(n + k) \approx O(n)$ 。
- 脚本主函数中的打印与绘图调用：打印操作视为 $O(n)$ （打印每条路径）；调用 *plot_routes* 为 $O(n)$ 。

6.5 模型潜在的改进点

6.5.1 改进方向 1：引入时间依赖的行驶时间（交通拥堵）

交通拥堵会导致行驶时间随出发时间或时段变化，例如早高峰比平峰时段耗时更长。为解决这一问题，可以将行驶时间建模为与时间相关的动态变量，并在模型中反映这种变化。由此我们选择从约束条件和目标函数两个角度进行改进：

● 在约束条件上的改进

在原模型中若存在时间窗口约束（如客户要求在一定时间段内送达），行驶时间通常是固定的（基于距离和恒定速度）。因此我们作出如下改进：

1. 收集历史交通数据，计算每个路段在不同时段的平均行驶时间；
2. 在 OR-Tools 中通过自定义时间回调函数实现，根据车辆从节点 i 出发的时间动态返回 $t_{ij}(t)$ ，即将行驶时间 t_{ij} （从节点 i 到节点 j 的时间）定义为出发时间 t 的函数。

- 在目标函数上的改进

由于交通拥堵可能会行驶时间，因此我们将目标改进为最小化总行驶时间如下：

$$\min \sum_{i,j} t_{ij}(a_i) x_{ij}, \quad (10)$$

其中， $t_{ij}(a_i)$ 是动态计算的行驶时间，反映交通状况对路径选择的影响。

若时间和距离重要性都较为重要，则可以采用加权目标函数如下：

$$\min w_1 \sum_{i,j} d_{ij} x_{ij} + w_2 \sum_{i,j} t_{ij}(a_i) x_{ij} \quad (11)$$

6.5.2 改进方向 2：考虑不同车辆速度差异

不同类型或负载的车辆(如重型卡车和轻型货车)具有不同的速度特性，假设所有车辆速度一致，这可能会导致时间估计偏差。为此，可以为每辆车定义独特的速度模型进行改进：

$$t_{ij}^k = d_{ij} / v_k, \quad (12)$$

其中， t_{ij}^k 表示每辆车 k 经过节点 i 和 j 的时间； d_{ij} 为两节点间距离； v_k 为每辆车的速度。

若进一步考虑负载对速度的影响，则可定义速度函数 $v_k(w)$ ，其中 w 是当前负载：

$$v_k(w) = v_k^{\text{空载}} - \alpha w, \quad (13)$$

其中， w 是当前负载； α 是负载影响系数。

- 在目标函数上的改进

同样地，由于原目标函数（最小化总距离）无法反映不同速度的差异，因此我们将目标改进为最小化总行驶时间如下：

$$\min \sum_{i,j,k} t_{ij}^k x_{ij}^k \quad (14)$$

若不同车辆有不同运营成本（如燃油消耗等），此时可引入成本系数 c_k ：

$$\min \sum_{i,j,k} c_k t_{ij}^k x_{ij}^k \quad (15)$$

七、问题二模型建立与求解

问题分析：问题二一个面向 4 类垃圾的多车辆协同调度的优化类问题，们需要以最小化每日总运输成本为目标函数，同时确定好约束条件，最后选择合适的算法进行求解，以及在添加时间限制约束条件之后对其进行求解，并分析时间限制约束对于线路选择的影响。

7.1 模型建立

- 优化目标函数设立

最小化每日总运输成本，即各类垃圾运输车辆的行驶距离与单位距离成本乘积之和：

$$\min \sum_{k=1}^4 \sum_{t=1}^{m_k} d_{t,k} C_k, \quad (16)$$

其中， k 为垃圾类型（1-厨余垃圾，2-可回收物，3-有害垃圾，4-其他垃圾）， t 为第 t 辆对应类型的车辆， m_k 为类型 k 的车辆数， $d_{t,k}$ 为车辆 t 的行驶距离， C_k 为类型 k 车辆的单位距离成本。

- 约束条件

基于问题一的约束条件，结合问题二对不同种类垃圾的要求我们得到如下约束条件：

1.任务完整性约束

为确保每个收集点的每种垃圾需求都被完全满足：

$$\sum_{t=1}^{m_k} \mathbb{I}(i \in R_{t,k}) w_{i,k} = w_{i,k}, \quad \forall i, k, \quad (17)$$

其中， $\mathbb{I}(\cdot)$ 为指示函数，若收集点 i 在车辆 t 的路径中则为1，否则为0。

2.载重可行约束

每辆车运输的垃圾重量不超过其最大载重 Q_k ：

$$\sum_{i \in R_{t,k}} w_{i,k} \leq Q_k, \quad \forall t, k, \quad (18)$$

其中， $R_{t,k}$ 为车辆 t 服务的收集点集合， $w_{i,k}$ 为收集点 i 的类型 k 垃圾重量。

3.容积限制约束

每辆车运输的垃圾体积不超过其最大容积 V_k ，体积由重量与比容（ V_k/Q_k ）计算得到。实际问题解决过程中，我们通过比容将重量转换为体积确保双重约束条件：

$$\sum_{i \in R_{t,k}} w_{i,k} \frac{V_k}{Q_k} \leq V_k, \quad \forall t, k \implies \sum_{i \in R_{t,k}} w_{i,k} \leq Q_k \quad (\text{等价于载重约束}) \quad (19)$$

4.路径闭约束

与问题一相同（具体公式见式(3)）。

➤ 综上，我们建立了前两问多车辆协同与载重约束下的带容量约束的异构车队车辆路径问题（HFVRP）调度模型如下：

$$\left\{ \begin{array}{l} \min \sum_{k=1}^4 \sum_{t=1}^{m_k} d_{t,k} C_k, \\ s.t. \left\{ \begin{array}{l} \sum_{j \in N} x_{0jk} = 1, \quad \sum_{i \in N} x_{i0k} = 1, \quad \forall k \in K \quad (\text{路径闭约束}) \\ \sum_{t=1}^{m_k} \mathbb{I}(i \in R_{t,k}) w_{i,k} = w_{i,k}, \quad \forall i, k, \quad (\text{任务完整性约束}) \\ \sum_{i \in R_{t,k}} w_{i,k} \leq Q_k, \quad \forall t, k, \quad (\text{载重可行约束}) \\ \sum_{i \in R_{t,k}} w_{i,k} \frac{V_k}{Q_k} \leq V_k, \quad (\text{容积限制约束}) \end{array} \right. \end{array} \right. \quad (20)$$

第三问根据题目要求在前两问的基础上扩展“车辆每日最大行驶时间约束”如下：

● 扩展时间约束

每辆车每日行驶时间不超过最大时长 H ，行驶时间由距离与速度（40 km/h）计算：

$$\frac{d_{t,k}}{40} \leq H, \quad \forall t, k \quad (21)$$

➤ 综上，我们建立了前两问多车辆协同与载重约束下的带时间窗口的异构车队车辆路径问题（HFVRPTW）调度模型如下：

$$\left\{ \begin{array}{l} \min \sum_{k=1}^4 \sum_{t=1}^{m_k} d_{t,k} C_k, \\ s.t. \left\{ \begin{array}{l} \sum_{j \in N} x_{0jk} = 1, \quad \sum_{i \in N} x_{i0k} = 1, \quad \forall k \in K \quad (\text{路径闭约束}) \\ \sum_{t=1}^{m_k} \mathbb{I}(i \in R_{t,k}) w_{i,k} = w_{i,k}, \quad \forall i, k, \quad (\text{任务完整性约束}) \\ \sum_{i \in R_{t,k}} w_{i,k} \leq Q_k, \quad \forall t, k, \quad (\text{载重可行约束}) \\ \sum_{i \in R_{t,k}} w_{i,k} \frac{V_k}{Q_k} \leq V_k, \quad (\text{容积限制约束}) \\ \frac{d_{t,k}}{40} \leq H, \quad \forall t, k \quad (\text{扩展时间约束}) \end{array} \right. \end{array} \right. \quad (22)$$

7.2 模型求解

7.2.1 无时间扩展约束

基于问题一的 OR-Tools 求解器（求解时间限制改为 18 秒），我们对其进行扩展采用分类型独立求解和全局成本汇总策略利用 Python 求解得到 4 类垃圾在无时间约束条件下的多车辆调度结果如下：

表 4 问题二无时间扩展约束四种垃圾车辆路径规划调度结果表

车辆编号	成本/元	载重/吨	路径距离/km	体积/m3
厨余垃圾车辆 1	160.25	7.96	64.10	19.90
厨余垃圾车辆 2	183.96	7.99	73.59	19.96
厨余垃圾车辆 3	203.68	7.56	81.47	18.90
厨余垃圾车辆 4	280.00	7.92	112.00	19.81
厨余垃圾车辆 5	213.49	7.96	85.39	19.90
厨余垃圾总计	1041.38	39.39	416.55	98.47
可回收物车辆 1	172.35	2.40	86.18	10.01
可回收物车辆 2	261.35	5.98	130.68	24.93
可回收物总计	433.71	8.38	216.85	34.94
有害垃圾车辆 1	877.42	2.31	175.48	7.67
有害垃圾总计	877.42	2.31	175.48	7.67
其他垃圾车辆 1	213.74	9.77	118.74	17.59
其他垃圾车辆 2	191.97	9.93	106.65	17.88
其他垃圾车辆 3	58.04	1.61	32.25	2.90
其他垃圾总计	463.75	31.31	257.64	38.37
所有垃圾总计	2816.26	81.39	1066.52	179.45

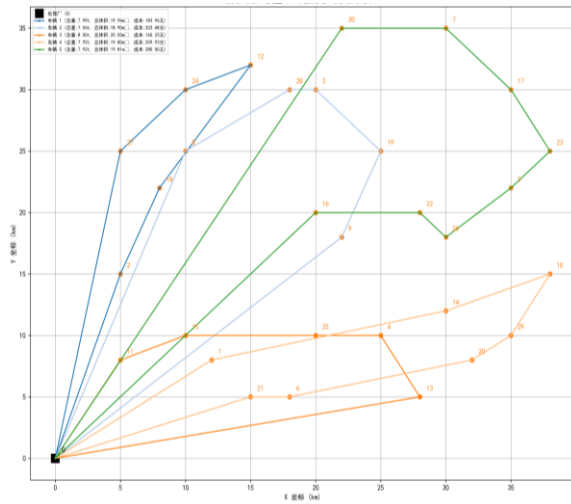


图 6 厨余垃圾路径规划

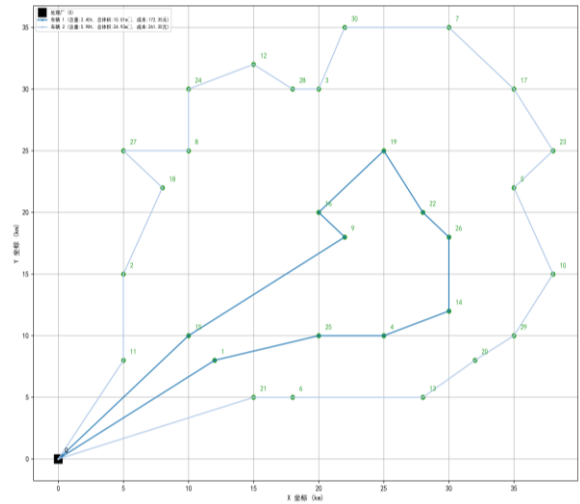


图 7 可回收物路径规划

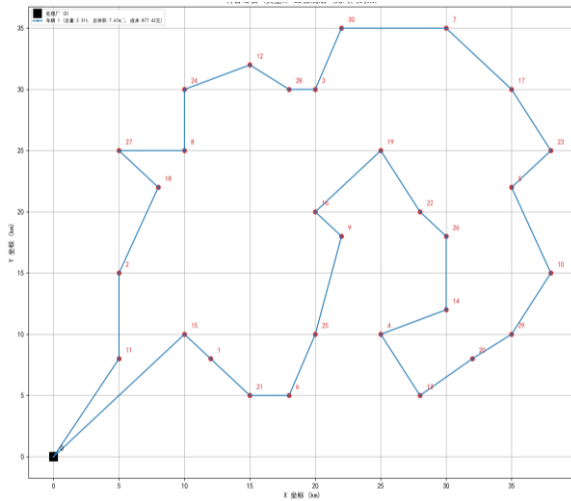


图 8 有害垃圾路径规划

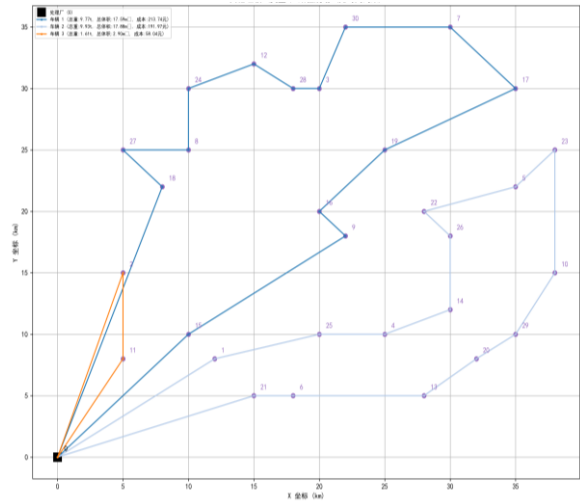


图 9 其他垃圾路径规划

观察上述结果我们可以得到，最终的最小成本为每天 2816.26 元，总载重为 81.39 吨，每日总路径距离为 1066.52km，通过观察具体的分布我们可以看到，对于每辆车的载重已经几乎接近其上限，车辆具有很高的利用率，同时通过观察路径规划图我们看出，对出数量最多的厨余垃圾我们利用了多辆车进行运输，对于数量最少的有害垃圾则是使用一辆车跑完了全部的收集点，这与我们在可视化部分所预想的结论基本相符，结果具有较高的可信度。

7.2.2 增加扩展时间约束

为了进一步体现时间约束对于成本及对应的路径选择的影响，我们将车辆的最大行驶时间约束动态设置 2h 到 10h，求解得出总成本变化的折线图如下图 10 所示：

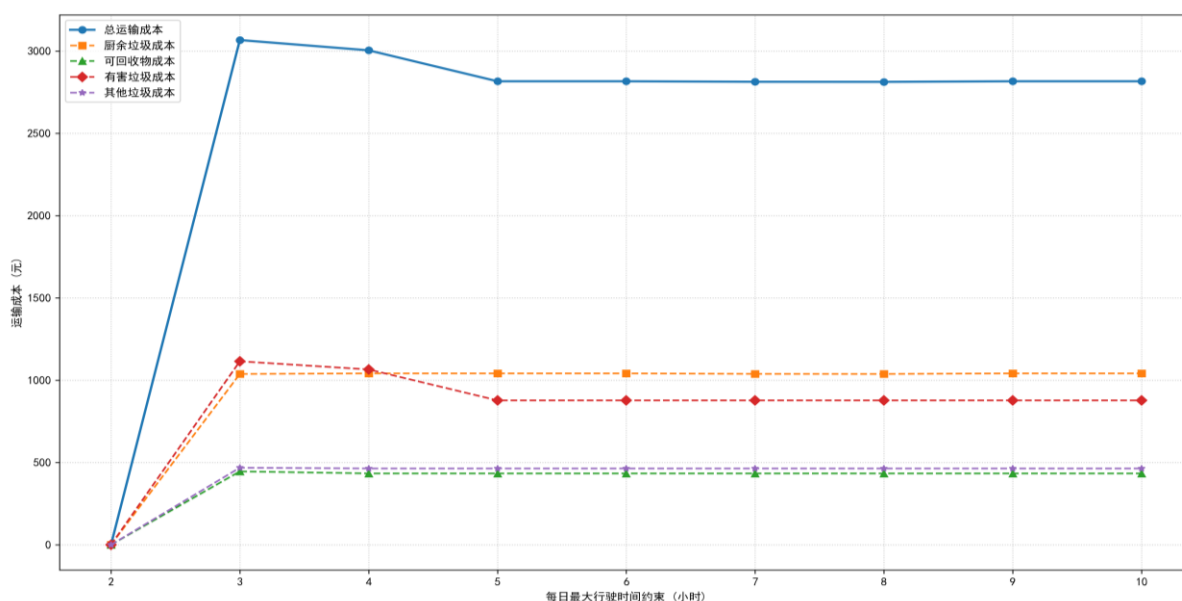


图 10 不同约束时间下的运输成本分析

为了进一步探究时间约束对路径规划的影响，我们分别绘制了 3h, 5h, 7h 情况下四种垃圾的路径规划的对比图以进一步探求时间限制对于路径规划的影响如下所示：

表 5 不同时间约束对四种垃圾成本影响对比

垃圾类型	3 小时成本	5 小时成本	7 小时成本
厨余垃圾	1036.88	1037.80	1037.80
可回收物	445.92	433.71	433.71
有害垃圾	1114.81	877.42	877.42
其他垃圾	471.83	463.75	463.75
总计	3069.44	2812.69	2812.69

表 6 问题二不同垃圾具体路径规划表

车辆编号	成本/元	载重/吨	路径距离/km	体积/m ³
厨余垃圾车辆 1	160.25	7.96	64.10	19.90
厨余垃圾车辆 2	183.96	7.99	73.59	19.96
厨余垃圾车辆 3	203.68	7.56	81.47	18.90
厨余垃圾车辆 4	280.00	7.92	112.00	19.81
厨余垃圾车辆 5	213.49	7.96	85.39	19.90
厨余垃圾总计	1041.38	39.39	416.55	98.47
可回收物车辆 1	172.35	2.40	86.18	10.01
可回收物车辆 2	261.35	5.98	130.68	24.93
可回收物总计	433.71	8.38	216.85	34.94
有害垃圾车辆 1	877.42	2.31	175.48	7.67
有害垃圾总计	877.42	2.31	175.48	7.67
其他垃圾车辆 1	213.74	9.77	118.74	17.59
其他垃圾车辆 2	191.97	9.93	106.65	17.88
其他垃圾车辆 3	58.04	1.61	32.25	2.90
其他垃圾总计	463.75	31.31	257.64	38.37
所有垃圾总计	2816.26	81.39	1066.52	179.45

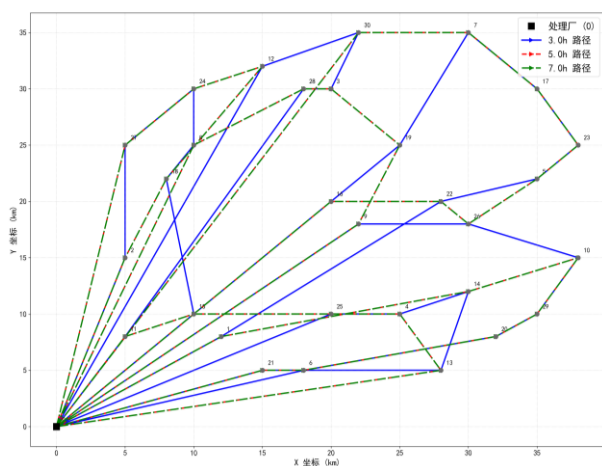


图 11 厨余垃圾指定时间约束下的路径对比

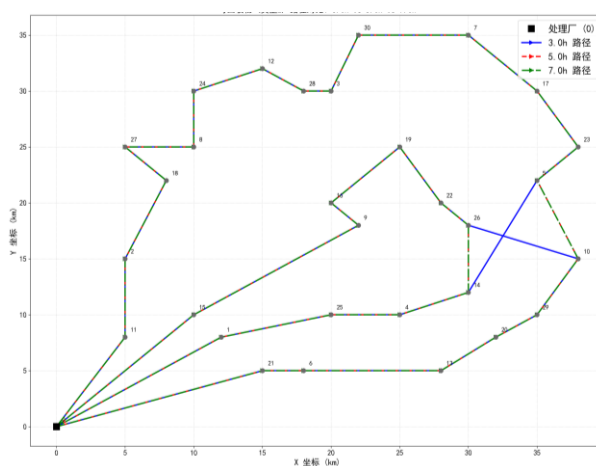


图 12 可回收物指定时间约束下的路径对比

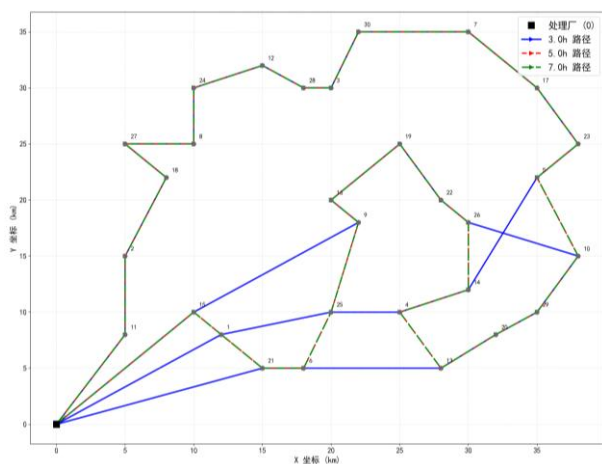


图 13 有害垃圾指定时间约束下的路径对比

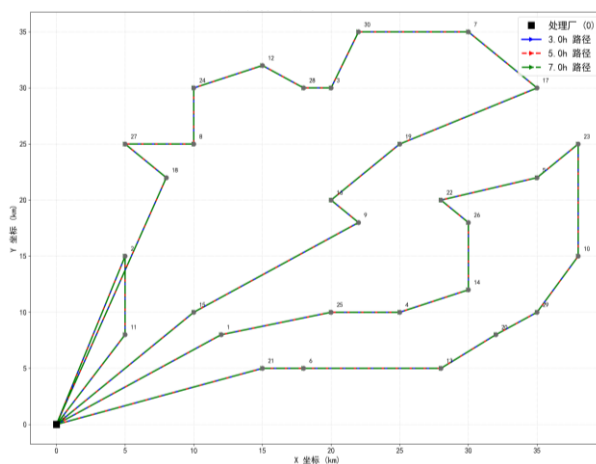


图 14 其他垃圾指定时间约束下的路径对比

通过观察不同时间限制下的路径图我们可以发现，当时间限制为 3h 时路径变化最为明显，当时间约束为 5h, 7h 时不再发生变化，在 3 小时约束下（蓝色实线）：路线较为简单直接，主要集中在图的中部和右部区域，这表明在时间紧张的情况下，车辆选择了更加直接的路径在 5 小时和 7 小时约束下（红色虚线和绿色点划线）路径覆盖逐渐覆盖全面，说明充足的时间允许车辆进行更完整的收集。

同时结合可视化分析部分的图 2 所示，厨余垃圾的数量最多，同时由图 11 我们可以观察到厨余垃圾的路径规划的随时间变动最大，说明垃圾数量多在一定程度上会影响路径规划的结果。

7.3 模型评价与灵敏度分析

7.3.1 多模型比较

在本部分中我们通过贪心算法，遗传算法[4]（小参数：population_size=60, generations=150, elite_size=6, mutation_rate=0.05；大参数：population_size=200, generations=5100, elite_size=20, mutation_rate=0.05），节约算法三种模型进行比较来说明我们模型的优越性，同时我们通过调节遗传算法的参数设置提高遗传算法精确求解的能力，来进一步探究在同一时间下不同模型之间的性能差异。

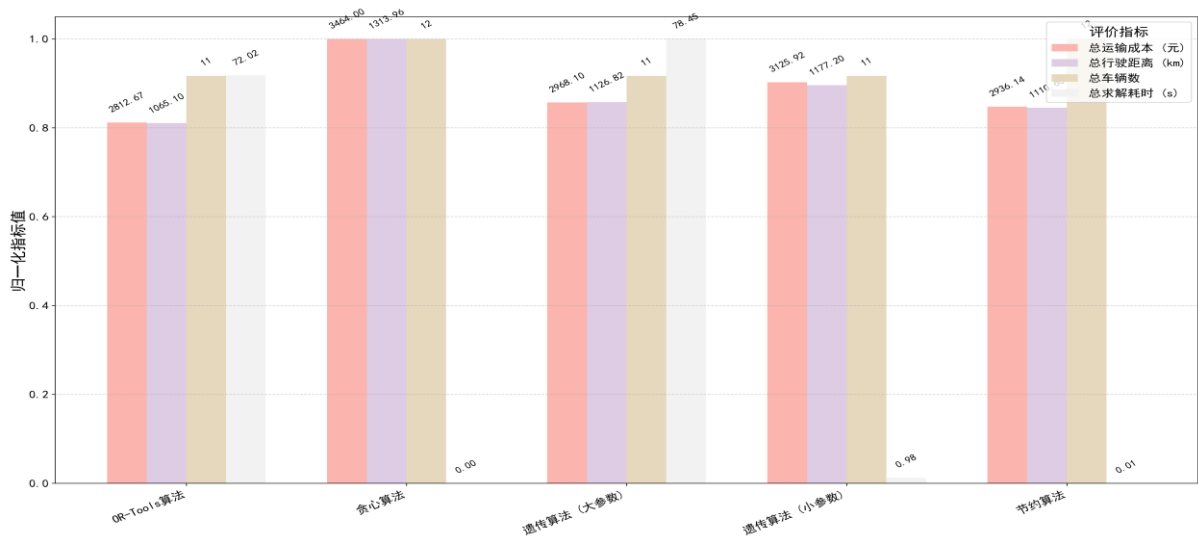


图 15 同一时间下不同模型之间性能差异对比图

表 7 OR-Tools 求解总成本和耗时结果

算法	总成本/元	总行驶距离/km	车辆数量	求解耗时/秒
OR-Tools	2812.67	1065.10	11	72.0154

表 8 不同算法求解总成本和耗时对比

算法	总成本/元	总行驶距离/km	车辆数量	求解耗时/秒
OR-Tools	2812.67	1065.10	11	72.0154
贪心算法	3464.00	1313.96	12	0.0020
遗传算法小参数	2968.10	1126.82	11	78.4547
遗传算法大参数	3125.92	1177.20	11	0.9840
节约算法	2936.14	1110.60	12	0.0080

通过观察结果我们可以看出，OR-Tools 在寻找精确解方面明显要优于其他的模型，并且不需要自己进行相关参数的设置，同时我们发现即使将遗传算法的相关参数调大，在接近相同的时间下仍然没有 OR-Tools 求解的效果好，因此即使时间略长但是在我们认为 OR-Tools 是在追求最优解情况下的最佳的算法。

7.3.2 OR-Tools 求解器求解时间限制灵敏度分析

在上述的求解过程中我们对于 OR-Tools 的最大限制时间设置为 18s，我们在此处对该参数设置进行模型的灵敏度分析，将求解时间限制从 1 秒到 20 秒（步长 1 秒）绘制得到总运输成本与参数的关系如下图 16 所示：

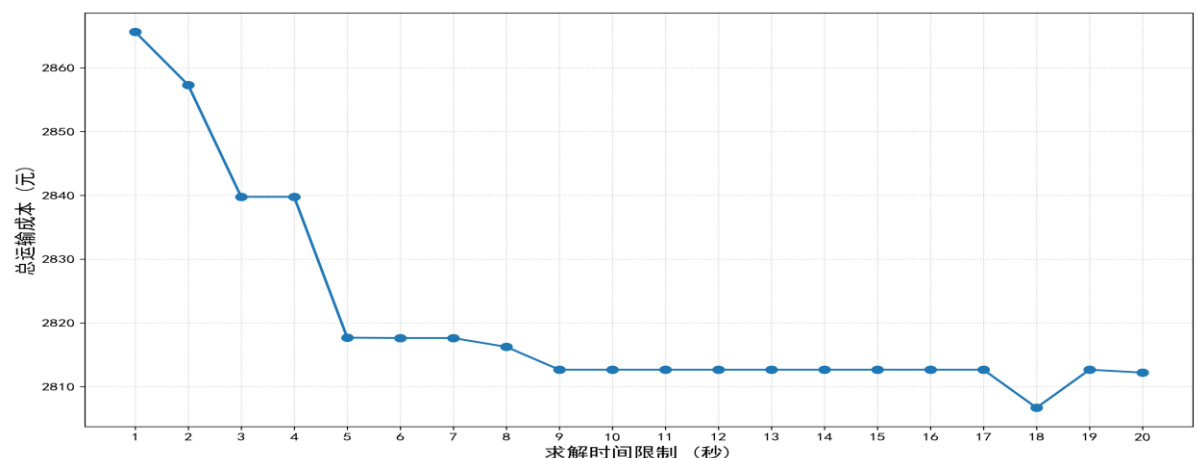


图 16 求解时间限制对总运输成本影响的灵敏度分析

观察图 16 我们可以看出，随着求解时间限制的逐渐提高总运输成本得到的值逐渐趋于精确值，且 9s 之后不再发生变化，从中可以看出 OR-Tools 求解器在参数调整简单的同时所求得的结果也十分准确，具有高精度，高稳定度的特点，因此我们认为 OR-Tools 为本题的最佳模型。

八、问题三模型建立与求解

问题分析：问题三是一个聚焦“中转站选址-路径优化-碳排放控制”综合场景的优化类问题，我们以最小化总成本和最小化碳排放量作为目标函数，并确定好相对应的约束条件，确定合适的中转站选择策略并选择合适的算法对多目标优化问题进行求解，随后添加路网的非对称约束对问题进一步分析求解。

8.1 模型建立

● 变量准备

1. 决策变量

为了准确刻画中转站选址、垃圾分配及路径规划等决策，给“中转站选址-路径优化-碳排放控制”的综合优化提供量化基础，构建得到以下决策变量：

$$x_j = \begin{cases} 1 & \text{选中转站 } j \\ 0 & \text{否则} \end{cases}, \quad (23)$$

$$z_{t,k,i,m} = \begin{cases} 1 & \text{车辆 } t \text{ 的 } k \text{ 类车从点 } i \text{ 行驶至点 } m \\ 0 & \text{否则} \end{cases}, \quad (24)$$

其中， $i, m \in I \cup J \cup \{0\}$ ； $I = \{1, \dots, 30\}$ 表示收集点集合； $J = \{31, \dots, 35\}$ 表示中转站候选集合。

● 优化目标函数设立

根据题目要求，本问需要最小化运输成本与中转站建设成本的同时控制碳排放，因此我们选择采用多目标优化框架： $\min(F_{\text{总}}, E_{\text{总}})$

➤ 最小化总成本 $F_{\text{总}}$ ：包括中转站建设的日均成本与运输成本，反映整个城市管理的经济投入。

➤ 最小化碳排放量 $E_{\text{总}}$ ：利用运输距离和载重量化碳排放目标。

(1) 中转站建设成本（日均摊值）

将一次性基建成本 T_j 均摊至每日（10 年使用期限），转化为与选址决策直接相关的日均成本以便与运输成本统一量纲，具体公式如下：

$$F_{\text{基建}} = \frac{1}{365 \times 10} \sum_{j \in J} T_j x_j \quad (25)$$

(2) 运输成本（两阶段拆分）： $F_{\text{运输}} = F_{\text{阶段1}} + F_{\text{阶段2}}$

● 第一阶段（收集点→设施）：按照垃圾类型 k 、收集点 i 及目标设施 m 计算单程运输成本，距离 $d_{i,m}$ 与单位成本 C_k 的乘积反映实际运输消耗：

$$F_{\text{阶段1}} = \sum_{k \in K} \sum_{i \in I} \sum_{m \in \{0\} \cup J} C_k \cdot d_{i,m} \cdot y_{i,k,m}, \quad (26)$$

其中， $y_{i,k,m}$ 表示收集点 i 的 k 类垃圾运往设施 m 的量（ $m = 0$ 为处理厂， $m \in J$ 为中转站）。

- 第二阶段（中转站→处理厂）：仅对启用的中转站（ $x_j = 1$ ）计算往返处理厂的成本， $d_{j,0} \cdot 2$ 为双向距离，表示中转站二次运输的额外成本：

$$F_{\text{阶段2}} = \sum_{k \in K} \sum_{j \in J} x_j \cdot 2 \cdot C_k \cdot d_{j,0} \cdot \sum_{i \in I} y_{i,k,j} \quad (27)$$

- （3）总碳排放量：根据题目要求量化碳排放公式如下：

$$E_{\text{总}} = \sum_{k \in K} \sum_{\text{车辆 } t} (\alpha_k \cdot D_{t,k} + \beta_k \cdot L_{t,k}), \quad (28)$$

其中，碳排放与行驶距离（ $D_{t,k}$ ）及载重（ $L_{t,k}$ ）正相关， α_k 和 β_k 为车辆类型特定系数。

● 约束条件

1. 中转站选址和容量约束

为确保未选中的中转站容量为 0，启用的中转站各类垃圾存储量不超过指定容量避免过载，我们得到如下约束条件：

$$\sum_{i \in I} y_{i,k,j} \leq S_{j,k} \cdot x_j, \quad \forall j \in J, k \in K \quad (29)$$

2. 垃圾分配完整性约束

为确保没有垃圾遗漏，我们人为设置每个收集点的每类垃圾必须被分配至处理厂或中转站得到如下约束条件：

$$\sum_{m \in \{0\} \cup J} y_{i,k,m} = w_{i,k}, \quad \forall i \in I, k \in K \quad (30)$$

3. 车辆载重与容积约束

为确保车辆同时满足双重限制避免超载或容积不足，我们得到如下约束条件：

$$\begin{cases} \sum_{i \in I} \sum_m z_{t,k,i,m} \cdot y_{i,k,m} \leq Q_k, & \forall t, k \in K \\ \sum_{i \in I} \sum_m z_{t,k,i,m} \cdot y_{i,k,m} \cdot v_k \leq V_k, & \forall t, k \in K \end{cases} \quad (31)$$

4. 时间窗口约束

由于处理厂的固定工作时间（6:00-18:00）与中转站允许停靠时段（ $[a_j, b_j]$ ）限制，为确保车辆作业符合实际运营规则，我们得到如下约束条件：

$$\begin{cases} 6 \leq t_{\text{出发},0} \leq 18, & \text{（处理厂工作时间窗口）} \\ a_j \leq t_{\text{到达},j} \leq b_j, & \forall j \in J \end{cases} \quad (32)$$

5. 路径连续性约束

为避免路径断裂或陷入无效循环，保证问题存在可行解结构，除起点（处理厂）和终点（处理厂）外，还要求车辆在中间节点的流入量等于流出量。由此可得到如下约束条件：

$$\sum_{i \in I \cup \{0\}} z_{t,k,i,m} = \sum_{i \in I \cup \{0\}} z_{t,k,m,i}, \quad \forall m \in I \cup J \cup \{0\}, t, k \in K \quad (33)$$

综上，我么建立城市垃圾分类运输中含中转站选址与时间窗口的多目标综合优化调度模型如下：

$$\begin{cases} \min(F_{\text{基建}} + F_{\text{运输}}, E_{\text{总}}) \\ s.t. \begin{cases} \sum_{i \in I} y_{i,k,j} \leq S_{j,k} \cdot x_j, \quad \forall j \in J, k \in K & (\text{中转站选址和容量约束}) \\ \sum_{m \in \{0\} \cup J} y_{i,k,m} = w_{i,k}, \quad \forall i \in I, k \in K & (\text{垃圾分配完整性约束}) \\ \begin{cases} \sum_{i \in I} \sum_m z_{t,k,i,m} \cdot y_{i,k,m} \leq Q_k, & \forall t, k \in K \\ \sum_{i \in I} \sum_m z_{t,k,i,m} \cdot y_{i,k,m} \cdot v_k \leq V_k, & \forall t, k \in K \end{cases} & (\text{车辆载重与容积约束}) \\ \begin{cases} 6 \leq t_{\text{出发},0} \leq 18, & (\text{处理厂工作时间窗口}) \\ a_j \leq t_{\text{到达},j} \leq b_j, & \forall j \in J \end{cases} & (\text{时间窗口约束}) \\ \sum_{i \in I \cup \{0\}} z_{t,k,i,m} = \sum_{i \in I \cup \{0\}} z_{t,k,m,i}, \quad \forall m \in I \cup J \cup \{0\}, t, k \in K & (\text{路径连续性约束}) \end{cases} \end{cases} \quad (34)$$

第三问根据题目要求考虑实际路网中存在单行道、禁行时段等非对称约束（点*i*到点*j*的距离与点*j*到点*i*的距离不同），因此调整决策变量和约束条件如下：

● **决策变量调整**

路径变量 $z_{t,k,i,m}$ 需区分方向，即 $z_{t,k,i,m}$ 与 $z_{t,k,m,i}$ 为独立变量，表示从*i*到*m*和从*m*到*i*的两个不同行驶决策。

● **约束条件调整**

1. 路径连续性约束调整

流入节点*m*的流量等于流出流量，但允许不同方向的行驶成本不同，即

$$\sum_{i \in N} z_{t,k,i,m} = \sum_{i \in N} z_{t,k,m,i}, \quad \forall m \in N, t, k \quad (N \text{ 为节点集合}) \quad (35)$$

2. 时间窗口约束调整

此时，到达节点*m*的时间需根据行驶方向的距离计算：

$$t_{\text{到达},m} = t_{\text{出发},i} + \frac{d_{i,m}}{v_k}, \quad (36)$$

其中， $d_{i,m}$ 取对应方向的非对称距离。

➤ 综上，我们建立第三问考虑非对称约束含中转站选址、时间窗口及非对称路网的多目标综合优化垃圾分配调度模型如下：

$$\begin{cases} \min(F_{\text{基建}} + F_{\text{运输}}, E_{\text{总}}) \\ s.t. \begin{cases} \sum_{i \in I} y_{i,k,j} \leq S_{j,k} \cdot x_j, \quad \forall j \in J, k \in K & (\text{中转站选址和容量约束}) \\ \sum_{m \in \{0\} \cup J} y_{i,k,m} = w_{i,k}, \quad \forall i \in I, k \in K & (\text{垃圾分配完整性约束}) \\ \begin{cases} \sum_{i \in I} \sum_m z_{t,k,i,m} \cdot y_{i,k,m} \leq Q_k, & \forall t, k \in K \\ \sum_{i \in I} \sum_m z_{t,k,i,m} \cdot y_{i,k,m} \cdot v_k \leq V_k, & \forall t, k \in K \end{cases} & (\text{车辆载重与容积约束}) \\ t_{\text{到达},m} = t_{\text{出发},i} + \frac{d_{i,m}}{v_k} & (\text{时间窗口约束}) \\ \sum_{i \in N} z_{t,k,i,m} = \sum_{i \in N} z_{t,k,m,i}, \quad \forall m \in N, t, k & (N \text{ 为节点集合}) \quad (\text{路径连续性约束}) \end{cases} \end{cases} \quad (37)$$

8.2 模型求解

在本问中我们继续选择利用 OR-Tools 求解器，来进行求解，题中表述“假设中转站每日均可清空。”在此处我们认为是中转站每日需要将所有的垃圾运送至处理站进行处理鉴于站点的数量只有 5 种，在此处我们不再采用 NSGA-II[7]或最近邻等算法来对中转站的选择进行判断，因为对于 NSGA-II 来讲他的时间复杂度将会非常高，而对于最近邻等贪心算法可能无法得到问题的最优解来，因此我们对于不同站点的组合采用穷举法进行求解可得共有 31 种组合情况（ $2^5-1=31$ 种），我们利用 DEAP 库结合 OR-Tools 求解器来寻找多目标优化的寻找帕累托最优解，最终我们得到下表结果：

表 9 问题三考虑中转站选址和时间窗口调度结果表

序号	组合	成本/元	排放/kg	路径距离/km
1	[35]	4564.09	1345.76	1640.41
2	[34]	4100.19	1208.98	1464.61
3	[34, 35]	5318.42	1528.43	1838.58
4	[33]	4531.89	1345.26	1616.24
5	[33, 35]	5539.99	1598.56	1918.29
6	[33, 34]	5169.55	1491.89	1782.70
7	[33, 34, 35]	6164.68	1726.41	2057.71
8	[32]	4615.40	1361.56	1652.05
9	[32, 35]	5538.82	1580.36	1894.44
10	[32, 34]	5243.43	1505.79	1802.94
11	[32, 34, 35]	5959.43	1646.41	1958.89
12	[32, 33]	5571.64	1601.90	1937.80
13	[32, 33, 35]	6349.94	1765.41	2104.28
14	[32, 33, 34]	6083.96	1697.31	2019.15
15	[32, 33, 34, 35]	7112.90	1948.54	2323.59
16	[31]	3617.67	1065.65	1286.99
17	[31, 35]	5191.83	1497.01	1790.15

18	[31, 34]	4759.06	1364.59	1622.45
19	[31, 34, 35]	5694.75	1582.50	1878.30
20	[31, 33]	5014.72	1450.77	1729.87
21	[31, 33, 35]	6015.65	1685.18	2004.29
22	[31, 33, 34]	5805.31	1625.00	1929.54
23	[31, 33, 34, 35]	6829.47	1880.02	2239.08
24	[31, 32]	5231.61	1510.86	1808.44
25	[31, 32, 35]	5963.22	1656.97	1972.83
26	[31, 32, 34]	5685.15	1580.93	1878.81
27	[31, 32, 34, 35]	6717.03	1832.61	2183.62
28	[31, 32, 33]	6051.51	1694.66	2014.97
29	[31, 32, 33, 35]	7043.80	1934.15	2305.95
30	[31, 32, 33, 34]	6661.33	1824.44	2169.74
31	[31, 32, 33, 34, 35]	7677.35	2071.30	2468.63

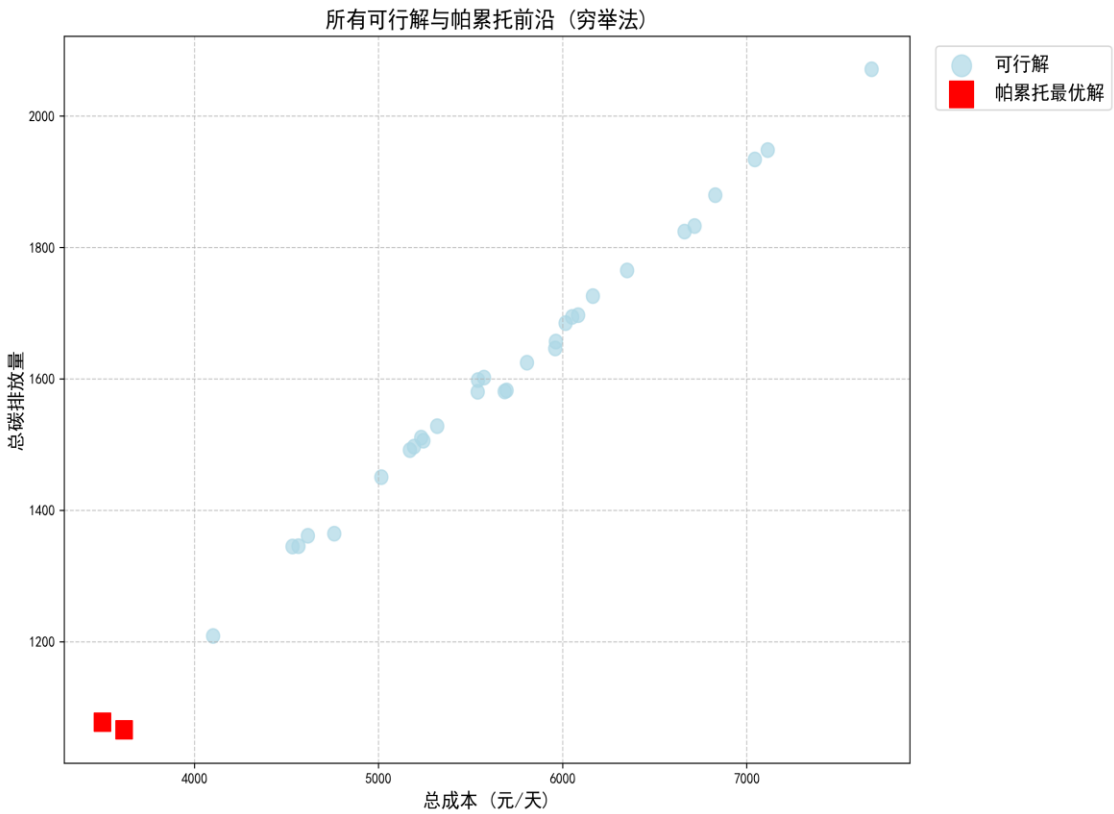


图 17 垃圾运输总成本-碳排放帕累托前沿图

表 10 最优站点选择下的结果汇总表

计量时间	组合	成本/元	排放/kg	路径距离/km
每日	[31]	3617.67	1065.65	1286.99
十年	[31]	1320.45 万	388.96 万	469.75 万

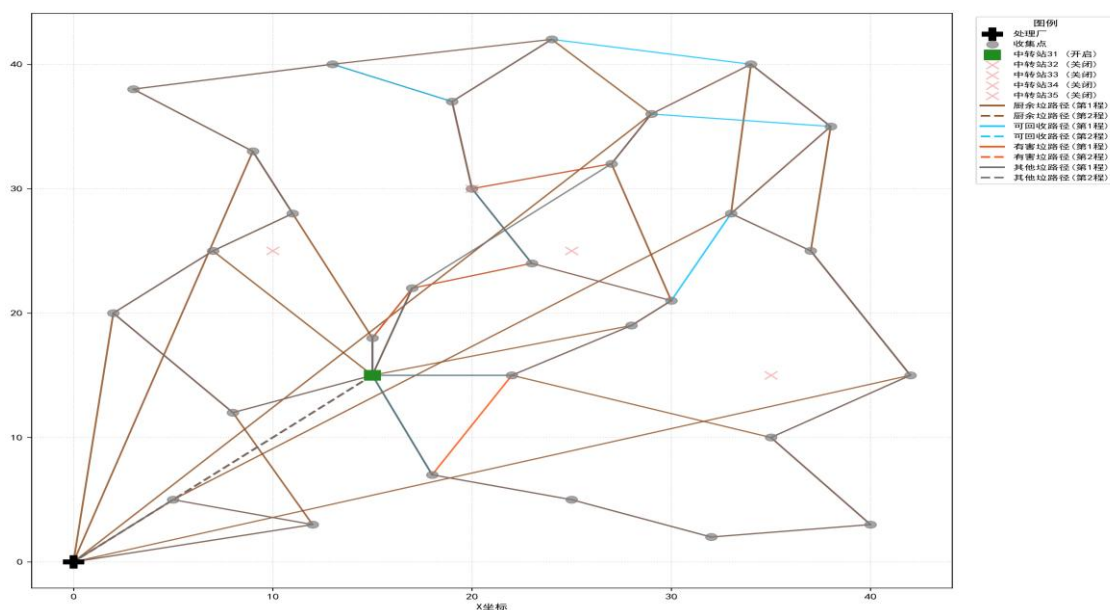


图 18 最优站点选择下的设施与运输路径图

观察上述结果可得，当我们仅开启中转站 31 时模型可以求得最优解，最大成本为每日 3617.67 元（十年为 1320.45 万元），每日碳排放量为 1065.65kg（十年为 388.96 万元），每日路径距离为 1286.99km(十年为 469.75 万元)，说明在加入时间窗约束等约束和中转站的情况下问题三的成本相对有提高，但是中转站的出现减小了垃圾到最近中转站的运输成本。

接着对于非对称路网约束条件下我们仍然采用 OR-Tools 求解器利用 Python 求解得到如下结果：

表 11 非对称约束下规划结果表

中转站组合	总成本/元	总排放/kg	总距离/km
[31]	3681.59	1086.13	1314.00

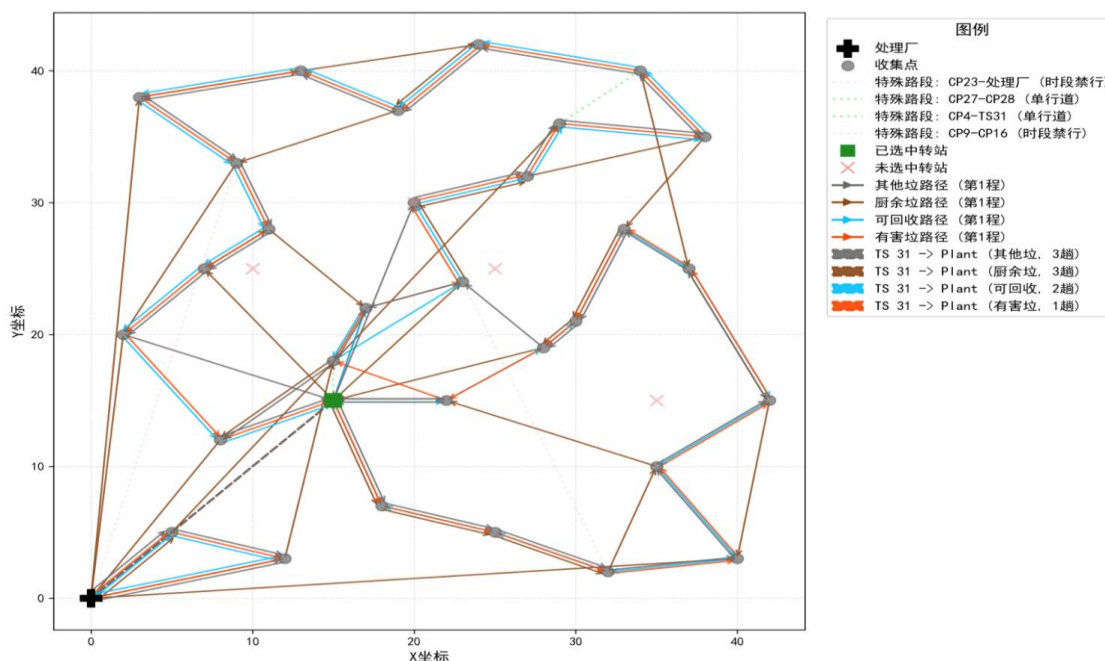


图 19 非对称约束下中转站和车辆路径调度结果图

观察上述结果可知，在纳入单行道、禁行时段等非对称路网约束后，同样仅开启中转站 31 时模型可以求得最优解，每日总成本为 3681.59 元，每日碳排放量为 1086.13kg，每日总距离为 1314.00km，相较单纯考虑对此路网情形各指标均有所增大，而且车辆运输各类型垃圾的趟数（如厨余垃圾运输趟数从对称路网的 3 趟增至 4 趟）和路径规划复杂性也有上升，说明在实际交通路网中的方向性约束显著增加了车辆调度的复杂性。

九、模型评价

● 模型优点：

1. OR-Tools 求解器的选择：该求解器通过贪心策略快速生成初始解，再利用元启发式算法在邻域内搜索更优解具备高精度与强稳健性，同时具有复杂约束高效处理能力对于不同问题均有良好的适用性，同时对于参数的设置简单模型扩展性与易用性强

2. 多模型对比和模型灵敏度分析：通过多模型对比证明模型在相同运行时间下求解精度更高，参数设置简单。

3. 约束条件和目标函数求解得当：对约束条件的精准添加和设置为模型提供了良好的求解环境，同时对于问题三中的多目标优化问题，采用帕累托前沿解的生成策略，可以分别挑选不同要求下的目标函数的最佳值

● 模型缺点：

1. 考虑情况简单：本题的求解中未对行驶速度进行固定值处理，同时并没有对交通情况等假设，因此得到的解实际上是理想情况下的一种最优解，在实际的应用当中，需要考虑多种综合因素才能更好的进行求解。

2. 问题三采用穷举法求解 31 种站点组合，当候选站点数量增加时（如超过 10 个），计算量将呈指数级增长（ 2^n 种组合），难以适用于大规模选址问题。在大规模求解中还需更换成如 NSGA-II 等方式。

本文中的模型可推广至城市垃圾分类运输多场景优化：问题一的 CVRP 模型适用于单一类型垃圾的中小规模路径规划，问题二的 HFVRP/HFVRPTW 模型可实现多类型垃圾协同调度，问题三的多目标综合模型则兼顾中转站选址与碳排放控制；模型依托 OR-Tools 求解器的高效性与多约束整合能力（如载重、容积、时间窗口），具备从单阶段路径优化到全链条调度的扩展潜力，未来可通过引入动态数据、优化算法复杂度进一步提升在复杂路网与实时调度中的适用性。

十、参考文献

- [1]. Dantzig, G. B., & Ramser, J. H., The Truck Dispatching Problem, Management Science, 6(1): 80-91, 1959.
- [2]. Clarke, G., & Wright, J. W., Scheduling of Vehicles from a Central Depot to a Number of Delivery Points, Operations Research, 12(4): 568-581, 1964.
- [3]. Dorigo, M., Maniezzo, V., & Coloni, A., Ant system: optimization by a colony of cooperating agents, IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics), 26(1): 29-41, 1996.
- [4]. Baker, B. M., & Ayechew, M. A., A genetic algorithm for the vehicle routing problem, Computers & Operations Research, 30(5): 787-800, 2003.

- [5].Lin, S., & Kernighan, B. W., An effective heuristic algorithm for the traveling-salesman problem, Operations research, 21(2): 498-516, 1973.
- [6].Google AI, Google OR-Tools, <https://developers.google.com/optimization>, (2025年5月24日) .
- [7].Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T., A fast and elitist multiobjective genetic algorithm: NSGA-II, IEEE Transactions on Evolutionary Computation, 6(2): 182-197, 2002.

十一、附录

问题一代码

```
import math
import random
from sklearn.cluster import KMeans
import numpy as n_p_1
from ortools.constraint_solver import routing_enums_pb2 as r_e_p_1
from ortools.constraint_solver import pywrapcp as p_w_c_p_1
import time
import matplotlib.pyplot as m_p_t_1
import matplotlib.cm as c_m_1
import copy

try:
    m_p_t_1.rcParams['font.sans-serif'] = ['SimHei']
    m_p_t_1.rcParams['axes.unicode_minus'] = False
    m_p_t_1.rcParams['font.weight'] = 'bold'
    m_p_t_1.rcParams['xtick.labelsize'] = 'medium'
    m_p_t_1.rcParams['ytick.labelsize'] = 'medium'
except Exception as e_x_1:
    print(f"Font setting warning: {e_x_1}. Chinese or bold font may not display correctly.")

P_DATA = {
    0: {"coords": (0, 0), "demand": 0, "label": "D0"},
    1: {"coords": (12, 8), "demand": 1.2, "label": "P1"},
    2: {"coords": (5, 15), "demand": 2.3, "label": "P2"},
    3: {"coords": (20, 30), "demand": 1.8, "label": "P3"},
    4: {"coords": (25, 10), "demand": 3.1, "label": "P4"},
    5: {"coords": (35, 22), "demand": 2.7, "label": "P5"},
    6: {"coords": (18, 5), "demand": 1.5, "label": "P6"},
    7: {"coords": (30, 35), "demand": 2.9, "label": "P7"},
    8: {"coords": (10, 25), "demand": 1.1, "label": "P8"},
    9: {"coords": (22, 18), "demand": 2.4, "label": "P9"},
    10: {"coords": (38, 15), "demand": 3.0, "label": "P10"},
}
```

```

11: {"coords": (5, 8), "demand": 1.7, "label": "P11"},
12: {"coords": (15, 32), "demand": 2.1, "label": "P12"},
13: {"coords": (28, 5), "demand": 3.2, "label": "P13"},
14: {"coords": (30, 12), "demand": 2.6, "label": "P14"},
15: {"coords": (10, 10), "demand": 1.9, "label": "P15"},
16: {"coords": (20, 20), "demand": 2.5, "label": "P16"},
17: {"coords": (35, 30), "demand": 3.3, "label": "P17"},
18: {"coords": (8, 22), "demand": 1.3, "label": "P18"},
19: {"coords": (25, 25), "demand": 2.8, "label": "P19"},
20: {"coords": (32, 8), "demand": 3.4, "label": "P20"},
21: {"coords": (15, 5), "demand": 1.6, "label": "P21"},
22: {"coords": (28, 20), "demand": 2.2, "label": "P22"},
23: {"coords": (38, 25), "demand": 3.5, "label": "P23"},
24: {"coords": (10, 30), "demand": 1.4, "label": "P24"},
25: {"coords": (20, 10), "demand": 2.0, "label": "P25"},
26: {"coords": (30, 18), "demand": 3.6, "label": "P26"},
27: {"coords": (5, 25), "demand": 1.0, "label": "P27"},
28: {"coords": (18, 30), "demand": 2.3, "label": "P28"},
29: {"coords": (35, 10), "demand": 3.7, "label": "P29"},
30: {"coords": (22, 35), "demand": 1.9, "label": "P30"}
}
V_CAPACITY = 5.0
D_NODE = 0

def f_1_e(p_1_i, p_2_i):
    p_c_1 = P_DATA[p_1_i]["coords"]
    p_c_2 = P_DATA[p_2_i]["coords"]
    return math.hypot(p_c_1[0] - p_c_2[0], p_c_1[1] - p_c_2[1])

def f_2_c(r_t_1):
    d_i_1 = 0
    for i_1 in range(len(r_t_1) - 1):
        d_i_1 += f_1_e(r_t_1[i_1], r_t_1[i_1+1])
    return d_i_1

def f_3_p(a_n_1, r_s_1, t_d_1, e_t_1):
    pass

def f_4_p(a_n_2, r_s_2, p_d_1, d_n_1):
    f_g_1, a_x_1 = m_p_t_1.subplots(figsize=(12, 10))
    a_x_1.set_title(f"Route Plan - {a_n_2}", fontweight='bold')
    a_x_1.set_xlabel("X Coord (km)", fontweight='bold')

```

```

a_x_1.set_ylabel("Y Coord (km)", fontweight='bold')

for p_i_1, d_a_1 in p_d_1.items():
    x_1, y_1 = d_a_1["coords"]
    if p_i_1 == d_n_1:
        a_x_1.plot(x_1, y_1, 'rs', markersize=10, label=f"Depot ({p_i_1})")
        a_x_1.text(x_1 + 0.5, y_1 + 0.5, str(p_i_1), color='red', fontweight='bold')
    else:
        a_x_1.plot(x_1, y_1, 'bo', markersize=5)
        a_x_1.text(x_1 + 0.5, y_1 + 0.5, str(p_i_1), color='blue', fontweight='bold')

c_m_l_1 = m_p_t_1.cm.tab20.colors

for i_2, r_i_1 in enumerate(r_s_2):
    p_n_1 = r_i_1['path']
    if not p_n_1: continue
    p_c_x_1 = [p_d_1[n_1]["coords"][0] for n_1 in p_n_1]
    p_c_y_1 = [p_d_1[n_1]["coords"][1] for n_1 in p_n_1]

    r_c_1 = c_m_l_1[i_2 % len(c_m_l_1)]
    a_x_1.plot(p_c_x_1, p_c_y_1, linestyle='-', marker='o', markersize=3, color=r_c_1,
label=f"V {i_2+1} (L: {r_i_1['load']:.2f}t)")

a_x_1.grid(True)
a_x_1.legend(loc='upper right', fontsize='small', prop={'weight':'bold', 'size':'small'})
for l_1 in a_x_1.get_xticklabels() + a_x_1.get_yticklabels():
    l_1.set_fontweight('bold')

m_p_t_1.tight_layout()
s_a_n_1 = "".join(c_1 if c_1.isalnum() or c_1 in (' ', '_') else " for c_1 in a_n_2).rstrip()
s_a_n_1 = s_a_n_1.replace(' ', '_')
m_p_t_1.savefig(f"route_map_{s_a_n_1}.png", dpi=300)

def f_5_s():
    c_i_1 = list(range(1, 31))
    u_c_1 = set(c_i_1)
    r_s_3 = []
    t_d_2 = 0

    while u_c_1:
        c_r_1 = [D_NODE]
        c_l_1 = 0

```

```

c_p_1 = D_NODE

r_p_1 = True
while r_p_1:
    r_p_1 = False
    b_n_c_1 = None
    m_d_t_n_1 = float('inf')

    t_u_o_1 = []
    for c_i_2 in u_c_1:
        if c_l_1 + P_DATA[c_i_2]["demand"] <= V_CAPACITY:
            d_i_2 = f_1_e(c_p_1, c_i_2)
            if d_i_2 < m_d_t_n_1:
                m_d_t_n_1 = d_i_2
                b_n_c_1 = c_i_2

    if b_n_c_1 is not None:
        c_r_1.append(b_n_c_1)
        c_l_1 += P_DATA[b_n_c_1]["demand"]
        c_p_1 = b_n_c_1
        u_c_1.remove(b_n_c_1)
        r_p_1 = True

    if not u_c_1 and not r_p_1 :
        break

    if not r_p_1 and not c_r_1 == [D_NODE]:
        break

    if len(c_r_1) > 1:
        c_r_1.append(D_NODE)
        r_d_1 = f_2_c(c_r_1)
        r_s_3.append({"path": c_r_1, "load": c_l_1, "distance": r_d_1})
        t_d_2 += r_d_1
    elif not u_c_1 and len(c_r_1) == 1:
        break

    return r_s_3, t_d_2

def f_6_s():
    c_c_1 = n_p_1.array([P_DATA[i_3]["coords"] for i_3 in range(1, 31)])
    c_i_3 = list(range(1, 31))

```

```

t_d_3 = sum(P_DATA[i_4]["demand"] for i_4 in c_i_3)
n_c_1 = math.ceil(t_d_3 / V_CAPACITY)
if n_c_1 == 0 and t_d_3 > 0 : n_c_1 = 1
if not c_i_3: return [], 0

try:
    k_m_1 = KMeans(n_clusters=n_c_1, random_state=42, n_init='auto')
    l_s_1 = k_m_1.fit_predict(c_c_1)
except ValueError:
    if c_i_3:
        k_m_1 = KMeans(n_clusters=len(c_i_3), random_state=42, n_init='auto')
        l_s_1 = k_m_1.fit_predict(c_c_1)
    else:
        l_s_1 = []

c_s_1 = [[] for _ in range(n_c_1)]
for i_5, l_2 in enumerate(l_s_1):
    c_s_1[l_2].append(c_i_3[i_5])

a_r_1 = []
t_d_4 = 0

s_c_o_1 = set()

for c_p_i_1 in c_s_1:
    if not c_p_i_1:
        continue

    c_c_u_1 = set(p_i_2 for p_i_2 in c_p_i_1 if p_i_2 not in s_c_o_1)

    while c_c_u_1:
        c_r_2 = [D_NODE]
        c_l_2 = 0
        c_p_2 = D_NODE
        r_p_2 = True

        while r_p_2:
            r_p_2 = False
            b_n_c_2 = None
            m_d_t_n_2 = float('inf')

            for c_i_4 in sorted(list(c_c_u_1), key=lambda c_id: f_1_e(c_p_2, c_id)):

```

```

        if c_l_2 + P_DATA[c_i_4]["demand"] <= V_CAPACITY:
            d_i_3 = f_1_e(c_p_2, c_i_4)
            if d_i_3 < m_d_t_n_2:
                b_n_c_2 = c_i_4
                m_d_t_n_2 = d_i_3
                break

    if b_n_c_2:
        c_r_2.append(b_n_c_2)
        c_l_2 += P_DATA[b_n_c_2]["demand"]
        c_p_2 = b_n_c_2
        c_c_u_1.remove(b_n_c_2)
        s_c_o_1.add(b_n_c_2)
        r_p_2 = True

    if not c_c_u_1 and not r_p_2 : break
    if not r_p_2 : break

    if len(c_r_2) > 1:
        c_r_2.append(D_NODE)
        r_d_2 = f_2_c(c_r_2)
        a_r_1.append({"path": c_r_2, "load": c_l_2, "distance": r_d_2})
        t_d_4 += r_d_2
    elif not c_c_u_1 and len(c_r_2) == 1:
        break

    r_u_1 = set(c_i_3) - s_c_o_1
    if r_u_1:
        pass

    return a_r_1, t_d_4

def f_7_s():
    n_v_1 = 30

    m_g_1 = p_w_c_p_1.RoutingIndexManager(len(P_DATA), n_v_1, D_NODE)
    r_g_1 = p_w_c_p_1.RoutingModel(m_g_1)

    def d_c_1(f_i_1, t_i_1):
        f_n_1 = m_g_1.IndexToNode(f_i_1)
        t_n_1 = m_g_1.IndexToNode(t_i_1)
        return int(f_1_e(f_n_1, t_n_1) * 100)

```

```

t_c_i_1 = r_g_1.RegisterTransitCallback(d_c_1)
r_g_1.SetArcCostEvaluatorOfAllVehicles(t_c_i_1)

def d_d_c_1(f_i_2):
    f_n_2 = m_g_1.IndexToNode(f_i_2)
    return int(P_DATA[f_n_2]["demand"] * 10)

d_d_c_i_1 = r_g_1.RegisterUnaryTransitCallback(d_d_c_1)
r_g_1.AddDimensionWithVehicleCapacity(
    d_d_c_i_1,
    0,
    [int(V_CAPACITY * 10)] * n_v_1,
    True,
    'Capacity')

s_p_1 = p_w_c_p_1.DefaultRoutingSearchParameters()
s_p_1.first_solution_strategy = (
    r_e_p_1.FirstSolutionStrategy.PATH_CHEAPEST_ARC)
s_p_1.local_search_metaheuristic = (
    r_e_p_1.LocalSearchMetaheuristic.GUIDED_LOCAL_SEARCH)
s_p_1.time_limit.seconds = 10

s_l_1 = r_g_1.SolveWithParameters(s_p_1)

r_s_4 = []
t_d_5 = 0
if s_l_1:
    t_d_o_u_1 = s_l_1.ObjectiveValue()
    t_d_5 = t_d_o_u_1 / 100.0

    for v_i_1 in range(n_v_1):
        i_d_1 = r_g_1.Start(v_i_1)
        r_f_v_1 = []
        r_l_1 = 0
        r_d_o_u_1 = 0

        while not r_g_1.IsEnd(i_d_1):
            n_i_1 = m_g_1.IndexToNode(i_d_1)
            r_f_v_1.append(n_i_1)
            if n_i_1 != D_NODE:
                r_l_1 += P_DATA[n_i_1]["demand"]

```

```

        p_i_3 = i_d_1
        i_d_1 = s_l_1.Value(r_g_1.NextVar(i_d_1))
        if i_d_1 is not None :
            r_d_o_u_1 += r_g_1.GetArcCostForVehicle(p_i_3, i_d_1, v_i_1)

    e_n_i_1 = m_g_1.IndexToNode(i_d_1)
    r_f_v_1.append(e_n_i_1)

    if len(r_f_v_1) > 2:
        a_r_d_1 = f_2_c(r_f_v_1)
        r_s_4.append({"path": r_f_v_1, "load": r_l_1, "distance": a_r_d_1})
    t_d_5 = sum(r_1['distance'] for r_1 in r_s_4)

    return r_s_4, t_d_5

def f_8_s():
    c_s_2 = list(range(1, 31))
    if not c_s_2: return [], 0

    s_s_1 = []
    for i_i_2 in range(len(c_s_2)):
        for j_i_1 in range(i_i_2 + 1, len(c_s_2)):
            c_1_1 = c_s_2[i_i_2]
            c_2_1 = c_s_2[j_i_1]
            s_v_1 = f_1_e(D_NODE, c_1_1) + \
                    f_1_e(D_NODE, c_2_1) - \
                    f_1_e(c_1_1, c_2_1)
            if s_v_1 > 0:
                s_s_1.append({'c1': c_1_1, 'c2': c_2_1, 'save': s_v_1})

    s_s_1.sort(key=lambda x_2: x_2['save'], reverse=True)

    r_s_5 = []
    for c_2 in c_s_2:
        if P_DATA[c_2]['demand'] <= V_CAPACITY:
            r_t_2 = [D_NODE, c_2, D_NODE]
            l_d_1 = P_DATA[c_2]['demand']
            r_s_5.append({'path': r_t_2, 'load': l_d_1, 'id': c_2})
        else:
            pass

```

```

def f_r_f_c_1(c_i_5, c_r_s_1):
    for r_i_2, r_i_3 in enumerate(c_r_s_1):
        p_h_1 = r_i_3['path']
        if c_i_5 in p_h_1:
            if p_h_1[1] == c_i_5 and p_h_1[-2] != c_i_5:
                return r_i_2, 'start'
            if p_h_1[-2] == c_i_5 and p_h_1[1] != c_i_5:
                return r_i_2, 'end'
            if p_h_1[1] == c_i_5 and p_h_1[-2] == c_i_5:
                return r_i_2, 'single'
    return -1, None

for s_i_1 in s_s_1:
    c_1_2, c_2_2 = s_i_1['c1'], s_i_1['c2']

    r_1_i_1, c_1_p_1 = f_r_f_c_1(c_1_2, r_s_5)
    r_2_i_1, c_2_p_1 = f_r_f_c_1(c_2_2, r_s_5)

    if r_1_i_1 != -1 and r_2_i_1 != -1 and r_1_i_1 != r_2_i_1:
        r_1_1 = r_s_5[r_1_i_1]
        r_2_1 = r_s_5[r_2_i_1]

        m_l_1 = r_1_1['load'] + r_2_1['load']
        if m_l_1 <= V_CAPACITY:
            if r_1_1['path'][-2] == c_1_2 and r_2_1['path'][1] == c_2_2:
                n_p_1 = r_1_1['path'][:-1] + r_2_1['path'][1:]
                n_l_1 = m_l_1
                n_r_t_1 = []
                for i_6 in range(len(r_s_5)):
                    if i_6 != r_1_i_1 and i_6 != r_2_i_1:
                        n_r_t_1.append(r_s_5[i_6])
                n_r_t_1.append({'path': n_p_1, 'load': n_l_1, 'id': None})
                r_s_5 = n_r_t_1
            elif r_2_1['path'][-2] == c_2_2 and r_1_1['path'][1] == c_1_2:
                n_p_2 = r_2_1['path'][:-1] + r_1_1['path'][1:]
                n_l_2 = m_l_1
                n_r_t_2 = []
                for i_7 in range(len(r_s_5)):
                    if i_7 != r_1_i_1 and i_7 != r_2_i_1:
                        n_r_t_2.append(r_s_5[i_7])
                n_r_t_2.append({'path': n_p_2, 'load': n_l_2, 'id': None})
                r_s_5 = n_r_t_2

```

```
f_r_1 = []
t_d_6 = 0
for r_i_4 in r_s_5:
    d_i_4 = f_2_c(r_i_4['path'])
    f_r_1.append({'path': r_i_4['path'], 'load': r_i_4['load'], 'distance': d_i_4})
    t_d_6 += d_i_4

return f_r_1, t_d_6
```

```
class Ant:
```

```
def __init__(self, n_n_2, v_c_1, p_d_2, d_n_2):
    self.n_n_2 = n_n_2
    self.v_c_1 = v_c_1
    self.p_d_2 = p_d_2
    self.d_n_2 = d_n_2
    self.reset()

def reset(self):
    self.tour = []
    self.c_r_3 = [self.d_n_2]
    self.c_l_3 = 0
    self.u_n_1 = list(range(1, self.n_n_2))
    random.shuffle(self.u_n_1)
    self.t_d_7 = 0

def s_n_n_1(self, p_m_1, a_p_1, b_t_1):
    c_n_1 = self.c_r_3[-1]
    p_s_1 = []
    a_n_s_1 = []

    for n_i_2 in self.u_n_1:
        if self.c_l_3 + self.p_d_2[n_i_2]['demand'] <= self.v_c_1:
            a_n_s_1.append(n_i_2)

    if not a_n_s_1:
        return None

    t_p_1 = 0
    for n_i_3 in a_n_s_1:
        tau_1 = p_m_1[c_n_1][n_i_3]
        eta_1 = 1.0 / (f_1_e(c_n_1, n_i_3) + 1e-9)
```

```

        p_b_1 = (tau_1 ** a_p_1) * (eta_1 ** b_t_1)
        p_s_1.append(p_b_1)
        t_p_1 += p_b_1

    if t_p_1 == 0:
        return random.choice(a_n_s_1) if a_n_s_1 else None

    p_s_1 = [p_1 / t_p_1 for p_1 in p_s_1]
    c_n_i_a_1 = n_p_1.random.choice(len(a_n_s_1), p=p_s_1)
    return a_n_s_1[c_n_i_a_1]

def c_s_2(self, p_m_2, a_p_2, b_t_2):
    self.reset()
    while self.u_n_1:
        n_n_3 = self.s_n_n_1(p_m_2, a_p_2, b_t_2)
        if n_n_3 is not None:
            self.c_r_3.append(n_n_3)
            self.c_l_3 += self.p_d_2[n_n_3]['demand']
            self.u_n_1.remove(n_n_3)
        else:
            self.c_r_3.append(self.d_n_2)
            self.tour.append({'path': list(self.c_r_3), 'load': self.c_l_3, 'distance': 0})
            self.c_r_3 = [self.d_n_2]
            self.c_l_3 = 0
            if not self.u_n_1: break

    if len(self.c_r_3) > 1:
        self.c_r_3.append(self.d_n_2)
        self.tour.append({'path': list(self.c_r_3), 'load': self.c_l_3, 'distance': 0})

    self.t_d_7 = 0
    for r_i_5 in self.tour:
        d_i_5 = f_2_c(r_i_5['path'])
        r_i_5['distance'] = d_i_5
        self.t_d_7 += d_i_5
    return self.tour, self.t_d_7

def f_9_s(n_a_1=30, n_i_4=2000, a_p_3=1.0, b_t_3=2.0, e_r_1=0.5, q_0_1=0.1):
    n_n_4 = len(P_DATA)
    p_m_3 = n_p_1.ones((n_n_4, n_n_4)) * (1.0 / n_n_4)
    b_t_o_1 = []
    b_d_o_1 = float('inf')

```

```

a_s_2 = [Ant(n_n_4, V_CAPACITY, P_DATA, D_NODE) for _ in range(n_a_1)]

for i_t_1 in range(n_i_4):
    a_t_t_i_1 = []
    for a_t_1 in a_s_2:
        t_r_1, t_d_8 = a_t_1.c_s_2(p_m_3, a_p_3, b_t_3)
        a_t_t_i_1.append({'tour': t_r_1, 'distance': t_d_8})
        if t_d_8 < b_d_o_1:
            b_d_o_1 = t_d_8
            b_t_o_1 = copy.deepcopy(t_r_1)

    p_m_3 *= (1.0 - e_r_1)

    a_t_t_i_1.sort(key=lambda x_3: x_3['distance'])
    b_t_i_1 = a_t_t_i_1[0]['tour']
    b_d_i_1 = a_t_t_i_1[0]['distance']

    for r_i_6 in b_t_i_1:
        p_h_2 = r_i_6['path']
        for i_8 in range(len(p_h_2) - 1):
            p_m_3[p_h_2[i_8]][p_h_2[i_8+1]] += (q_0_1 / (b_d_i_1 + 1e-9))
            p_m_3[p_h_2[i_8+1]][p_h_2[i_8]] += (q_0_1 / (b_d_i_1 + 1e-9))

    return b_t_o_1, b_d_o_1

def f_10_p(r_s_6):
    a_n_s_2 = [r_2['name'] for r_2 in r_s_6]
    t_d_s_1 = [r_3['total_distance'] for r_3 in r_s_6]
    e_t_s_1 = [r_4['execution_time'] for r_4 in r_s_6]

    x_4 = n_p_1.arange(len(a_n_s_2))
    w_h_1 = 0.35

    f_g_2, a_x_2 = m_p_t_1.subplots(figsize=(14, 8))
    f_g_2.suptitle('Algo Perf Comp', fontsize=16, fontweight='bold')

    c_d_1 = 'tab:blue'
    a_x_2.set_xlabel('Algorithm', fontweight='bold')
    a_x_2.set_ylabel('Total Distance (km)', color=c_d_1, fontweight='bold')
    b_s_1 = a_x_2.bar(x_4 - w_h_1/2, t_d_s_1, w_h_1, label='Total Distance', color=c_d_1)
    a_x_2.tick_params(axis='y', labelcolor=c_d_1)

```

```

for l_4 in a_x_2.get_xticklabels() + a_x_2.get_yticklabels():
    l_4.set_fontweight('bold')
for l_5 in a_x_2.get_xticklabels():
    l_5.set_rotation(15)
    l_5.set_ha('right')

a_x_3 = a_x_2.twinx()
c_t_1 = 'tab:red'
a_x_3.set_ylabel('Solve Time (s)', color=c_t_1, fontweight='bold')
b_s_2 = a_x_3.bar(x_4 + w_h_1/2, e_t_s_1, w_h_1, label='Solve Time', color=c_t_1)
a_x_3.tick_params(axis='y', labelcolor=c_t_1)
for l_6 in a_x_3.get_yticklabels():
    l_6.set_fontweight('bold')

a_x_2.set_xticks(x_4)
a_x_2.set_xticklabels(a_n_s_2)

l_s_2, l_s_3 = a_x_2.get_legend_handles_labels()
l_s_4, l_s_5 = a_x_3.get_legend_handles_labels()
a_x_3.legend(l_s_2 + l_s_4, l_s_3 + l_s_5, loc='upper center', prop={'weight':'bold',
'size':'medium'})

for b_1 in b_s_1:
    y_v_1 = b_1.get_height()
    a_x_2.text(b_1.get_x() + b_1.get_width()/2.0, y_v_1 + 0.05 * max(t_d_s_1),
f'{y_v_1:.2f}', ha='center', va='bottom', fontweight='bold', fontsize='small')
    for b_2 in b_s_2:
        y_v_2 = b_2.get_height()
        a_x_3.text(b_2.get_x() + b_2.get_width()/2.0, y_v_2 + 0.05 * max(e_t_s_1),
f'{y_v_2:.4f}', ha='center', va='bottom', fontweight='bold', fontsize='small')

f_g_2.tight_layout(rect=[0, 0, 1, 0.96])
m_p_t_1.savefig("algo_perf_compare.png", dpi=300)

if __name__ == "__main__":
    r_s_7 = []

    print("Running Greedy Algo...")
    s_t_1 = time.time()
    g_r_1, g_t_d_1 = f_5_s()
    e_t_2 = time.time()
    g_t_1 = e_t_2 - s_t_1

```

```

r_s_7.append({
    "name": "Greedy (NN)",
    "routes": g_r_1,
    "total_distance": g_t_d_1,
    "execution_time": g_t_1,
    "num_vehicles": len(g_r_1)
})
print("Greedy Algo done.")

print("\nRunning Cluster + TSP Heuristic...")
s_t_2 = time.time()
c_r_s_2, c_t_d_1 = f_6_s()
e_t_3 = time.time()
c_t_2 = e_t_3 - s_t_2
r_s_7.append({
    "name": "Cluster+TSP (KMeans)",
    "routes": c_r_s_2,
    "total_distance": c_t_d_1,
    "execution_time": c_t_2,
    "num_vehicles": len(c_r_s_2)
})
print("Cluster + TSP Heuristic done.")

print("\nRunning OR-Tools Solver...")
s_t_3 = time.time()
o_r_1, o_t_d_1 = f_7_s()
e_t_4 = time.time()
o_t_1 = e_t_4 - s_t_3
r_s_7.append({
    "name": "OR-Tools (CVRP)",
    "routes": o_r_1,
    "total_distance": o_t_d_1,
    "execution_time": o_t_1,
    "num_vehicles": len(o_r_1)
})
print("OR-Tools Solver done.")

print("\nRunning Savings Algo (Clarke-Wright)...")
s_t_4 = time.time()
s_r_s_1, s_t_d_1 = f_8_s()
e_t_5 = time.time()
s_t_5 = e_t_5 - s_t_4

```

```

r_s_7.append({
    "name": "Savings (Clarke-Wright)",
    "routes": s_r_s_1,
    "total_distance": s_t_d_1,
    "execution_time": s_t_5,
    "num_vehicles": len(s_r_s_1)
})
print("Savings Algo done.")

print("\nRunning Ant Colony Opt (ACO)... (may take time)")
s_t_6 = time.time()
a_r_2, a_t_d_1 = f_9_s(num_ants=30, num_iterations=200, alpha=1.0, beta=3.0,
evaporation_rate=0.5, q0=1.0)
e_t_6 = time.time()
a_t_2 = e_t_6 - s_t_6
r_s_7.append({
    "name": "ACO",
    "routes": a_r_2,
    "total_distance": a_t_d_1,
    "execution_time": a_t_2,
    "num_vehicles": len(a_r_2)
})
print("ACO done.")

print("\n\n--- Algo Route Visualizations (Close plots one by one) ---")
for r_5 in r_s_7:
    print(f"\nGenerating route map for \"{r_5['name']}\"...")
    if r_5['routes']:
        f_4_p(r_5['name'], r_5['routes'], P_DATA, D_NODE)
    else:
        print(f"Algo {r_5['name']} did not generate routes for plotting.")

print("\n\n--- Problem 1: Single Vehicle Type Base Opt & Sched (n=30, Q=5t) ---")
print("--- Model Run Summary ---")
print("{:<40} | {:<18} | {:<10} | {:<18}".format(
    "Algo Name", "Total Dist (km)", "Num Veh", "Solve Time (s)"
))
print("-" * 95)
for r_6 in r_s_7:
    print("{:<40} | {:<18.2f} | {:<10} | {:<18.4f}".format(
        r_6['name'],
        r_6['total_distance'],

```

```

        r_6['num_vehicles'],
        r_6['execution_time']
    ))

    print("\n--- Algo Route Details ---")
    for r_7 in r_s_7:
        print(f"\n--- {r_7['name']} ---")
        print(f"Total Dist: {r_7['total_distance']:.2f} km, Num Veh: {r_7['num_vehicles']},
Solve Time: {r_7['execution_time']:.4f} s")
        if not r_7['routes']:
            print("    No valid routes generated.")
        else:
            for i_9, r_i_7 in enumerate(r_7['routes']):
                p_h_3 = r_i_7['path']
                l_d_2 = r_i_7.get('load', 0)
                d_i_6 = r_i_7['distance']
                print(f"    Veh {i_9+1}: Path: {' -> '.join(map(str, p_h_3))}, Load:
{l_d_2:.2f} t, Route Dist: {d_i_6:.2f} km")

    print("\nGenerating summary bar chart...")
    f_10_p(r_s_7)

```

问题二代码

```

import math
import time
import matplotlib.pyplot as p_1_t
import matplotlib.cm as c_1_m
from ortools.constraint_solver import routing_enums_pb2 as r_1_p
from ortools.constraint_solver import pywrapcp as p_2_p
import copy

```

```

try:
    p_1_t.rcParams['font.sans-serif'] = ['SimHei']
    p_1_t.rcParams['axes.unicode_minus'] = False
    p_1_t.rcParams['font.weight'] = 'bold'
    p_1_t.rcParams['xtick.labelsize'] = 'medium'
    p_1_t.rcParams['ytick.labelsize'] = 'medium'
except Exception as e_1_x:
    print(f"Font Warn: {e_1_x}.")

```

```

D_NODE = 0
U_SPEED = 40.0

```

D_MAX_HR = 12.0

C_SCALE = 100

M_SCALE = 1000

P_COORDS = {

0: {"coords": (0, 0), "label": "D0"},
1: {"coords": (12, 8), "label": "P1"},
2: {"coords": (5, 15), "label": "P2"},
3: {"coords": (20, 30), "label": "P3"},
4: {"coords": (25, 10), "label": "P4"},
5: {"coords": (35, 22), "label": "P5"},
6: {"coords": (18, 5), "label": "P6"},
7: {"coords": (30, 35), "label": "P7"},
8: {"coords": (10, 25), "label": "P8"},
9: {"coords": (22, 18), "label": "P9"},
10: {"coords": (38, 15), "label": "P10"},
11: {"coords": (5, 8), "label": "P11"},
12: {"coords": (15, 32), "label": "P12"},
13: {"coords": (28, 5), "label": "P13"},
14: {"coords": (30, 12), "label": "P14"},
15: {"coords": (10, 10), "label": "P15"},
16: {"coords": (20, 20), "label": "P16"},
17: {"coords": (35, 30), "label": "P17"},
18: {"coords": (8, 22), "label": "P18"},
19: {"coords": (25, 25), "label": "P19"},
20: {"coords": (32, 8), "label": "P20"},
21: {"coords": (15, 5), "label": "P21"},
22: {"coords": (28, 20), "label": "P22"},
23: {"coords": (38, 25), "label": "P23"},
24: {"coords": (10, 30), "label": "P24"},
25: {"coords": (20, 10), "label": "P25"},
26: {"coords": (30, 18), "label": "P26"},
27: {"coords": (5, 25), "label": "P27"},
28: {"coords": (18, 30), "label": "P28"},
29: {"coords": (35, 10), "label": "P29"},
30: {"coords": (22, 35), "label": "P30"}

}

V_DATA = {

1: {"name": "TypeA", "Qk_ton": 8, "Vk_m3": 20, "Ck_yuan_km": 2.5, "alpha_k": 0.8,
"beta_k": 0.3},

2: {"name": "TypeB", "Qk_ton": 6, "Vk_m3": 25, "Ck_yuan_km": 2.0, "alpha_k": 0.6,

```

"beta_k": 0.2},
    3: {"name": "TypeC", "Qk_ton": 3, "Vk_m3": 10, "Ck_yuan_km": 5.0, "alpha_k": 1.2,
"beta_k": 0.5},
    4: {"name": "TypeD", "Qk_ton": 10, "Vk_m3": 18, "Ck_yuan_km": 1.8, "alpha_k": 0.7,
"beta_k": 0.25}
}
for k_id, d_val in V_DATA.items():
    if d_val["Qk_ton"] > 0:
        d_val["specific_volume_m3_ton"] = d_val["Vk_m3"] / d_val["Qk_ton"]
    else:
        d_val["specific_volume_m3_ton"] = float('inf')

P_DEMANDS = {
    1: {1: 0.72, 2: 0.12, 3: 0.06, 4: 0.3},
    2: {1: 1.38, 2: 0.23, 3: 0.05, 4: 0.64},
    3: {1: 1.08, 2: 0.18, 3: 0.04, 4: 0.5},
    4: {1: 1.55, 2: 0.31, 3: 0.06, 4: 1.18},
    5: {1: 1.62, 2: 0.27, 3: 0.05, 4: 0.76},
    6: {1: 1.76, 2: 0.384, 3: 0.096, 4: 0.96},
    7: {1: 0.77, 2: 0.168, 3: 0.042, 4: 0.42},
    8: {1: 1.02, 2: 0.238, 3: 0.068, 4: 0.374},
    9: {1: 1.32, 2: 0.176, 3: 0.044, 4: 0.66},
    10: {1: 1.45, 2: 0.3, 3: 0.075, 4: 0.675},
    11: {1: 1.35, 2: 0.27, 3: 0.108, 4: 0.972},
    12: {1: 1.87, 2: 0.51, 3: 0.068, 4: 0.952},
    13: {1: 2.58, 2: 0.516, 3: 0.129, 4: 1.075},
    14: {1: 1.134, 2: 0.21, 3: 0.063, 4: 0.693},
    15: {1: 0.78, 2: 0.13, 3: 0.065, 4: 0.325},
    16: {1: 0.768, 2: 0.192, 3: 0.08, 4: 0.56},
    17: {1: 0.72, 2: 0.27, 3: 0.09, 4: 0.72},
    18: {1: 1.595, 2: 0.348, 3: 0.087, 4: 0.87},
    19: {1: 1.5, 2: 0.36, 3: 0.09, 4: 1.05},
    20: {1: 1.08, 2: 0.18, 3: 0.09, 4: 0.45},
    21: {1: 0.912, 2: 0.19, 3: 0.038, 4: 0.76},
    22: {1: 0.9, 2: 0.195, 3: 0.075, 4: 0.33},
    23: {1: 0.99, 2: 0.27, 3: 0.072, 4: 0.468},
    24: {1: 1.44, 2: 0.24, 3: 0.048, 4: 0.672},
    25: {1: 1.74, 2: 0.319, 3: 0.116, 4: 0.725},
    26: {1: 1.17, 2: 0.39, 3: 0.13, 4: 0.91},
    27: {1: 1.7, 2: 0.34, 3: 0.17, 4: 1.19},
    28: {1: 2.64, 2: 0.66, 3: 0.044, 4: 1.056},
    29: {1: 0.864, 2: 0.216, 3: 0.072, 4: 0.648},

```

```

    30: {1: 0.986, 2: 0.204, 3: 0.085, 4: 0.425}
}

def f_1_e(p1_id, p2_id):
    pt1 = P_COORDS[p1_id]["coords"]
    pt2 = P_COORDS[p2_id]["coords"]
    return math.hypot(pt1[0] - pt2[0], pt1[1] - pt2[1])

def f_2_m(r_nodes, t_k_id):
    d_val = 0
    for i_val in range(len(r_nodes) - 1):
        d_val += f_1_e(r_nodes[i_val], r_nodes[i_val+1])
    c_val = d_val * V_DATA[t_k_id]["Ck_yuan_km"]
    return d_val, c_val

def f_3_p(t_name, t_k_id, r_sol, t_suffix=""):
    f_fig, ax_plot = p_1_t.subplots(figsize=(12, 10))
    p_title = f"{t_name} (T{t_k_id}) Plan {t_suffix}"
    ax_plot.set_title(p_title, fontweight='bold')
    ax_plot.set_xlabel("X (km)", fontweight='bold')
    ax_plot.set_ylabel("Y (km)", fontweight='bold')

    for p_id, d_item in P_COORDS.items():
        x_coord, y_coord = d_item["coords"]
        is_on = any(p_id in r_info['path'] for r_info in r_sol)
        if p_id == D_NODE:
            ax_plot.plot(x_coord, y_coord, 'ks', markersize=12, label=f"Depot ({p_id})")
            ax_plot.text(x_coord + 0.5, y_coord + 0.5, str(p_id), color='black',
fontweight='bold')
        elif is_on:
            ax_plot.plot(x_coord, y_coord, 'o', markersize=6,
color=p_1_t.cm.tab10(t_k_id % 10))
            ax_plot.text(x_coord + 0.5, y_coord + 0.5, str(p_id),
color=p_1_t.cm.tab10(t_k_id % 10), fontweight='bold')

    r_colors = [p_1_t.cm.tab20(i_val) for i_val in range(20)]

    for i_val, r_info in enumerate(r_sol):
        p_nodes = r_info['path']
        if not p_nodes: continue
        p_x = [P_COORDS[node]["coords"][0] for node in p_nodes]
        p_y = [P_COORDS[node]["coords"][1] for node in p_nodes]

```

```

        r_clr = r_colors[i_val % len(r_colors)]
        lbl_txt = f"V {i_val+1} (W:{r_info['load_weight']:.2f}t,
Vol:{r_info['load_volume']:.2f}m3, C:{r_info['cost']:.2f}Y)"
        if 'time_hours' in r_info:
            lbl_txt += f", T:{r_info['time_hours']:.2f}h"
        ax_plot.plot(p_x, p_y, linestyle='-', marker='.', markersize=4, color=r_clr,
label=lbl_txt)

    ax_plot.grid(True)
    ax_plot.legend(loc='best', fontsize=14, prop={'weight':'bold', 'size':14})
    for lbl in ax_plot.get_xticklabels() + ax_plot.get_yticklabels():
        lbl.set_fontweight('bold')

    p_1_t.tight_layout()
    s_p_title = "".join(c_char if c_char.isalnum() or c_char in (' ', '_') else " " for c_char in
p_title).rstrip().replace(' ', '_')
    f_name = f"plot_{s_p_title}.png"
    p_1_t.savefig(f_name, dpi=300)
    print(f"Plot saved: {f_name}")
    p_1_t.close(f_fig)

def f_4_t(t_name, t_k_id, r_details, t_suffix=""):
    if not r_details:
        print(f"Type {t_name} (ID: {t_k_id}) no data, skip table.")
        return

    col_lbls = ["V_ID", "Path_Nodes", "Load_W_Ton", "Dist_KM"]
    cell_d = []
    for i_val, r_info in enumerate(r_details):
        v_num = f"V {i_val+1}"
        p_str = ' -> '.join(map(str, r_info['path']))
        l_str = f"{r_info['load_weight']:.2f}"
        d_str = f"{r_info['distance']:.2f}"
        cell_d.append([v_num, p_str, l_str, d_str])

    if not cell_d: return

    num_r = len(cell_d)
    f_h = max(3, 1.0 + num_r * 0.45)
    f_w = 12

```

```

f_fig, ax_plot = p_1_t.subplots(figsize=(f_w, f_h))
ax_plot.axis('off')

tbl_title = f"Table: {t_name} (T{t_k_id}) Plan {t_suffix}"
ax_plot.set_title(tbl_title, fontsize=16, fontweight='bold', pad=20)

col_w = [0.15, 0.50, 0.15, 0.20]

tbl = p_1_t.table(cellText=cell_d,
                  colLabels=col_lbls,
                  colWidths=col_w,
                  loc='center',
                  cellLoc='center')

tbl.auto_set_font_size(False)
tbl.set_fontsize(11)

for key_val, cell_val in tbl.get_celld().items():
    row_idx, col_idx = key_val
    cell_val.set_edgecolor('black')
    if row_idx == 0:
        cell_val.set_facecolor('white')
        cell_val.set_text_props(fontweight='bold', color='black')
    else:
        cell_val.set_facecolor('#E6F5E6')
        cell_val.set_text_props(fontweight='normal', color='black')

p_1_t.tight_layout(pad=1.0)

s_t_name = "".join(c_char if c_char.isalnum() else " " for c_char in t_name)
s_t_suffix = "_" + "".join(c_char if c_char.isalnum() else " " for c_char in
t_suffix.strip("(")")) if t_suffix else ""
f_name = f"table_{s_t_name}_T{t_k_id}{s_t_suffix}.png"

try:
    p_1_t.savefig(f_name, dpi=300, bbox_inches='tight')
    print(f"Table saved: {f_name}")
except Exception as e_1_x:
    print(f"Save table '{f_name}' failed: {e_1_x}")
finally:
    p_1_t.close(f_fig)

```

```

def f_5_s(
    inc_t_constr=False,
    max_v_type=15,
    ort_t_lim=10,
    cur_max_hr=None,
    gen_ind_plots=True
):
    g_total_c = 0
    all_sols = {}
    prob_sum = []

    run_desc = "TimeConstr: "
    if inc_t_constr:
        eff_hr = cur_max_hr if cur_max_hr is not None else D_MAX_HR
        run_desc += f"ON ({eff_hr:.1f} hr)"
    else:
        run_desc += "OFF"

    if not gen_ind_plots:
        print(f"\n--- Solving ({run_desc}, IndivPlots OFF) ---")

    for t_k_id, v_props in V_DATA.items():
        t_name = v_props["name"]

        act_pts_type = {D_NODE: P_COORDS[D_NODE]}
        p_ids_type = [D_NODE]
        orig_cust_ids = []
        for p_id, dems in P_DEMANDS.items():
            if dems.get(t_k_id, 0) > 0:
                act_pts_type[p_id] = P_COORDS[p_id]
                orig_cust_ids.append(p_id)

        if not orig_cust_ids:
            all_sols[t_k_id] = {"routes": [], "total_distance": 0, "total_cost": 0,
"num_vehicles":0, "name": t_name}
            prob_sum.append(f"Type {t_k_id} ({t_name}): No demand, cost 0.00")
            continue

        solv_n_to_orig = [D_NODE] + sorted(orig_cust_ids)
        num_solv_n = len(solv_n_to_orig)

        dist_mat = [[0] * num_solv_n for _ in range(num_solv_n)]

```

```

w_dems_solv = [0] * num_solv_n
v_dems_solv = [0] * num_solv_n

for i_val in range(num_solv_n):
    orig_i = solv_n_to_orig[i_val]
    if orig_i != D_NODE:
        w_dems_solv[i_val] = int(P_DEMANDS[orig_i].get(t_k_id, 0) *
M_SCALE)
        v_dems_solv[i_val] = int(P_DEMANDS[orig_i].get(t_k_id, 0) *
v_props["specific_volume_m3_ton"] * M_SCALE)
        for j_val in range(num_solv_n):
            orig_j = solv_n_to_orig[j_val]
            dist_mat[i_val][j_val] = f_1_e(orig_i, orig_j)

m_mgr = p_2_p.RoutingIndexManager(num_solv_n, max_v_type, 0)
r_mdl = p_2_p.RoutingModel(m_mgr)

def cb_1_c(fr_idx, to_idx):
    fr_n_solv = m_mgr.IndexToNode(fr_idx)
    to_n_solv = m_mgr.IndexToNode(to_idx)
    return int(dist_mat[fr_n_solv][to_n_solv] * v_props["Ck_yuan_km"] *
C_SCALE)
cb_1_transit = r_mdl.RegisterTransitCallback(cb_1_c)
r_mdl.SetArcCostEvaluatorOfAllVehicles(cb_1_transit)

def cb_2_w(fr_idx):
    fr_n_solv = m_mgr.IndexToNode(fr_idx)
    return w_dems_solv[fr_n_solv]
cb_2_w_idx = r_mdl.RegisterUnaryTransitCallback(cb_2_w)
r_mdl.AddDimensionWithVehicleCapacity(cb_2_w_idx, 0, [int(v_props["Qk_ton"]
* M_SCALE)] * max_v_type, True, "W")

def cb_3_v(fr_idx):
    fr_n_solv = m_mgr.IndexToNode(fr_idx)
    return v_dems_solv[fr_n_solv]
cb_3_v_idx = r_mdl.RegisterUnaryTransitCallback(cb_3_v)
r_mdl.AddDimensionWithVehicleCapacity(cb_3_v_idx, 0, [int(v_props["Vk_m3"]
* M_SCALE)] * max_v_type, True, "V")

if inc_t_constr:
    eff_max_hr_dim = cur_max_hr
    if eff_max_hr_dim is None:

```

```

    eff_max_hr_dim = D_MAX_HR

    def cb_4_t(fr_idx, to_idx):
        fr_n_solv = m_mgr.IndexToNode(fr_idx)
        to_n_solv = m_mgr.IndexToNode(to_idx)
        return int((dist_mat[fr_n_solv][to_n_solv] / U_SPEED) * 60)
    cb_4_t_idx = r_mdl.RegisterTransitCallback(cb_4_t)
    max_t_min_dim = int(eff_max_hr_dim * 60)
    r_mdl.AddDimension(cb_4_t_idx, 0, max_t_min_dim, True, "Time")

    s_params = p_2_p.DefaultRoutingSearchParameters()
    s_params.first_solution_strategy =
r_1_p.FirstSolutionStrategy.PATH_CHEAPEST_ARC
    s_params.local_search_metaheuristic =
r_1_p.LocalSearchMetaheuristic.GUIDED_LOCAL_SEARCH
    s_params.time_limit.seconds = ort_t_lim

    sol_obj = r_mdl.SolveWithParameters(s_params)

    r_s_type = []
    t_dist_type = 0
    v_used_type = 0

    if sol_obj:
        for v_id in range(max_v_type):
            p_nodes_solv = []
            idx_val = r_mdl.Start(v_id)
            while not r_mdl.IsEnd(idx_val):
                n_solv_idx = m_mgr.IndexToNode(idx_val)
                p_nodes_solv.append(n_solv_idx)
                idx_val = sol_obj.Value(r_mdl.NextVar(idx_val))
            p_nodes_solv.append(m_mgr.IndexToNode(idx_val))

            if len(p_nodes_solv) > 2:
                v_used_type += 1
                orig_p_nodes = [solv_n_to_orig[s_idx] for s_idx in p_nodes_solv]
                r_dist_km, r_cost_y = f_2_m(orig_p_nodes, t_k_id)
                t_dist_type += r_dist_km

                cur_r_w_sum_s = sum(w_dems_solv[s_idx] for s_idx in
p_nodes_solv if s_idx != 0)
                cur_r_v_sum_s = sum(v_dems_solv[s_idx] for s_idx in

```

p_nodes_solv if s_idx != 0)

```
    r_info = {
        "path": orig_p_nodes,
        "distance": r_dist_km,
        "cost": r_cost_y,
        "load_weight": cur_r_w_sum_s / M_SCALE,
        "load_volume": cur_r_v_sum_s / M_SCALE
    }
    if inc_t_constr:
        t_dim = r_mdl.GetDimensionOrDie("Time")
        end_idx_v_r = r_mdl.End(v_id)
        r_t_min_total = sol_obj.Value(t_dim.CumulVar(end_idx_v_r))
        r_info["time_hours"] = r_t_min_total / 60.0
```

```
    r_s_type.append(r_info)
```

```
cur_t_actual_total_c = sum(r_item['cost'] for r_item in r_s_type)
```

```
g_total_c += cur_t_actual_total_c
```

```
all_sols[t_k_id] = {
    "routes": r_s_type,
    "total_distance": t_dist_type,
    "total_cost": cur_t_actual_total_c,
    "num_vehicles": v_used_type,
    "name": t_name
}
```

```
if gen_ind_plots and r_s_type:
```

```
    t_suf_str = ""
```

```
    if inc_t_constr:
```

```
        eff_hr_title = cur_max_hr if cur_max_hr is not None else
```

D_MAX_HR

```
        t_suf_str = f"(TimeConstrON: {eff_hr_title:.1f}h)"
```

```
    else:
```

```
        t_suf_str = "(TimeConstrOFF)"
```

```
    f_3_p(t_name, t_k_id, r_s_type, t_suf_str)
```

```
if r_s_type:
```

```
    f_4_t(
```

```
        t_name=t_name,
```

```
        t_k_id=t_k_id,
```

```
        r_details=r_s_type,
```

```

        t_suffix="(NoTimeConstr)" if not inc_t_constr else
f'(TimeConstrON: {cur_max_hr if cur_max_hr else D_MAX_HR:.1f}h)"
    )

    print(f"\nType: {t_name} (ID: {t_k_id})")
    print(f"    Cost: {cur_t_actual_total_c:.2f} Y, Dist: {t_dist_type:.2f} km,
V_Num: {v_used_type}")
    if not r_s_type:
        print("    No routes.")
    else:
        for i_val, r_info in enumerate(r_s_type):
            p_str = ' -> '.join(map(str, r_info['path']))
            det_str = f"    V {i_val+1}: {p_str}"
            det_str += f" (D:{r_info['distance']:.2f}km, C:{r_info['cost']:.2f}Y"
            det_str += f", W:{r_info['load_weight']:.2f}t,
Vol:{r_info['load_volume']:.2f}m3)"
            print(det_str)
        else:
            all_sols[t_k_id] = {"routes": [], "total_distance": 0, "total_cost": 0,
"num_vehicles":0, "name": t_name}

    return g_total_c, all_sols

def f_6_c(hrs_test, tot_costs, type_c_dict, v_data_plot):
    f_fig, ax_plot = p_1_t.subplots(figsize=(14, 8))

    ax_plot.plot(hrs_test, tot_costs, marker='o', linestyle='-', linewidth=2, label='Total Cost')

    t_ids_sort = sorted(type_c_dict.keys())

    mrkrs = ['s', '^', 'D', '*', 'p', 'X']

    for i_val, t_k_id in enumerate(t_ids_sort):
        c_vals = type_c_dict[t_k_id]
        t_name = v_data_plot.get(t_k_id, {}).get('name', f'Type {t_k_id}')
        ax_plot.plot(hrs_test, c_vals, marker=mrkrs[i_val % len(mrkrs)], linestyle='--',
label=f'{t_name} Cost')

    ax_plot.set_xlabel("Max Travel Hr Constr (hr)", fontweight='bold', fontsize=18)
    ax_plot.set_ylabel("Transport Cost (Yuan)", fontweight='bold', fontsize=16)
    ax_plot.set_title("Cost Analysis vs Time Constr", fontweight='bold', fontsize=18)
    ax_plot.legend(prop={'weight':'bold', 'size':'x-large'})

```

```

ax_plot.grid(True, linestyle=':', alpha=0.7)
ax_plot.set_xticks(hrs_test)

for lbl in ax_plot.get_xticklabels() + ax_plot.get_yticklabels():
    lbl.set_fontweight('bold')
    lbl.set_fontsize(14)

p_1_t.tight_layout()
f_name = "cost_vs_time_analysis.png"
p_1_t.savefig(f_name, dpi=300)
print(f"\nAnalysis plot saved: {f_name}")
p_1_t.close(f_fig)

if __name__ == "__main__":
    print("Math Model (Prob2):")
    print("Obj: Min TotalCost = SUM(Dist * UnitCost)")
    print("Constr:")
    print("  1. Dedicated vehicles per type.")
    print("  2. All vehicles start/end at depot.")
    print("  3. All demands met.")
    print("  4. Max weight Qk.")
    print("  5. Max volume Vk.")
    print("  6. (Prob2.3) Max travel time (e.g., 12hr).")
    print("Model Ext: CVRP per type. Cost obj. Vol dim. Time dim (2.3).")
    print("Note: Specific volume estimated from Vk/Qk.\n")

    print("\n" + "="*50)
    print("Solving Base (No Time Constr)")
    print("="*50 + "\n")
    tc_no_t, sols_no_t = f_5_s(
        inc_t_constr=False,
        gen_ind_plots=True
    )
    print(f"\n--- Base (No Time Constr) Solved ---")
    print(f"Total Cost (No Time Constr): {tc_no_t:.2f} Yuan")
    if sols_no_t:
        print("\nDetailed Plan (No Time Constr):")
        for t_k_id_rep, sol_d_rep in sols_no_t.items():
            t_name = sol_d_rep.get('name', f"Type {t_k_id_rep}")

            if sol_d_rep.get('routes'):
                f_4_t(

```

```

        t_name=t_name,
        t_k_id=t_k_id_rep,
        r_details=sol_d_rep.get('routes', []),
        t_suffix="(NoTimeConstr)"
    )

    print(f"\nType: {t_name} (ID: {t_k_id_rep})")
    print(f"        Cost:    {sol_d_rep.get('total_cost', 0):.2f}    Y,    Dist:
{sol_d_rep.get('total_distance', 0):.2f} km, V_Num: {sol_d_rep.get('num_vehicles',0)}")
    if not sol_d_rep.get('routes'):
        print("        No routes.")
    else:
        for i_val, r_info in enumerate(sol_d_rep.get('routes', [])):
            p_str = ' -> '.join(map(str, r_info['path']))
            det_str = f"        V {i_val+1}: {p_str}"
            det_str += f" (D:{r_info['distance']:.2f}km, C:{r_info['cost']:.2f}Y"
            det_str += f"        f",        W:{r_info['load_weight']:.2f}t,
Vol:{r_info['load_volume']:.2f}m3)"
            print(det_str)

    else:
        print("No sol for no time constr.")

    print("\n" + "="*50)
    print("Starting Time Constr Sensitivity Analysis...")
    print("(Test max travel hr 2 to 10, step 1hr)")
    print("="*50 + "\n")

    hrs_list = list(range(2, 11, 1))

    an_tot_c = []
    an_type_c = {k_id: [] for k_id in V_DATA.keys()}

    for hr_lim in hrs_list:
        print(f"\n--- Analyzing: MaxTravelHr = {hr_lim} hr ---")

        cur_tot_c, cur_sols_type = f_5_s(
            inc_t_constr=True,
            cur_max_hr=float(hr_lim),
            gen_ind_plots=False
        )

        an_tot_c.append(cur_tot_c)

```

```

        for t_k_id in V_DATA.keys():
            c_for_type = cur_sols_type.get(t_k_id, {}).get('total_cost', 0)
            an_type_c[t_k_id].append(c_for_type)

        print(f"    HrLimit: {hr_lim}h, TotalCost: {cur_tot_c:.2f} Y")
        for t_k_id_rep, sol_d_rep in cur_sols_type.items():
            if sol_d_rep:
                print(f"        Type {t_k_id_rep} ({sol_d_rep.get('name', 'N/A')}) : Cost
{sol_d_rep.get('total_cost', 0):.2f} Y, V_Num {sol_d_rep.get('num_vehicles', 0)}")

    if hrs_list:
        f_6_c(
            hrs_list,
            an_tot_c,
            an_type_c,
            V_DATA
        )
    else:
        print("No time constr analysis, no plot.")

    print("\nScript done.")

```

问题三前两问代码

```

import numpy as n_p_1
from typing import List, Tuple, Dict
import random as r_a_1
from dataclasses import dataclass as d_c_1
import matplotlib.pyplot as m_p_t_1
from collections import defaultdict as d_d_1
import sys as s_y_1
import itertools as i_t_1

```

```

@d_c_1
class C_1_TS:
    id: int
    x: float
    y: float
    cost: float
    time_window: Tuple[int, int]
    storage_capacity: List[float]

```

```

@d_c_1

```

```

class C_2_V:
    type_id: int
    capacity: float
    volume: float
    cost_per_km: float
    carbon_per_km: float
    carbon_per_ton: float

@d_c_1
class C_3_R:
    vehicle_type: int
    points: List[int]
    load: float
    volume: float
    distance: float
    arrival_times: List[float]

class C_4_WCD:
    def __init__(self):
        self.collection_points = {
            'coords': n_p_1.array([
                [0, 0],
                [12, 8], [5, 15], [20, 30], [25, 10], [35, 22],
                [18, 5], [30, 35], [10, 25], [22, 18], [38, 15],
                [5, 8], [15, 32], [28, 5], [30, 12], [10, 10],
                [20, 20], [35, 30], [8, 22], [25, 25], [32, 8],
                [15, 5], [28, 20], [38, 25], [10, 30], [20, 10],
                [30, 18], [5, 25], [18, 30], [35, 10], [22, 35]
            ]),
            'waste_amounts': n_p_1.array([
                [0, 0, 0, 0],
                [0.72, 0.12, 0.06, 0.30], [1.38, 0.23, 0.05, 0.64],
                [1.08, 0.18, 0.04, 0.50], [1.55, 0.31, 0.06, 1.18],
                [1.62, 0.27, 0.05, 0.76], [1.76, 0.384, 0.096, 0.96],
                [0.77, 0.168, 0.042, 0.42], [1.02, 0.238, 0.068, 0.374],
                [1.32, 0.176, 0.044, 0.66], [1.45, 0.30, 0.075, 0.675],
                [1.35, 0.27, 0.108, 0.972], [1.87, 0.51, 0.068, 0.952],
                [2.58, 0.516, 0.129, 1.075], [1.134, 0.21, 0.063, 0.693],
                [0.78, 0.13, 0.065, 0.325], [0.768, 0.192, 0.08, 0.56],
                [0.72, 0.27, 0.09, 0.72], [1.595, 0.348, 0.087, 0.87],
                [1.5, 0.36, 0.09, 1.05], [1.08, 0.18, 0.09, 0.45],
                [0.912, 0.19, 0.038, 0.76], [0.9, 0.195, 0.075, 0.33],

```

```

        [0.99, 0.27, 0.072, 0.468], [1.44, 0.24, 0.048, 0.672],
        [1.74, 0.319, 0.116, 0.725], [1.17, 0.39, 0.13, 0.91],
        [1.7, 0.34, 0.17, 1.19], [2.64, 0.66, 0.044, 1.056],
        [0.864, 0.216, 0.072, 0.648], [0.986, 0.204, 0.085, 0.425]
    ])
}
self.vehicles = [
    C_2_V(1, 8, 20, 2.5, 0.8, 0.3),
    C_2_V(2, 6, 25, 2.0, 0.6, 0.2),
    C_2_V(3, 3, 10, 5.0, 1.2, 0.5),
    C_2_V(4, 10, 18, 1.8, 0.7, 0.25)
]
self.transfer_stations = [
    C_1_TS(31, 15, 15, 50, (6, 18), [20, 15, 5, 30]),
    C_1_TS(32, 25, 25, 60, (8, 16), [25, 20, 6, 35]),
    C_1_TS(33, 35, 15, 45, (10, 14), [18, 12, 4, 25]),
    C_1_TS(34, 10, 25, 55, (7, 17), [22, 18, 5, 32]),
    C_1_TS(35, 20, 30, 58, (9, 15), [24, 22, 7, 38])
]

def f_1_cd(self, p_1_1: int, p_1_2: int) -> float:
    if p_1_1 >= 31:
        s_1_1 = self.transfer_stations[p_1_1 - 31]
        c_1_1 = n_p_1.array([s_1_1.x, s_1_1.y])
    else:
        c_1_1 = self.collection_points['coords'][p_1_1]
    if p_1_2 >= 31:
        s_1_2 = self.transfer_stations[p_1_2 - 31]
        c_1_2 = n_p_1.array([s_1_2.x, s_1_2.y])
    else:
        c_1_2 = self.collection_points['coords'][p_1_2]
    return n_p_1.sqrt(n_p_1.sum((c_1_1 - c_1_2) ** 2))

class C_5_VRP:
    def __init__(self, d_1_d: C_4_WCD):
        self.data = d_1_d
        self.depot = 0
        self.vehicle_speed = 40.0
        self.service_time = 0.0
        self.start_time = 6.0

    def f_1_cd(self, p_1_1: int, p_1_2: int) -> float:

```

```

    return self.data.f_1_cd(p_1_1, p_1_2)

def f_2_cs(self, i_1_i: int, j_1_j: int, d_2_d: int) -> float:
    return (self.f_1_cd(d_2_d, i_1_i) +
            self.f_1_cd(d_2_d, j_1_j) -
            self.f_1_cd(i_1_i, j_1_j))

def f_3_cat(self, p_2_p: List[int], v_1_v: C_2_V) -> List[float]:
    t_1_t = [self.start_time]
    for i_2_i in range(1, len(p_2_p)):
        d_3_d = self.f_1_cd(p_2_p[i_2_i-1], p_2_p[i_2_i])
        t_2_t = d_3_d / self.vehicle_speed
        if p_2_p[i_2_i-1] > 0 and p_2_p[i_2_i-1] < 31:
            s_t_1 = self.service_time
        else:
            s_t_1 = 0
        n_t_1 = t_1_t[-1] + t_2_t + s_t_1
        if p_2_p[i_2_i] >= 31:
            s_1_3 = self.data.transfer_stations[p_2_p[i_2_i] - 31]
            if n_t_1 < s_1_3.time_window[0]:
                n_t_1 = s_1_3.time_window[0]
        t_1_t.append(n_t_1)
    return t_1_t

def _f_4_cpd(self, p_3_p: List[int]) -> float:
    if not p_3_p or len(p_3_p) < 2:
        return 0.0
    d_4_d = 0.0
    for i_3_i in range(len(p_3_p) - 1):
        d_4_d += self.f_1_cd(p_3_p[i_3_i], p_3_p[i_3_i+1])
    return d_4_d

def _f_5_a2opt(self, c_p_1: List[int], v_2_v: C_2_V,
               i_t_s_1: bool = False, s_n_i_a_1: int = -1) -> List[int]:
    if len(c_p_1) <= 3:
        return c_p_1
    b_p_1 = list(c_p_1)
    b_d_1 = self._f_4_cpd(b_p_1)
    i_1_imp = True
    s_i_2opt_1 = 1
    if i_t_s_1:
        e_i_2opt_1 = len(b_p_1) - 3

```

```

else:
    e_i_2opt_1 = len(b_p_1) - 2
    if s_i_2opt_1 >= e_i_2opt_1 or e_i_2opt_1 <= 0 :
        return b_p_1
    while i_1_imp:
        i_1_imp = False
        for i_4_i in range(s_i_2opt_1, e_i_2opt_1):
            for j_2_j in range(i_4_i + 1, e_i_2opt_1 + 1):
                if j_2_j - i_4_i < 1 :
                    continue
                n_p_2 = b_p_1[i_4_i+1] + b_p_1[i_4_i+1:j_2_j+1][::-1] +
b_p_1[j_2_j+1:]
                n_d_1 = self._f_4_cpd(n_p_2)
                if n_d_1 < b_d_1:
                    a_t_c_1 = self.f_3_cat(n_p_2, v_2_v)
                    f_t_1 = True
                    if i_t_s_1:
                        s_a_t_1 = a_t_c_1[len(n_p_2) - 2]
                        d_a_t_1 = a_t_c_1[len(n_p_2) - 1]
                        s_o_1 = self.data.transfer_stations[s_n_i_a_1 - 31]
                        if not (s_o_1.time_window[0] <= s_a_t_1 <=
s_o_1.time_window[1] and \
                                d_a_t_1 <= 18.0):
                            f_t_1 = False
                    else:
                        if a_t_c_1[-1] > 18.0:
                            f_t_1 = False
                    if f_t_1:
                        b_p_1 = list(n_p_2)
                        b_d_1 = n_d_1
                        i_1_imp = True
                        break
                if i_1_imp:
                    break
    return b_p_1

def _f_6_brfp(self, p_4_p: List[int], v_3_v: C_2_V, w_t_1: int) -> C_3_R:
    r_l_1 = sum(self.data.collection_points['waste_amounts'][p_5_p][w_t_1] for p_5_p
in p_4_p if 0 < p_5_p < 31)
    r_v_1 = r_l_1 * 1.2
    r_d_1 = self._f_4_cpd(p_4_p)
    a_t_2 = self.f_3_cat(p_4_p, v_3_v)

```

```

    return C_3_R(
        vehicle_type=w_t_1,
        points=p_4_p,
        load=r_l_1,
        volume=r_v_1,
        distance=r_d_1,
        arrival_times=a_t_2
    )

def _f_7_crc(self, r_p_1: List[int], v_4_v: C_2_V, w_t_2: int,
               i_t_s_2: bool = False, s_n_i_a_2: int = -1) -> bool:
    c_l_1 = sum(self.data.collection_points['waste_amounts'][p_6_p][w_t_2] for p_6_p
in r_p_1 if 0 < p_6_p < 31)
    c_v_1 = c_l_1 * 1.2
    if c_l_1 > v_4_v.capacity or c_v_1 > v_4_v.volume:
        return False
    a_t_3 = self.f_3_cat(r_p_1, v_4_v)
    if not a_t_3:
        return False
    f_a_a_d_1 = a_t_3[-1]
    if f_a_a_d_1 > 18.0:
        return False
    if i_t_s_2:
        if s_n_i_a_2 not in r_p_1:
            return False
        try:
            s_a_i_1 = r_p_1.index(s_n_i_a_2)
            s_a_t_2 = a_t_3[s_a_i_1]
            s_o_2 = self.data.transfer_stations[s_n_i_a_2 - 31]
            if not (s_o_2.time_window[0] <= s_a_t_2 <= s_o_2.time_window[1]):
                return False
        except ValueError:
            return False
    return True

def _f_8_asa(
    self,
    p_t_r_1: List[int],
    v_5_v: C_2_V,
    w_t_3: int,
    i_t_s_3: bool = False,
    s_n_i_a_3: int = -1

```

```

) -> List[C_3_R]:
    f_r_o_1: List[C_3_R] = []
    if not p_t_r_1:
        return []
    c_r_1: List[List[int]] = []
    for p_7_p in p_t_r_1:
        if i_t_s_3:
            if not (31 <= s_n_i_a_3 <= 35):
                print(f"Error: Invalid s_n_i_a_3 {s_n_i_a_3} for to-station savings
alg.")
                continue
            p_5_p = [self.depot, p_7_p, s_n_i_a_3, self.depot]
        else:
            p_5_p = [self.depot, p_7_p, self.depot]
        if self.f_7_crc(p_5_p, v_5_v, w_t_3, i_t_s_3, s_n_i_a_3):
            c_r_1.append(p_5_p)
        else:
            pass
    s_l_1: List[Tuple[float, int, int]] = []
    c_f_s_1 = s_n_i_a_3 if i_t_s_3 else self.depot
    for i_idx_1 in range(len(p_t_r_1)):
        for j_idx_1 in range(i_idx_1 + 1, len(p_t_r_1)):
            p_1_3 = p_t_r_1[i_idx_1]
            p_2_2 = p_t_r_1[j_idx_1]
            d_c_p_1_1 = self.f_1_cd(c_f_s_1, p_1_3)
            d_c_p_2_1 = self.f_1_cd(c_f_s_1, p_2_2)
            d_p_1_p_2_1 = self.f_1_cd(p_1_3, p_2_2)
            s_1_4 = d_c_p_1_1 + d_c_p_2_1 - d_p_1_p_2_1
            if s_1_4 > 0:
                s_l_1.append((s_1_4, p_1_3, p_2_2))
    s_l_1.sort(key=lambda x_1_x: x_1_x[0], reverse=True)
    if not i_t_s_3:
        print(f"Debug Savings (type {w_t_3}): Initial {len(c_r_1)} routes for
{len(p_t_r_1)} points. Example initial routes (max 3):")
        for r_idx_1, r_p_d_1 in enumerate(c_r_1[:3]):
            print(f"    Route {r_idx_1}: {r_p_d_1}")
        print(f"Debug Savings (type {w_t_3}): Top 5 savings (Total {len(s_l_1)}): ")
        for s_idx_1, s_v_1 in enumerate(s_l_1[:5]):
            print(f"    Saving {s_idx_1}: {s_v_1}")
    if not p_t_r_1 or not c_r_1:
        pass
    else:

```

```

r_i_m_1 = [False] * len(c_r_1)
n_c_r_t_p_1 = []
for s_2_s, p_1_4, p_2_3 in s_l_1:
    r_1_idx_1, r_2_idx_1 = -1, -1
    for r_idx_2, r_p_c_1 in enumerate(c_r_1):
        if r_i_m_1[r_idx_2]:
            continue
        p_c_1 = []
        if not i_t_s_3:
            p_c_1 = r_p_c_1[1:-1] if len(r_p_c_1) >= 3 else []
        else:
            p_c_1 = r_p_c_1[1:-2] if len(r_p_c_1) >= 4 else []
        if p_c_1 and p_c_1[-1] == p_1_4:
            r_1_idx_1 = r_idx_2
            break
    if r_1_idx_1 == -1: continue
    for r_idx_3, r_p_c_2 in enumerate(c_r_1):
        if r_i_m_1[r_idx_3]:
            continue
        if r_idx_3 == r_1_idx_1:
            continue
        p_c_2 = []
        if not i_t_s_3:
            p_c_2 = r_p_c_2[1:-1] if len(r_p_c_2) >= 3 else []
        else:
            p_c_2 = r_p_c_2[1:-2] if len(r_p_c_2) >= 4 else []
        if p_c_2 and p_c_2[0] == p_2_3:
            r_2_idx_1 = r_idx_3
            break
    if r_2_idx_1 == -1: continue
    p_1_o_1 = c_r_1[r_1_idx_1]
    p_2_o_1 = c_r_1[r_2_idx_1]
    p_1_c_s_1 = (p_1_o_1[1:-1] if not i_t_s_3 else p_1_o_1[1:-2])
    p_2_c_s_1 = (p_2_o_1[1:-1] if not i_t_s_3 else p_2_o_1[1:-2])
    if not (p_1_c_s_1 and p_1_c_s_1[-1] == p_1_4 and \
            p_2_c_s_1 and p_2_c_s_1[0] == p_2_3):
        continue
    t_m_c_1 = p_1_c_s_1 + p_2_c_s_1
    m_p_c_1 = ([self.depot] + t_m_c_1 + [s_n_i_a_3, self.depot]) if i_t_s_3 \
        else ([self.depot] + t_m_c_1 + [self.depot])
    if self.f_7_crc(m_p_c_1, v_5_v, w_t_3, i_t_s_3, s_n_i_a_3):
        n_c_r_t_p_1.append(m_p_c_1)

```

```

        r_i_m_1[r_1_idx_1] = True
        r_i_m_1[r_2_idx_1] = True
        if not i_t_s_3 and w_t_3 == 0 :
            print(f"Debug Savings Merge (type {w_t_3}, saving
{s_2_s:.2f}): Merged R{r_1_idx_1}{p_1_o_1} and R{r_2_idx_1}{p_2_o_1} into
{m_p_c_1}")
        f_r_a_m_p_1 = []
        for r_idx_4, r_p_2 in enumerate(c_r_1):
            if not r_i_m_1[r_idx_4]:
                f_r_a_m_p_1.append(r_p_2)
        f_r_a_m_p_1.extend(n_c_r_t_p_1)
        c_r_1 = f_r_a_m_p_1
        o_f_r_1: List[List[int]] = []
        for r_p_3 in c_r_1:
            o_p_1 = self._f_5_a2opt(r_p_3, v_5_v, i_t_s_3, s_n_i_a_3)
            o_f_r_1.append(o_p_1)
        if not i_t_s_3:
            print(f"Debug Savings (type {w_t_3}): Num routes after merge & 2-opt:
{len(o_f_r_1)}. Example final routes (max 3):")
            for r_idx_5, r_p_d_2 in enumerate(o_f_r_1[:3]):
                print(f"Route {r_idx_5}: {r_p_d_2}, Dist:
{self._f_4_cpd(r_p_d_2):.2f}")
            for o_p_2 in o_f_r_1:
                f_r_o_1.append(self._f_6_brfp(o_p_2, v_5_v, w_t_3))
        return f_r_o_1

def f_9_pr(self, w_t_4: int, a_1_a: Dict[int, int]) -> List[C_3_R]:
    v_6_v = self.data.vehicles[w_t_4]
    a_f_r_1: List[C_3_R] = []
    s_g_1 = d_d_1(list)
    d_p_l_1: List[int] = []
    s_l_2 = d_d_1(lambda: [0.0] * 4)
    for p_8_p, s_c_i_1 in a_1_a.items():
        w_a_1 = self.data.collection_points['waste_amounts'][p_8_p][w_t_4]
        if w_a_1 > 0:
            if s_c_i_1 > 0:
                s_a_i_2 = self.data.transfer_stations[s_c_i_1 - 1].id
                s_i_i_d_1 = s_c_i_1 - 1
                if s_l_2[s_i_i_d_1][w_t_4] + w_a_1 <= \
                    self.data.transfer_stations[s_i_i_d_1].storage_capacity[w_t_4]:
                    s_g_1[s_i_i_d_1].append(p_8_p)
                    s_l_2[s_i_i_d_1][w_t_4] += w_a_1

```

```

        else:
            d_p_l_1.append(p_8_p)
    else:
        d_p_l_1.append(p_8_p)
if d_p_l_1:
    d_r_1 = self._f_8_asa(
        points_to_route=d_p_l_1,
        vehicle=v_6_v,
        waste_type=w_t_4,
        is_to_station=False,
        station_node_if_any=-1
    )
    a_f_r_1.extend(d_r_1)
for s_i_i_d_2, p_f_s_1 in s_g_1.items():
    if not p_f_s_1:
        continue
    s_a_i_3 = self.data.transfer_stations[s_i_i_d_2].id
    s_r_1 = self._f_8_asa(
        points_to_route=p_f_s_1,
        vehicle=v_6_v,
        waste_type=w_t_4,
        is_to_station=True,
        station_node_if_any=s_a_i_3
    )
    a_f_r_1.extend(s_r_1)
return a_f_r_1

def f_10_gtd(self, r_2_r: List[C_3_R]) -> float:
    return sum(r_3_r.distance for r_3_r in r_2_r)

def f_11_gtc(self, r_4_r: List[C_3_R]) -> float:
    t_c_1 = 0
    for r_5_r in r_4_r:
        v_7_v = self.data.vehicles[r_5_r.vehicle_type]
        t_c_1 += r_5_r.distance * v_7_v.cost_per_km
    return t_c_1

def f_12_gtc(self, r_6_r: List[C_3_R]) -> float:
    t_c_2 = 0
    for r_7_r in r_6_r:
        v_8_v = self.data.vehicles[r_7_r.vehicle_type]
        t_c_2 += r_7_r.distance * v_8_v.carbon_per_km

```

```
t_c_2 += r_7_r.load * v_8_v.carbon_per_ton
return t_c_2
```

```
class C_6_NSGA2:
```

```
    def __init__(self, d_5_d: C_4_WCD, p_s_1: int = 200):
        self.data = d_5_d
        self.pop_size = p_s_1
        self.crossover_rate = 0.9
        self.mutation_rate = 0.1
        self.planner = C_5_VRP(d_5_d)
```

```
    def f_13_ip(self) -> List[n_p_1.ndarray]:
```

```
        p_6_p = []
        n_s_i_1 = 0
        if self.pop_size > n_s_i_1:
            c_d_1 = n_p_1.zeros(35, dtype=int)
            p_6_p.append(c_d_1)
            n_s_i_1 += 1
        if self.pop_size > n_s_i_1:
            c_s_31_1 = n_p_1.zeros(35, dtype=int)
            c_s_31_1[0] = 1
            for i_5_i in range(30):
                p_i_1 = i_5_i + 1
                if r_a_1.random() < 0.7:
                    s_o_3 = self.data.transfer_stations[0]
                    c_a_1 = True
                    for w_i_1 in range(4):
                        if self.data.collection_points['waste_amounts'][p_i_1][w_i_1] >
s_o_3.storage_capacity[w_i_1]:
                            c_a_1 = False
                            break
                    if c_a_1:
                        c_s_31_1[i_5_i+5] = 1
                    else:
                        c_s_31_1[i_5_i+5] = 0
            else:
                c_s_31_1[i_5_i+5] = 0
            p_6_p.append(c_s_31_1)
            n_s_i_1 += 1
        for _ in range(self.pop_size - n_s_i_1):
            c_1_c = n_p_1.zeros(35, dtype=int)
            n_s_t_o_1 = r_a_1.randint(1, 2)
```

```

s_i_t_o_1 = r_a_1.sample(range(5), n_s_t_o_1)
for i_6_i in s_i_t_o_1:
    c_1_c[i_6_i] = 1
o_s_i_1 = [i_7_i for i_7_i, b_1_b in enumerate(c_1_c[:5]) if b_1_b == 1]
if o_s_i_1:
    for i_8_i in range(30):
        p_i_i_c_1 = i_8_i + 5
        if r_a_1.random() < 0.6:
            b_s_c_i_1 = -1
            m_d_1 = float('inf')
            c_p_c_1 = self.data.collection_points['coords'][i_8_i+1]
            for s_c_i_2 in o_s_i_1:
                s_o_4 = self.data.transfer_stations[s_c_i_2]
                d_6_d = n_p_1.sqrt(n_p_1.sum((c_p_c_1 -
n_p_1.array([s_o_4.x, s_o_4.y])**2))
                if d_6_d < m_d_1:
                    s_c_t_1 = True
                    for w_t_i_1 in range(4):
                        if
self.data.collection_points['waste_amounts'][i_8_i+1][w_t_i_1] >
s_o_4.storage_capacity[w_t_i_1]:
                            s_c_t_1 = False
                            break
                    if s_c_t_1:
                        m_d_1 = d_6_d
                        b_s_c_i_1 = s_c_i_2
                    if b_s_c_i_1 != -1:
                        c_1_c[p_i_i_c_1] = b_s_c_i_1 + 1
                    else:
                        c_1_c[p_i_i_c_1] = 0
                else:
                    c_1_c[p_i_i_c_1] = 0
            else:
                c_1_c[p_i_i_c_1] = 0
        else:
            for i_9_i in range(30):
                c_1_c[i_9_i+5] = 0
    p_6_p.append(c_1_c)
return p_6_p

def f_14_co(self, c_2_c: n_p_1.ndarray) -> Tuple[float, float]:
    s_s_1 = []
    c_c_1 = 0
    for i_10_i, b_2_b in enumerate(c_2_c[:5]):

```

```

        if b_2_b == 1:
            s_s_1.append(self.data.transfer_stations[i_10_i])
            d_c_1 = self.data.transfer_stations[i_10_i].cost * 10000 / (10 * 365)
            c_c_1 += d_c_1
    a_2_a = {}
    for i_11_i, s_i_1 in enumerate(c_2_c[5:]):
        a_2_a[i_11_i+1] = int(s_i_1)
    for p_9_p, s_a_t_3 in a_2_a.items():
        if s_a_t_3 > 0:
            s_c_i_3 = s_a_t_3 - 1
            if c_2_c[s_c_i_3] == 0:
                return float('inf'), float('inf')
    p_1_p = C_5_VRP(self.data)
    a_r_1 = []
    t_t_c_1 = 0
    t_c_3 = 0
    w_t_c_1 = [0] * 4
    try:
        for w_t_5 in range(4):
            r_8_r = p_1_p.f_9_pr(w_t_5, a_2_a)
            if not r_8_r:
                return float('inf'), float('inf')
            a_r_1.extend(r_8_r)
            t_c_4 = 0
            for r_9_r in r_8_r:
                v_9_v = self.data.vehicles[w_t_5]
                d_c_2 = r_9_r.distance * v_9_v.cost_per_km
                t_c_4 += d_c_2
                t_c_3 += r_9_r.distance * v_9_v.carbon_per_km + r_9_r.load *
v_9_v.carbon_per_ton
            w_t_c_1[w_t_5] = t_c_4
            t_t_c_1 += t_c_4
        if not self._f_15_if(c_2_c, a_r_1):
            return float('inf'), float('inf')
        t_c_5 = c_c_1 + t_t_c_1
        return t_c_5, t_c_3
    except Exception as e_1_e:
        return float('inf'), float('inf')

def _f_15_if(self, c_3_c: n_p_1.ndarray, r_10_r: List[C_3_R]) -> bool:
    s_l_3 = d_d_1(lambda: [0, 0, 0, 0])
    for r_11_r in r_10_r:

```

```

        for p_10_p in r_11_r.points:
            if p_10_p >= 31:
                s_i_2 = p_10_p - 31
                s_l_3[s_i_2][r_11_r.vehicle_type] += r_11_r.load
                if s_l_3[s_i_2][r_11_r.vehicle_type] > \

self.data.transfer_stations[s_i_2].storage_capacity[r_11_r.vehicle_type]:
    return False

for r_12_r in r_10_r:
    if r_12_r.points[-1] == self.planner.depot:
        if r_12_r.arrival_times[-1] > 18.0:
            return False
    for i_12_i, p_11_p in enumerate(r_12_r.points):
        if p_11_p >= 31:
            s_1_5 = self.data.transfer_stations[p_11_p - 31]
            a_t_4 = r_12_r.arrival_times[i_12_i]
            if not (s_1_5.time_window[0] <= a_t_4 <= s_1_5.time_window[1]):
                return False
    for r_13_r in r_10_r:
        v_10_v = self.data.vehicles[r_13_r.vehicle_type]
        if r_13_r.load > v_10_v.capacity:
            return False
        if r_13_r.volume > v_10_v.volume:
            return False
    return True

def f_16_nds(self, p_7_p: List[n_p_1.ndarray]) -> List[List[int]]:
    n_1_n = len(p_7_p)
    if n_1_n == 0:
        return []
    d_c_3 = n_p_1.zeros(n_1_n)
    d_s_1 = [[] for _ in range(n_1_n)]
    f_1_f = [[]]
    u_1_u = set(range(n_1_n))
    o_1_o = [self.f_14_co(c_4_c) for c_4_c in p_7_p]
    for i_13_i in range(n_1_n):
        for j_3_j in range(i_13_i + 1, n_1_n):
            if o_1_o[i_13_i][0] <= o_1_o[j_3_j][0] and o_1_o[i_13_i][1] <=
o_1_o[j_3_j][1]:
                if o_1_o[i_13_i][0] < o_1_o[j_3_j][0] or o_1_o[i_13_i][1] <
o_1_o[j_3_j][1]:
                    d_s_1[i_13_i].append(j_3_j)

```

```

        d_c_3[j_3_j] += 1
        elif o_1_o[j_3_j][0] <= o_1_o[i_13_i][0] and o_1_o[j_3_j][1] <=
o_1_o[i_13_i][1]:
            if o_1_o[j_3_j][0] < o_1_o[i_13_i][0] or o_1_o[j_3_j][1] <
o_1_o[i_13_i][1]:
                d_s_1[j_3_j].append(i_13_i)
                d_c_3[i_13_i] += 1
    while u_1_u:
        c_f_1 = []
        for i_14_i in u_1_u:
            if d_c_3[i_14_i] == 0:
                c_f_1.append(i_14_i)
        if not c_f_1:
            break
        f_1_f.append(c_f_1)
        for i_15_i in c_f_1:
            u_1_u.remove(i_15_i)
            for j_4_j in d_s_1[i_15_i]:
                d_c_3[j_4_j] -= 1
    if not f_1_f[0]:
        f_1_f.pop(0)
    return f_1_f

def f_17_cdi(self, f_2_f: List[int], o_2_o: List[Tuple[float, float]]) -> List[float]:
    if len(f_2_f) <= 2:
        return [float('inf')] * len(f_2_f)
    d_7_d = [0.0] * len(f_2_f)
    f_s_1 = len(f_2_f)
    for o_i_1 in range(2):
        o_v_1 = [(i_16_i, o_2_o[f_2_f[i_16_i]][o_i_1]) for i_16_i in range(f_s_1)]
        o_v_1.sort(key=lambda x_2_x: x_2_x[1])
        d_7_d[o_v_1[0][0]] = float('inf')
        d_7_d[o_v_1[-1][0]] = float('inf')
        o_r_1 = o_2_o[f_2_f[o_v_1[-1][0]]][o_i_1] - \
            o_2_o[f_2_f[o_v_1[0][0]]][o_i_1]
        if o_r_1 == 0:
            continue
        for i_17_i in range(1, f_s_1 - 1):
            d_7_d[o_v_1[i_17_i][0]] += (o_v_1[i_17_i+1][1] - o_v_1[i_17_i-1][1]) /
o_r_1
    return d_7_d

```

```

def f_18_cc(self, i_1_1: int, i_2_1: int, f_3_f: List[List[int]],
              c_d_2: List[float]) -> bool:
    r_1_1 = next(i_18_i for i_18_i, f_4_f in enumerate(f_3_f) if i_1_1 in f_4_f)
    r_2_1 = next(i_19_i for i_19_i, f_5_f in enumerate(f_3_f) if i_2_1 in f_5_f)
    if r_1_1 != r_2_1:
        return r_1_1 < r_2_1
    return c_d_2[i_1_1] > c_d_2[i_2_1]

def f_19_s(self, p_8_p: List[n_p_1.ndarray], f_6_f: List[List[int]],
            c_d_3: Dict[int, float]) -> List[n_p_1.ndarray]:
    s_3_s = []
    i_20_i = 0
    while len(s_3_s) < self.pop_size:
        if len(s_3_s) + len(f_6_f[i_20_i]) <= self.pop_size:
            s_3_s.extend([p_8_p[i_7_i] for i_7_i in f_6_f[i_20_i]])
        else:
            l_f_1 = f_6_f[i_20_i]
            l_f_d_1 = [(i_7_i, c_d_3[i_7_i]) for i_7_i in l_f_1]
            l_f_d_1.sort(key=lambda x_3_x: x_3_x[1], reverse=True)
            r_3_r = self.pop_size - len(s_3_s)
            s_3_s.extend([p_8_p[i_7_i] for i_7_i, _ in l_f_d_1[:r_3_r]])
            i_20_i += 1
    return s_3_s

def f_20_c(self, p_1_5: n_p_1.ndarray, p_2_4: n_p_1.ndarray) -> Tuple[n_p_1.ndarray,
n_p_1.ndarray]:
    c_1_2 = p_1_5.copy()
    c_2_2 = p_2_4.copy()
    if r_a_1.random() < self.crossover_rate:
        c_p_2 = r_a_1.randint(1, 4)
        c_1_2[:c_p_2] = p_2_4[:c_p_2]
        c_2_2[:c_p_2] = p_1_5[:c_p_2]
        for i_21_i in range(5, len(p_1_5)):
            if r_a_1.random() < 0.5:
                c_1_2[i_21_i], c_2_2[i_21_i] = c_2_2[i_21_i], c_1_2[i_21_i]
    return c_1_2, c_2_2

def f_21_m(self, c_5_c: n_p_1.ndarray) -> n_p_1.ndarray:
    m_1_m = c_5_c.copy()
    if r_a_1.random() < 0.05:
        m_2_m = r_a_1.choice(["all_closed", "all_open", "single_best_31",
"single_best_34"])

```

```

    if m_2_m == "all_closed":
        m_1_m[:5] = 0
        m_1_m[5:] = 0
    elif m_2_m == "all_open":
        m_1_m[:5] = 1
        for i_22_i in range(30):
            if r_a_1.random() < 0.8:
                m_1_m[i_22_i+5] = r_a_1.randint(1,5)
            else:
                m_1_m[i_22_i+5] = 0
    elif m_2_m == "single_best_31":
        m_1_m[:5] = 0
        m_1_m[0] = 1
        for i_23_i in range(30):
            m_1_m[i_23_i+5] = 1 if r_a_1.random() < 0.7 else 0
    elif m_2_m == "single_best_34":
        m_1_m[:5] = 0
        m_1_m[3] = 1
        for i_24_i in range(30):
            m_1_m[i_24_i+5] = 4 if r_a_1.random() < 0.7 else 0
    return m_1_m
if r_a_1.random() < self.mutation_rate:
    c_o_i_1 = [i_25_i for i_25_i, b_3_b in enumerate(m_1_m[:5]) if b_3_b == 1]
    n_c_o_1 = len(c_o_i_1)
    a_3_a = r_a_1.choice(["flip_one", "ensure_one_if_all_closed"])
    if a_3_a == "flip_one":
        m_p_s_1 = r_a_1.randint(0, 4)
        m_1_m[m_p_s_1] = 1 - m_1_m[m_p_s_1]
    if n_p_1.sum(m_1_m[:5]) == 0:
        m_1_m[r_a_1.randint(0,4)] = 1
o_s_i_2 = [i_26_i+1 for i_26_i, b_4_b in enumerate(m_1_m[:5]) if b_4_b == 1]
for i_27_i in range(5, 35):
    if r_a_1.random() < self.mutation_rate:
        c_a_2 = m_1_m[i_27_i]
        p_a_i_1 = i_27_i - 4
        if not o_s_i_2:
            m_1_m[i_27_i] = 0
            continue
        c_3_c = [0]
        c_3_c.extend(o_s_i_2)
        n_a_1 = c_a_2
        while n_a_1 == c_a_2 and len(c_3_c) > 1:

```

```

        n_a_1 = r_a_1.choice(c_3_c)
        m_1_m[i_27_i] = n_a_1
    return m_1_m

def f_22_o(self, g_1_g: int = 100) -> List[Tuple[float, float]]:
    p_9_p = self.f_13_ip()
    for g_2_g in range(g_1_g):
        o_3_o = [self.f_14_co(c_6_c) for c_6_c in p_9_p]
        f_7_f = self.f_16_nds(p_9_p)
        c_d_4 = {}
        for f_8_f in f_7_f:
            f_d_1 = self.f_17_cdi(f_8_f, o_3_o)
            for i_7_i, d_8_d in zip(f_8_f, f_d_1):
                c_d_4[i_7_i] = d_8_d
        s_4_s = self.f_19_s(p_9_p, f_7_f, c_d_4)
        o_4_o = []
        while len(o_4_o) < self.pop_size:
            p_1_6 = r_a_1.choice(s_4_s)
            p_2_5 = r_a_1.choice(s_4_s)
            c_1_3, c_2_3 = self.f_20_c(p_1_6, p_2_5)
            c_1_3 = self.f_21_m(c_1_3)
            c_2_3 = self.f_21_m(c_2_3)
            o_4_o.extend([c_1_3, c_2_3])
        p_9_p = o_4_o[:self.pop_size]
        if g_2_g % 10 == 0:
            if f_7_f and f_7_f[0]:
                b_s_i_1 = f_7_f[0]
                print(f"Gen {g_2_g}, Non-dominated sols: {len(b_s_i_1)}")
                print("Best sols sample (max 3):")
                for i_28_i, i_7_i in enumerate(b_s_i_1[:3]):
                    c_7_c, c_8_c = o_3_o[i_7_i]
                    print(f"    Cost: {c_7_c:.2f}, Carbon: {c_8_c:.2f}kg")
            else:
                print(f"Gen {g_2_g}, No non-dominated sol found.")
        self.population = p_9_p
        f_o_1 = [self.f_14_co(c_7_c) for c_7_c in self.population]
        f_f_1 = self.f_16_nds(self.population)
        self.final_fronts = f_f_1
        if f_f_1 and f_f_1[0]:
            return [f_o_1[i_7_i] for i_7_i in f_f_1[0]]
        else:
            print("Warning: Opt ended, Pareto front not determined. Returning empty

```

```

list.")
        return []

def f_23_ppf(s_5_s: List[Tuple[float, float]], t_3_t: str = "Pareto Front"):
    c_4_c = [s_6_s[0] for s_6_s in s_5_s]
    e_1_e = [s_7_s[1] for s_7_s in s_5_s]
    m_p_t_1.figure(figsize=(10, 6))
    m_p_t_1.scatter(c_4_c, e_1_e, c='b', marker='o')
    m_p_t_1.xlabel("Total Cost (Yuan)")
    m_p_t_1.ylabel("Total Carbon (kg)")
    m_p_t_1.title(t_3_t)
    m_p_t_1.grid(True)
    m_p_t_1.show()

def f_24_ds(c_8_c: n_p_1.ndarray, d_9_d: C_4_WCD) -> str:
    s_s_2 = c_8_c[:5]
    p_a_2 = c_8_c[5:]
    r_4_r = []
    r_4_r.append("Selected Transfer Stations:")
    for i_29_i, s_6_s in enumerate(s_s_2):
        if s_6_s:
            s_1_6 = d_9_d.transfer_stations[i_29_i]
            r_4_r.append(f"TS {s_1_6.id} (Loc: {s_1_6.x}, {s_1_6.y})")
    r_4_r.append("\nCollection Point Assignments:")
    for i_30_i, a_4_a in enumerate(p_a_2):
        if a_4_a == 0:
            r_4_r.append(f"CP {i_30_i+1} -> Direct to Depot")
        else:
            s_i_3 = d_9_d.transfer_stations[a_4_a-1].id
            r_4_r.append(f"CP {i_30_i+1} -> TS {s_i_3}")
    return "\n".join(r_4_r)

def f_25_asc(d_10_d: C_4_WCD):
    r_5_r = []
    print("\nTesting 0 TS (all direct)...")
    c_z_s_1 = n_p_1.zeros(35, dtype=int)
    o_t_n_1 = C_6_NSQA2(d_10_d, pop_size=1)
    c_z_1, ca_z_1 = o_t_n_1.f_14_co(c_z_s_1)
    if c_z_1 != float('inf'):
        r_5_r.append({
            'num_stations': 0,
            'station_ids': [],

```

```

        'total_cost': c_z_1,
        'carbon_emissions': ca_z_1
    })
    print(f"0 TS: Cost: {c_z_1:.2f}, Carbon: {ca_z_1:.2f}kg")
else:
    print("0 TS (all direct) plan infeasible or cost is inf.")
for n_s_1 in range(1, 6):
    a_s_i_1 = list(range(5))
    for o_s_i_t_1 in i_t_1.combinations(a_s_i_1, n_s_1):
        o_s_i_3 = list(o_s_i_t_1)
        c_b_1 = n_p_1.zeros(35, dtype=int)
        for s_i_i_c_1 in o_s_i_3:
            c_b_1[s_i_i_c_1] = 1
        c_s_i_f_c_1 = [d_10_d.transfer_stations[i_7_i].id for i_7_i in o_s_i_3]
        print(f"\nTesting TS combo: {c_s_i_f_c_1} (Num: {n_s_1})")
        c_f_a_1 = c_b_1.copy()
        for c_p_i_1 in range(1, 31):
            b_a_m_1 = d_10_d.f_1_cd(c_p_i_1, 0)
            a_f_i_f_c_1 = 0
            for s_c_i_4 in o_s_i_3:
                s_o_5 = d_10_d.transfer_stations[s_c_i_4]
                d_c_p_t_s_1 = d_10_d.f_1_cd(c_p_i_1, s_o_5.id)
                d_s_t_d_1 = d_10_d.f_1_cd(s_o_5.id, 0)
                m_v_s_1 = d_c_p_t_s_1 + d_s_t_d_1
                if m_v_s_1 < b_a_m_1:
                    b_a_m_1 = m_v_s_1
                    a_f_i_f_c_1 = s_c_i_4 + 1
            c_f_a_1[c_p_i_1 + 4] = a_f_i_f_c_1
        c_5_c, ca_1_c = o_t_n_1.f_14_co(c_f_a_1)
        if c_5_c != float('inf'):
            r_5_r.append({
                'num_stations': n_s_1,
                'station_ids': c_s_i_f_c_1,
                'total_cost': c_5_c,
                'carbon_emissions': ca_1_c
            })
        print(f"    Combo {c_s_i_f_c_1}: Cost: {c_5_c:.2f}, Carbon: {ca_1_c:.2f}kg (based on total travel proxy dist allocation)")
    else:
        print(f"    Combo {c_s_i_f_c_1} no feasible plan found (based on total travel proxy dist allocation).")
    r_5_r.sort(key=lambda x_4_x: x_4_x['total_cost'])

```

```

return r_5_r

if __name__ == "__main__":
    r_a_1.seed(42)
    n_p_1.random.seed(42)
    d_11_d = C_4_WCD()
    print("Starting analysis of different TS combos...")
    r_c_1 = f_25_asc(d_11_d)
    print("\nTS combo analysis complete. Result summary:")
    if r_c_1:
        s_7_s = {}
        for r_c_2 in r_c_1:
            if r_c_2['total_cost'] != float('inf'):
                n_s_2 = r_c_2['num_stations']
                if n_s_2 not in s_7_s or r_c_2['total_cost'] < s_7_s[n_s_2]['cost']:
                    s_7_s[n_s_2] = {'cost': r_c_2['total_cost'], 'carbon':
r_c_2['carbon_emissions'], 'ids': r_c_2['station_ids']}
                for n_2_n, d_s_2 in sorted(s_7_s.items()):
                    print(f"Best plan for {n_2_n} open TS: Cost {d_s_2['cost']:.2f}, Carbon
{d_s_2['carbon']:.2f}, TS: {d_s_2['ids']}")
            else:
                print("TS combo analysis yielded no valid results.")
        print("\nStarting NSGA-II opt...")
        o_5_o = C_6_NSQA2(d_11_d, pop_size=100)
        g_3_g = 50
        p_s_2 = o_5_o.f_22_o(generations=g_3_g)
        if p_s_2:
            print(f"\nNSGA-II opt complete. Found {len(p_s_2)} Pareto optimal sols.")
            print("Pareto optimal sols sample (max 5):")
            for i_31_i, s_8_s in enumerate(p_s_2[:5]):
                print(f"    Plan {i_31_i+1}: Total Cost: {s_8_s[0]:.2f}, Total Carbon:
{s_8_s[1]:.2f}kg")
            if o_5_o.population and o_5_o.final_fronts and o_5_o.final_fronts[0]:
                try:
                    f_s_c_1 = o_5_o.population[o_5_o.final_fronts[0][0]]
                    print("\nDetailed decoded plan for first Pareto optimal sol:")
                    print(f_24_ds(f_s_c_1, d_11_d))
                except IndexError:
                    print("\nCannot decode details of first Pareto sol (index error).")
                except AttributeError:
                    print("\nCannot decode details of first Pareto sol (attribute error).")
            f_23_ppf(p_s_2)

```

```
else:
    print("NSGA-II found no Pareto optimal sols.")
```

问题三第三问代码

```
import math
from ortools.constraint_solver import routing_enums_pb2 as r_1_p, pywrapcp as p_1_p
from deap import base as b_1_s, creator as c_1_r, tools as t_1_s
import random
from functools import lru_cache as l_1_c
import statistics
import numpy as n_1_p
import matplotlib.pyplot as p_2_t
from matplotlib.font_manager import FontProperties as F_1_p
import matplotlib.patches as p_3_s
import traceback
import sys
import matplotlib.lines as m_1_s

c_1_r.create("FitnessMulti", b_1_s.Fitness, weights=(-1.0, -1.0))
c_1_r.create("Individual", list, fitness=c_1_r.FitnessMulti)

p_2_t.rcParams['font.sans-serif'] = ['SimHei']
p_2_t.rcParams['axes.unicode_minus'] = False

G_TYPE_COLORS = {
    1: 'saddlebrown',
    2: 'deepskyblue',
    3: 'orangered',
    4: 'dimgray'
}

P_NODE_ID = 0
V_SPEED = 40
T_UNIT_SCALER = 100
S_TIME_UNITS = 0
N_G_TYPES = 4

P_PLANT_COORDS = {'x': 0, 'y': 0}
C_POINTS_COORDS = [
    {'id': 1, 'x': 5, 'y': 5}, {'id': 2, 'x': 8, 'y': 12}, {'id': 3, 'x': 12, 'y': 3}, {'id': 4, 'x': 15, 'y': 18},
    {'id': 5, 'x': 18, 'y': 7}, {'id': 6, 'x': 22, 'y': 15}, {'id': 7, 'x': 25, 'y': 5}, {'id': 8, 'x': 28, 'y':
```

```

19},
    {'id': 9, 'x': 32, 'y': 2}, {'id': 10, 'x': 35, 'y': 10}, {'id': 11, 'x': 2, 'y': 20}, {'id': 12, 'x': 7, 'y':
25},
    {'id': 13, 'x': 11, 'y': 28}, {'id': 14, 'x': 17, 'y': 22}, {'id': 15, 'x': 20, 'y': 30}, {'id': 16, 'x':
23, 'y': 24},
    {'id': 17, 'x': 27, 'y': 32}, {'id': 18, 'x': 30, 'y': 21}, {'id': 19, 'x': 33, 'y': 28}, {'id': 20, 'x':
38, 'y': 35},
    {'id': 21, 'x': 40, 'y': 3}, {'id': 22, 'x': 3, 'y': 38}, {'id': 23, 'x': 9, 'y': 33}, {'id': 24, 'x': 13,
'y': 40},
    {'id': 25, 'x': 19, 'y': 37}, {'id': 26, 'x': 24, 'y': 42}, {'id': 27, 'x': 29, 'y': 36}, {'id': 28, 'x':
34, 'y': 40},
    {'id': 29, 'x': 37, 'y': 25}, {'id': 30, 'x': 42, 'y': 15}
]
G_AT_C_POINTS = {
    1: [0.72, 0.12, 0.06, 0.30], 2: [1.38, 0.23, 0.05, 0.64], 3: [1.08, 0.18, 0.04, 0.50],
    4: [1.55, 0.31, 0.06, 1.18], 5: [1.62, 0.27, 0.05, 0.76], 6: [1.76, 0.384, 0.096, 0.96],
    7: [0.77, 0.168, 0.042, 0.42], 8: [1.02, 0.238, 0.068, 0.374], 9: [1.32, 0.176, 0.044, 0.66],
    10: [1.45, 0.30, 0.075, 0.675], 11: [1.35, 0.27, 0.108, 0.972], 12: [1.87, 0.51, 0.068,
0.952],
    13: [2.58, 0.516, 0.129, 1.075], 14: [1.134, 0.21, 0.063, 0.693], 15: [0.78, 0.13, 0.065,
0.325],
    16: [0.768, 0.192, 0.08, 0.56], 17: [0.72, 0.27, 0.09, 0.72], 18: [1.595, 0.348, 0.087,
0.87],
    19: [1.5, 0.36, 0.09, 1.05], 20: [1.08, 0.18, 0.09, 0.45], 21: [0.912, 0.19, 0.038,
0.76],
    22: [0.9, 0.195, 0.075, 0.33], 23: [0.99, 0.27, 0.072, 0.468], 24: [1.44, 0.24, 0.048, 0.672],
    25: [1.74, 0.319, 0.116, 0.725], 26: [1.17, 0.39, 0.13, 0.91], 27: [1.7, 0.34, 0.17, 1.19],
    28: [2.64, 0.66, 0.044, 1.056], 29: [0.864, 0.216, 0.072, 0.648], 30: [0.986, 0.204, 0.085,
0.425]
}
T_STATIONS_DATA = {
    31: {'x': 15, 'y': 15, 'cost': 500000, 'tw': (6, 18), 'capacities': [20, 15, 5, 30]},
    32: {'x': 25, 'y': 25, 'cost': 600000, 'tw': (8, 16), 'capacities': [25, 20, 6, 35]},
    33: {'x': 35, 'y': 15, 'cost': 450000, 'tw': (10, 14), 'capacities': [18, 12, 4, 25]},
    34: {'x': 10, 'y': 25, 'cost': 550000, 'tw': (7, 17), 'capacities': [22, 18, 5, 32]},
    35: {'x': 20, 'y': 30, 'cost': 580000, 'tw': (9, 15), 'capacities': [24, 22, 7, 38]},
}
V_PARAMS = {
    1: {'Q': 8, 'V': 20, 'C': 2.5, 'alpha': 0.8, 'beta': 0.3, 'name': 'GT_A'},
    2: {'Q': 6, 'V': 25, 'C': 2.0, 'alpha': 0.6, 'beta': 0.2, 'name': 'GT_B'},
    3: {'Q': 3, 'V': 10, 'C': 5.0, 'alpha': 1.2, 'beta': 0.5, 'name': 'GT_C'},
    4: {'Q': 10, 'V': 18, 'C': 1.8, 'alpha': 0.7, 'beta': 0.25, 'name': 'GT_D'},

```

```

}
P_OP_WINDOW = (6, 18)

for k_id, d_val in V_PARAMS.items():
    if d_val["Q"] > 0:
        d_val["specific_volume_m3_ton"] = d_val["V"] / d_val["Q"]
    else:
        d_val["specific_volume_m3_ton"] = float('inf')

N_TS_CANDIDATES = len(T_STATIONS_DATA)
TS_IDS_LIST = sorted(list(T_STATIONS_DATA.keys()))

def f_1_e(c1_dict, c2_dict):
    return math.sqrt((c1_dict['x'] - c2_dict['x'])**2 + (c1_dict['y'] - c2_dict['y'])**2)

A_NODES_COORDS = {P_NODE_ID: P_PLANT_COORDS}
for cp_d in C_POINTS_COORDS:
    A_NODES_COORDS[cp_d['id']] = {'x': cp_d['x'], 'y': cp_d['y']}
for ts_id, ts_d_val in T_STATIONS_DATA.items():
    A_NODES_COORDS[ts_id] = {'x': ts_d_val['x'], 'y': ts_d_val['y']}

G_DIST_MATRIX = {}
A_SYS_NODE_IDS = sorted(list(A_NODES_COORDS.keys()))
for n1_id in A_SYS_NODE_IDS:
    G_DIST_MATRIX[n1_id] = {}
    for n2_id in A_SYS_NODE_IDS:
        if n1_id == n2_id:
            G_DIST_MATRIX[n1_id][n2_id] = 0
        else:
            dist_val = f_1_e(A_NODES_COORDS[n1_id], A_NODES_COORDS[n2_id])
            G_DIST_MATRIX[n1_id][n2_id] = round(dist_val)

if 4 in G_DIST_MATRIX and 31 in G_DIST_MATRIX[4]:
    G_DIST_MATRIX[4][31] = 18
if 31 in G_DIST_MATRIX and 4 in G_DIST_MATRIX[31]:
    G_DIST_MATRIX[31][4] = 15
if 27 in G_DIST_MATRIX and 28 in G_DIST_MATRIX[27]:
    G_DIST_MATRIX[27][28] = 14
if 28 in G_DIST_MATRIX and 27 in G_DIST_MATRIX[28]:
    G_DIST_MATRIX[28][27] = 18
if 23 in G_DIST_MATRIX and P_NODE_ID in G_DIST_MATRIX[23]:
    G_DIST_MATRIX[23][P_NODE_ID] = 45

```

```

if P_NODE_ID in G_DIST_MATRIX and 23 in G_DIST_MATRIX[P_NODE_ID]:
    G_DIST_MATRIX[P_NODE_ID][23] = 40
if 9 in G_DIST_MATRIX and 16 in G_DIST_MATRIX[9]:
    G_DIST_MATRIX[9][16] = 8
if 16 in G_DIST_MATRIX and 9 in G_DIST_MATRIX[16]:
    G_DIST_MATRIX[16][9] = 10

def f_2_s(d_node_id, c_node_ids, d_at_c_tons,
          v_cap_tons, v_cap_vol_m3,
          v_cost_km, v_spec_vol,
          n_time_windows, n_v_avail,
          g_dist_mat,
          s_time_val=S_TIME_UNITS,
          sp_kmh=V_SPEED,
          t_scaler=T_UNIT_SCALER):
    if not c_node_ids: return float('inf'), float('inf'), []

    loc_n_id_list = [d_node_id] + c_node_ids
    m_mgr = p_1_p.RoutingIndexManager(len(loc_n_id_list), n_v_avail, 0)
    r_mdl = p_1_p.RoutingModel(m_mgr)

    d_scale = 100
    def cb_1_d(fr_idx, to_idx):
        fr_n_loc_idx = m_mgr.IndexToNode(fr_idx)
        to_n_loc_idx = m_mgr.IndexToNode(to_idx)
        g_fr_id = loc_n_id_list[fr_n_loc_idx]
        g_to_id = loc_n_id_list[to_n_loc_idx]
        return int(g_dist_mat[g_fr_id][g_to_id] * d_scale)
    cb_1_transit = r_mdl.RegisterTransitCallback(cb_1_d)
    r_mdl.SetArcCostEvaluatorOfAllVehicles(cb_1_transit)

    dem_scale = 100
    sc_dems_w = [0] + [int(d_val * dem_scale) for d_val in d_at_c_tons]
    def cb_2_dem_w(fr_idx):
        fr_n_loc_idx = m_mgr.IndexToNode(fr_idx)
        return sc_dems_w[fr_n_loc_idx]
    cb_2_dem_w_idx = r_mdl.RegisterUnaryTransitCallback(cb_2_dem_w)
    r_mdl.AddDimension(cb_2_dem_w_idx, 0, int(v_cap_tons * dem_scale), True, 'CapW')

    sc_dems_v = [0] + [int(d_tons * v_spec_vol * dem_scale) for d_tons in d_at_c_tons]
    def cb_3_dem_v(fr_idx):
        fr_n_loc_idx = m_mgr.IndexToNode(fr_idx)

```

```

        return sc_dems_v[fr_n_loc_idx]
    cb_3_dem_v_idx = r_mdl.RegisterUnaryTransitCallback(cb_3_dem_v)
    r_mdl.AddDimension(cb_3_dem_v_idx, 0, int(v_cap_vol_m3 * dem_scale), True,
'CapV')

    if n_time_windows:
        def cb_4_t(fr_idx, to_idx):
            fr_n_loc_idx = m_mgr.IndexToNode(fr_idx)
            to_n_loc_idx = m_mgr.IndexToNode(to_idx)
            g_fr_id = loc_n_id_list[fr_n_loc_idx]
            g_to_id = loc_n_id_list[to_n_loc_idx]
            trav_d_km = g_dist_mat[g_fr_id][g_to_id]
            trav_t_units = int((trav_d_km / sp_kmh) * t_scaler)
            cur_s_time = s_time_val if fr_n_loc_idx != 0 else 0
            return trav_t_units + cur_s_time
        cb_4_t_transit = r_mdl.RegisterTransitCallback(cb_4_t)

        large_horiz = 24 * t_scaler
        r_mdl.AddDimension(cb_4_t_transit,
                           large_horiz,
                           large_horiz,
                           False,
                           'Time')
        t_dim = r_mdl.GetDimensionOrDie('Time')

        if d_node_id in n_time_windows:
            d_tw_start, d_tw_end = n_time_windows[d_node_id]
            t_dim.CumulVar(m_mgr.NodeToIndex(0)).SetRange(int(d_tw_start),
int(d_tw_end))
            for i_val, c_id in enumerate(c_node_ids):
                if c_id in n_time_windows:
                    c_tw_start, c_tw_end = n_time_windows[c_id]
                    t_dim.CumulVar(m_mgr.NodeToIndex(i_val
+
1)).SetRange(int(c_tw_start), int(c_tw_end))

            s_params = p_1_p.DefaultRoutingSearchParameters()
            s_params.first_solution_strategy =
(r_1_p.FirstSolutionStrategy.PATH_CHEAPEST_ARC)
            s_params.local_search_metaheuristic =
(r_1_p.LocalSearchMetaheuristic.GUIDED_LOCAL_SEARCH)
            s_params.time_limit.FromSeconds(10)

```

```

sol_obj = r_mdl.SolveWithParameters(s_params)

fin_tot_dist_km = float('inf')
fin_tot_cost = float('inf')
fin_routes_list = []

if sol_obj:
    calc_dist_km_sol = 0.0
    tmp_routes_list_sol = []

    for v_id in range(n_v_avail):
        idx_val = r_mdl.Start(v_id)
        r_dist_sc_val = 0
        r_nodes_g_ids_list_v = []

        while not r_mdl.IsEnd(idx_val):
            n_idx_loc = m_mgr.IndexToNode(idx_val)
            r_nodes_g_ids_list_v.append(loc_n_id_list[n_idx_loc])
            prev_idx = idx_val
            idx_val = sol_obj.Value(r_mdl.NextVar(idx_val))
            r_dist_sc_val += r_mdl.GetArcCostForVehicle(prev_idx, idx_val, v_id)

        n_idx_loc_end = m_mgr.IndexToNode(idx_val)
        r_nodes_g_ids_list_v.append(loc_n_id_list[n_idx_loc_end])

        if len(r_nodes_g_ids_list_v) > 2:
            tmp_routes_list_sol.append(r_nodes_g_ids_list_v)
            calc_dist_km_sol += (r_dist_sc_val / float(d_scale))

    fin_tot_dist_km = calc_dist_km_sol
    fin_tot_cost = fin_tot_dist_km * v_cost_km
    fin_routes_list = tmp_routes_list_sol

    cur_g_type_id_dbg = None
    for type_id_loop, params_loop in V_PARAMS.items():
        if params_loop['C'] == v_cost_km:
            cur_g_type_id_dbg = type_id_loop
            break

    if cur_g_type_id_dbg == 2:
        g_type_name_dbg = V_PARAMS.get(2, {}).get('name', 'TypeB_Unknown')

```

```

        print(f"\nDBG_START:   ORT   VRP   for   '{g_type_name_dbg}'   (TID:
{cur_g_type_id_dbg}) in 3.3.py")
        print(f"   Func: f_2_s")
        print(f"   DepotID: {d_node_id}")
        print(f"   NumCust: {len(c_node_ids)}")
        print(f"   VCap(T): {v_cap_tons}, VCap(Vol m3): {v_cap_vol_m3}")
        print(f"   ORT SolFound: {'Yes' if sol_obj else 'No'}")
        print(f"   NumRoutesFinal: {len(fin_routes_list)}")

    if not fin_routes_list:
        if sol_obj:
            print("   ORT sol found, but 0 routes constructed.")
        else:
            print("   No ORT sol found, 0 routes.")
    elif fin_routes_list:
        for i_val, r_path_nodes in enumerate(fin_routes_list):
            print(f"   Route {i_val + 1} (NIDs): {r_path_nodes}")

    dist_prn = f"{fin_tot_dist_km:.2f}" if fin_tot_dist_km != float('inf') else "Inf"
    cost_prn = f"{fin_tot_cost:.2f}" if fin_tot_cost != float('inf') else "Inf"
    print(f"   VRP Res - Dist: {dist_prn} km, Cost: {cost_prn} Yuan")
    print("DBG_END\n")

    return fin_tot_dist_km, fin_tot_cost, fin_routes_list

@l_1_c(maxsize=2000)
def f_3_c(d_id, c_tuple, dems_tuple, v_cap_w, v_cap_v,
          v_cost, v_s_vol, n_tw_dict_tuple_items):
    c_list = list(c_tuple); dems_list = list(dems_tuple)
    n_tw_vrp = dict(n_tw_dict_tuple_items)
    return f_2_s(d_id, c_list, dems_list,
                 v_cap_w, v_cap_v,
                 v_cost, v_s_vol,
                 n_tw_vrp,
                 len(c_list),
                 G_DIST_MATRIX)

def f_4_e(ind_list):
    open_s_ids = []
    cur_constr_cost = 0
    for i_val, is_opn in enumerate(ind_list):
        if is_opn:

```

```

        s_id = TS_IDS_LIST[i_val]
        open_s_ids.append(s_id)
        cur_constr_cost += T_STATIONS_DATA[s_id]['cost'] / 3650.0

    tot_trans_cost_acc = 0
    tot_emis_acc = 0
    tot_dist_acc = 0.0
    cp_allocs = {cp_id: {} for cp_id in G_AT_C_POINTS}
    tmp_ts_loads_tons = {ts_id: [0.0] * N_G_TYPES for ts_id in open_s_ids}

    aprx_l2_costs_ton_ts_type = {}
    for ts_id_calc in open_s_ids:
        aprx_l2_costs_ton_ts_type[ts_id_calc] = {}
        for g_type_id_calc in range(1, N_G_TYPES + 1):
            v_s_params = V_PARAMS[g_type_id_calc]
            cost_p_km_val = v_s_params['C']
            q_v_val = v_s_params['Q']
            v_v_val = v_s_params['V']
            s_vol_val = v_s_params['specific_volume_m3_ton']

            dist_ts_pl_l2_est = G_DIST_MATRIX[ts_id_calc].get(P_NODE_ID,
float('inf'))
            dist_pl_ts_l2_est = G_DIST_MATRIX[P_NODE_ID].get(ts_id_calc,
float('inf'))

            if dist_ts_pl_l2_est == float('inf') or dist_pl_ts_l2_est == float('inf'):
                aprx_l2_costs_ton_ts_type[ts_id_calc][g_type_id_calc] = float('inf')
                continue

            rnd_trip_dist_l2_est = dist_ts_pl_l2_est + dist_pl_ts_l2_est
            cost_p_dir_trip = rnd_trip_dist_l2_est * cost_p_km_val

            eff_tons_p_trip = 0
            if q_v_val > 0:
                tons_by_q = q_v_val
                if v_v_val > 0 and s_vol_val > 0 and s_vol_val != float('inf'):
                    tons_by_v = v_v_val / s_vol_val
                    eff_tons_p_trip = min(tons_by_q, tons_by_v)
                else:
                    eff_tons_p_trip = tons_by_q

            if eff_tons_p_trip > 0:

```

```

        aprx_l2_costs_ton_ts_type[ts_id_calc][g_type_id_calc] = cost_p_dir_trip
/ eff_tons_p_trip
    else:
        aprx_l2_costs_ton_ts_type[ts_id_calc][g_type_id_calc] = float('inf')

for cp_id, g_amts_type in G_AT_C_POINTS.items():
    for g_type_idx, g_amt_tons in enumerate(g_amts_type):
        act_g_type_id = g_type_idx + 1
        if g_amt_tons == 0: continue

        v_spec_cp_type = V_PARAMS[act_g_type_id]
        v_ck_l1 = v_spec_cp_type['C']

        dist_to_pl = G_DIST_MATRIX[cp_id][P_NODE_ID]
        cost_dir_pl_l1_aprx = dist_to_pl * v_ck_l1

        best_fac_id_cp = P_NODE_ID
        min_est_path_cost = cost_dir_pl_l1_aprx

    for ts_id_opt in open_s_ids:
        ts_s_data = T_STATIONS_DATA[ts_id_opt]
        ts_max_cap_type = ts_s_data['capacities'][g_type_idx]
        cur_load_ts_type = tmp_ts_loads_tons[ts_id_opt][g_type_idx]

        if cur_load_ts_type + g_amt_tons <= ts_max_cap_type:
            dist_to_ts_val = G_DIST_MATRIX[cp_id][ts_id_opt]
            cost_l1_ts_aprx = dist_to_ts_val * v_ck_l1

            est_add_l2_cost = 0
            avg_l2_cost_ton =
aprx_l2_costs_ton_ts_type[ts_id_opt].get(act_g_type_id, float('inf'))

            if avg_l2_cost_ton != float('inf'):
                est_add_l2_cost = g_amt_tons * avg_l2_cost_ton
            else:
                est_add_l2_cost = float('inf')

            cur_ts_path_tot_cost_aprx = cost_l1_ts_aprx + est_add_l2_cost

            if cur_ts_path_tot_cost_aprx < min_est_path_cost:
                min_est_path_cost = cur_ts_path_tot_cost_aprx
                best_fac_id_cp = ts_id_opt

```

```

        cp_allocs[cp_id][act_g_type_id] = best_fac_id_cp
        if best_fac_id_cp != P_NODE_ID:
            tmp_ts_loads_tons[best_fac_id_cp][g_type_idx] += g_amt_tons

for fac_n_id_l1 in [P_NODE_ID] + open_s_ids:
    cur_fac_tw_raw = P_OP_WINDOW
    if fac_n_id_l1 != P_NODE_ID:
        cur_fac_tw_raw = T_STATIONS_DATA[fac_n_id_l1]['tw']

    fac_tw_start_sc = cur_fac_tw_raw[0] * T_UNIT_SCALER
    fac_tw_end_sc = cur_fac_tw_raw[1] * T_UNIT_SCALER

    n_tw_fac_vrp = {
        fac_n_id_l1: (fac_tw_start_sc, fac_tw_end_sc)
    }

    for g_type_id_l1 in range(1, N_G_TYPES + 1):
        v_spec_l1 = V_PARAMS[g_type_id_l1]
        c_for_vrp_l1 = []
        dems_for_vrp_l1 = []

        cur_vrp_n_tw = n_tw_fac_vrp.copy()

        for cp_id_l1, allocs_cp_item in cp_allocs.items():
            if allocs_cp_item.get(g_type_id_l1) == fac_n_id_l1:
                dem_this_type_cp = G_AT_C_POINTS[cp_id_l1][g_type_id_l1-1]
                if dem_this_type_cp > 0:
                    c_for_vrp_l1.append(cp_id_l1)
                    dems_for_vrp_l1.append(dem_this_type_cp)
                    cur_vrp_n_tw[cp_id_l1] = (fac_tw_start_sc, fac_tw_end_sc)

        if not c_for_vrp_l1: continue

        n_tw_tuple_items_l1 = tuple(sorted(cur_vrp_n_tw.items()))

        dist_km_l1_vrp, cost_l1_val, routes_l1_list = f_3_c(
            d_id=fac_n_id_l1,
            c_tuple=tuple(sorted(c_for_vrp_l1)),
            dems_tuple=tuple(dems_for_vrp_l1),
            v_cap_w=v_spec_l1['Q'],
            v_cap_v=v_spec_l1['V'],

```

```

        v_cost=v_spec_l1['C'],
        v_s_vol=v_spec_l1['specific_volume_m3_ton'],
        n_tw_dict_tuple_items=n_tw_tuple_items_l1
    )

    if dist_km_l1_vrp == float('inf') or cost_l1_val == float('inf'):
        return float('inf'), float('inf'), float('inf')

    tot_trans_cost_acc += cost_l1_val
    tot_dist_acc += dist_km_l1_vrp
    load_on_l1_r = sum(dems_for_vrp_l1)
    tot_emis_acc += (dist_km_l1_vrp * v_spec_l1['alpha']) + \
        (load_on_l1_r * v_spec_l1['beta'])

l2_det_routes = {}
for ts_id_l2 in open_s_ids:
    for g_type_idx_l2, tot_load_ts in enumerate(tmp_ts_loads_tons[ts_id_l2]):
        act_g_type_id_l2 = g_type_idx_l2 + 1
        if tot_load_ts == 0: continue

        v_spec_l2 = V_PARAMS[act_g_type_id_l2]

        dist_ts_pl_l2 = G_DIST_MATRIX[ts_id_l2].get(P_NODE_ID, float('inf'))
        dist_pl_ts_l2 = G_DIST_MATRIX[P_NODE_ID].get(ts_id_l2, float('inf'))

        if dist_ts_pl_l2 == float('inf') or dist_pl_ts_l2 == float('inf'):
            fin_tot_cost = float('inf')
            fin_tot_emis = float('inf')
            tot_dist_acc = float('inf')
            return fin_tot_cost, fin_tot_emis, tot_dist_acc

        n_trips_l2_w = 0
        if v_spec_l2['Q'] > 0:
            n_trips_l2_w = math.ceil(tot_load_ts / v_spec_l2['Q'])
        else:
            n_trips_l2_w = float('inf') if tot_load_ts > 0 else 0

        cur_ts_g_vol = tot_load_ts * v_spec_l2['specific_volume_m3_ton']
        n_trips_l2_v = 0
        if v_spec_l2['V'] > 0:
            if v_spec_l2['specific_volume_m3_ton'] == float('inf') and tot_load_ts >
0:

```

```

        n_trips_l2_v = 0
        elif cur_ts_g_vol == float('inf') and v_spec_l2['V'] > 0:
            n_trips_l2_v = float('inf')
        elif v_spec_l2['specific_volume_m3_ton'] > 0:
            n_trips_l2_v = math.ceil(cur_ts_g_vol / v_spec_l2['V'])
        else:
            n_trips_l2_v = 0 if cur_ts_g_vol == 0 else float('inf')
    elif cur_ts_g_vol > 0:
        n_trips_l2_v = float('inf')

n_trips_l2 = max(n_trips_l2_w, n_trips_l2_v)

if n_trips_l2 > 0 and n_trips_l2 != float('inf'):
    cur_l2_rnd_trip_dist = dist_ts_pl_l2 + dist_pl_ts_l2
    l2_dist_type_ts = cur_l2_rnd_trip_dist * n_trips_l2

    tot_trans_cost_acc += l2_dist_type_ts * v_spec_l2['C']
    tot_dist_acc += l2_dist_type_ts
    tot_emis_acc += (l2_dist_type_ts * v_spec_l2['alpha']) + \
                    (tot_load_ts * v_spec_l2['beta'])

    l2_det_routes[(ts_id_l2, act_g_type_id_l2)] = {
        'trips': n_trips_l2,
        'path': [ts_id_l2, P_NODE_ID] }

fin_tot_cost = cur_constr_cost + tot_trans_cost_acc
fin_tot_emis = tot_emis_acc
return fin_tot_cost, fin_tot_emis, tot_dist_acc

def f_5_p(sol_ind_list, sol_idx, r_details, is_rep_best=False, f_tag=None):
    f_fig, ax_plot = p_2_t.subplots(figsize=(14, 11))
    ax_plot.scatter([P_PLANT_COORDS['x']], [P_PLANT_COORDS['y']], c='black',
s=250, marker='P', label='Plant', zorder=5)
    cp_x_coords = [cp_item['x'] for cp_item in C_POINTS_COORDS]; cp_y_coords =
[cp_item['y'] for cp_item in C_POINTS_COORDS]
    ax_plot.scatter(cp_x_coords, cp_y_coords, c='dimgray', s=80, marker='o', label='CP',
alpha=0.7, zorder=3)

    spec_seg_map = {
        frozenset({4, 31}): "SpecSeg: CP4-TS31 (OneWay)",
        frozenset({27, 28}): "SpecSeg: CP27-CP28 (OneWay)",
        frozenset({23, P_NODE_ID}): "SpecSeg: CP23-Plant (TimeRestrict)",

```

```

        frozenset({9, 16}): "SpecSeg: CP9-CP16 (TimeRestrict)",
    }
    spec_seg_leg_handles = {}

    for seg_nodes_fset, desc_str in spec_seg_map.items():
        n_ids = list(seg_nodes_fset)
        if len(n_ids) == 2:
            n1_id, n2_id = n_ids[0], n_ids[1]
            if n1_id in A_NODES_COORDS and n2_id in A_NODES_COORDS:
                c1_dict, c2_dict = A_NODES_COORDS[n1_id],
A_NODES_COORDS[n2_id]
                def_line_clr = 'lightgray'
                def_linestyle = ':'
                def_linewidth = 2.0
                z_ord_spec_bg = 1

                if "TimeRestrict" in desc_str: def_line_clr = 'mistyrose'
                elif "OneWay" in desc_str: def_line_clr = 'lightgreen'

                line_obj, = ax_plot.plot([c1_dict['x'], c2_dict['x']], [c1_dict['y'],
c2_dict['y']],
                                        color=def_line_clr, linewidth=def_linewidth,
linestyle=def_linestyle,
                                        alpha=0.7, zorder=z_ord_spec_bg, label=desc_str)
                if desc_str not in spec_seg_leg_handles:
                    spec_seg_leg_handles[desc_str] = line_obj
                else:
                    print(f"Warn: SpecSeg {desc_str} node coords not found.")

    open_s_ids_plot = r_details.get('open_stations', [])
    for i_val, ts_cand_id in enumerate(TS_IDS_LIST):
        s_data = T_STATIONS_DATA[ts_cand_id]
        is_open_flg = (ts_cand_id in open_s_ids_plot)
        clr_val = 'forestgreen' if is_open_flg else 'lightcoral'
        mrkr_style = 's' if is_open_flg else 'x'; s_val = 150 if is_open_flg else 100
        alpha_val_plot = 1.0 if is_open_flg else 0.5
        ax_plot.scatter([s_data['x']], [s_data['y']], c=clr_val, s=s_val, marker=mrkr_style,
                        label=f"TS{ts_cand_id}" + (' (Open)' if is_open_flg else ' (Closed)'),
                        alpha=alpha_val_plot, zorder=4)

    base_off_d_units = 0.25
    arr_lw = 1.3

```

```

arr_head_sc = 10

seg_use_l1 = {}
l1_r_data = r_details.get('leg1', {})
for (fac_id, g_type_id), r_list_items in l1_r_data.items():
    for r_item in r_list_items:
        for i_val_r in range(len(r_item) - 1):
            n1_val, n2_val = r_item[i_val_r], r_item[i_val_r+1]
            seg_key_val = (n1_val, n2_val)
            if seg_key_val not in seg_use_l1:
                seg_use_l1[seg_key_val] = []
            # Ensure type_id is added only once per segment
            if g_type_id not in seg_use_l1[seg_key_val]:
                seg_use_l1[seg_key_val].append(g_type_id)

# 为每个路段的使用类型排序，以保证偏移顺序一致
for seg_k in seg_use_l1:
    seg_use_l1[seg_k].sort()

for (fac_id_plot, g_type_id_plot), r_list_vrp_plot in l1_r_data.items():
    for r_plot_item in r_list_vrp_plot:
        for i_r_n_plot in range(len(r_plot_item) - 1):
            n1_id = r_plot_item[i_r_n_plot]
            n2_id = r_plot_item[i_r_n_plot+1]

            if n1_id == n2_id: continue

            c1_dict = A_NODES_COORDS[n1_id]
            c2_dict = A_NODES_COORDS[n2_id]

            seg_fset_undir = frozenset({n1_id, n2_id})
            if seg_fset_undir in spec_seg_map:
                continue

            dir_seg_key = (n1_id, n2_id)
            types_on_dir_seg = seg_use_l1.get(dir_seg_key, [])

            if not types_on_dir_seg: continue

            n_types_on_seg = len(types_on_dir_seg)

try:

```

```

        cur_type_idx = types_on_dir_seg.index(g_type_id_plot)
    except ValueError:
        cur_type_idx = 0
        n_types_on_seg = 1
        print(f"Warn: Type {g_type_id_plot} not in seg {dir_seg_key}
users.")

    dx_val = c2_dict['x'] - c1_dict['x']
    dy_val = c2_dict['y'] - c1_dict['y']
    seg_len = math.hypot(dx_val, dy_val)
    if seg_len == 0: continue

    perp_dx_val = -dy_val / seg_len
    perp_dy_val = dx_val / seg_len

    off_val_calc = (cur_type_idx - (n_types_on_seg - 1) / 2.0) *
base_off_d_units

    x1_off_val = c1_dict['x'] + off_val_calc * perp_dx_val
    y1_off_val = c1_dict['y'] + off_val_calc * perp_dy_val
    x2_off_val = c2_dict['x'] + off_val_calc * perp_dx_val
    y2_off_val = c2_dict['y'] + off_val_calc * perp_dy_val

    seg_clr = G_TYPE_COLORS.get(g_type_id_plot, 'black')

    arr_obj = p_3_s.FancyArrowPatch((x1_off_val, y1_off_val), (x2_off_val,
y2_off_val),
                                arrowstyle='->',
                                mutation_scale=arr_head_sc,
                                color=seg_clr, linewidth=arr_lw,
                                alpha=0.85, zorder=2.5)

    ax_plot.add_patch(arr_obj)

l2_r_data = r_details.get('leg2', {})
for (ts_id_plot, g_type_id_plot), l2_info_item in l2_r_data.items():
    r_clr_l2 = G_TYPE_COLORS.get(g_type_id_plot, 'magenta')
    p_nodes_val = l2_info_item['path']
    n_trips_val = l2_info_item['trips']

    if len(p_nodes_val) == 2 and n_trips_val > 0:
        n1_id, n2_id = p_nodes_val[0], p_nodes_val[1]
        c1_dict, c2_dict = A_NODES_COORDS[n1_id],

```

A_NODES_COORDS[n2_id]

```
        seg_fset_undir_l2 = frozenset({n1_id, n2_id})
        lbl_this_l2_arr      =      f"TS      {ts_id_plot}      ->      Plant
({V_PARAMS[g_type_id_plot]["name"][:-2]}, {n_trips_val} trips)'

        if seg_fset_undir_l2 in spec_seg_map:
            continue

        arr_l2_obj = p_3_s.FancyArrowPatch((c1_dict['x'], c1_dict['y']), (c2_dict['x'],
c2_dict['y']),

                                                arrowstyle='->',

mutation_scale=arr_head_sc,

                                                color=r_clr_l2,
                                                linewidth=arr_lw + 0.5,
                                                linestyle='--',
                                                alpha=0.9,                        zorder=2,

label=lbl_this_l2_arr)
        ax_plot.add_patch(arr_l2_obj)

h_list, l_list = ax_plot.get_legend_handles_labels()
uniq_l_dict = {}

for h_item, l_item in zip(h_list, l_list):
    if l_item == 'Plant' and l_item not in uniq_l_dict: uniq_l_dict[l_item] = h_item
    if l_item == 'CP' and l_item not in uniq_l_dict: uniq_l_dict[l_item] = h_item

for h_item, l_item in zip(h_list, l_list):
    if l_item in spec_seg_map.values() and l_item not in uniq_l_dict:
        uniq_l_dict[l_item] = h_item

has_open_ts, has_closed_ts = False, False
for l_chk in l_list:
    if 'TS' in l_chk and '(Open)' in l_chk : has_open_ts = True
    if 'TS' in l_chk and '(Closed)' in l_chk : has_closed_ts = True

if has_open_ts:
    if 'OpenTS' not in uniq_l_dict:
        uniq_l_dict['OpenTS'] = m_1_s.Line2D([], [], color='forestgreen', marker='s',
linestyle='None', markersize=10, label='OpenTS')
if has_closed_ts:
    if 'ClosedTS' not in uniq_l_dict:
```

```

        uniq_l_dict['ClosedTS'] = m_1_s.Line2D([], [], color='lightcoral', marker='x',
linestyle='None', markersize=10, label='ClosedTS')

    for g_type_id_leg, clr_val_leg in G_TYPE_COLORS.items():
        v_name_base_leg = V_PARAMS[g_type_id_leg]["name"][:-2]
        lbl_txt_l1 = f'{v_name_base_leg} Path (L1)'
        if lbl_txt_l1 not in uniq_l_dict:
            uniq_l_dict[lbl_txt_l1] = m_1_s.Line2D([0], [0], color=clr_val_leg, lw=arr_lw
+ 0.5, label=lbl_txt_l1, marker='>')

    for h_item, l_item in zip(h_list, l_list):
        if l_item and 'TS' in l_item and '-> Plant' in l_item and l_item not in uniq_l_dict:
            is_already_spec = any(spec_desc_str in l_item for spec_desc_str in
spec_seg_map.values())
            if not is_already_spec:
                uniq_l_dict[l_item] = h_item

    fin_leg_h = list(uniq_l_dict.values())
    fin_leg_l = list(uniq_l_dict.keys())

    sort_ord_map = {
        'Plant': 0, 'CP': 1,
        **{desc_str: i_val+2 for i_val, desc_str in
enumerate(sorted(spec_seg_map.values()))},
        'OpenTS': len(spec_seg_map) + 2,
        'ClosedTS': len(spec_seg_map) + 3,
    }
    cur_sort_idx = len(sort_ord_map)

    l1_p_labels = sorted([l_str for l_str in fin_leg_l if 'Path (L1)' in l_str])
    for lbl_str in l1_p_labels:
        if lbl_str not in sort_ord_map:
            sort_ord_map[lbl_str] = cur_sort_idx
            cur_sort_idx += 1

    l2_p_labels = sorted([l_str for l_str in fin_leg_l if 'TS' in l_str and '-> Plant' in l_str and
not any(s_desc_str in l_str for s_desc_str in spec_seg_map.values())])
    for lbl_str in l2_p_labels:
        if lbl_str not in sort_ord_map:
            sort_ord_map[lbl_str] = cur_sort_idx
            cur_sort_idx += 1

```

```

comb_leg_items_sort = []
tmp_added_l_sort = set()
for h_itm, l_itm in zip(fin_leg_h, fin_leg_l):
    if l_itm not in tmp_added_l_sort:
        comb_leg_items_sort.append((h_itm, l_itm))
        tmp_added_l_sort.add(l_itm)

def f_6_g(item_tpl_leg):
    lbl_key_leg = item_tpl_leg[1]
    return sort_ord_map.get(lbl_key_leg, float('inf')), lbl_key_leg

comb_leg_items_fin_sort = sorted(comb_leg_items_sort, key=f_6_g)

if not comb_leg_items_fin_sort:
    sort_h_leg, sort_l_leg = [], []
else:
    sort_h_leg, sort_l_leg = zip(*comb_leg_items_fin_sort)

ax_plot.legend(list(sort_h_leg), list(sort_l_leg), loc='upper left', bbox_to_anchor=(1.02,
1), fontsize=11, title="Legend", title_fontsize=13)

ax_plot.set_xlabel('X-Coord', fontsize=12, fontweight='bold')
ax_plot.set_ylabel('Y-Coord', fontsize=12, fontweight='bold')
ax_plot.grid(True, linestyle=':', alpha=0.6)
ax_plot.set_aspect('equal', adjustable='box')
p_2_t.tight_layout(rect=[0.03, 0.03, 0.75, 0.97])

cust_fname = ""
if f_tag:
    cust_fname = f'sol_routes_map_{f_tag}_arr.png'
elif is_rep_best:
    cust_fname = f'sol_routes_map_rep_best_exh_arr.png'
else:
    cust_fname = f'sol_routes_map_par_exh_{sol_idx+1}_arr.png'

p_2_t.savefig(cust_fname, dpi=300)
print(f"Routes map saved: {cust_fname}", flush=True)
p_2_t.close(f_fig)

def f_7_g(ind_list_param):
    open_s_ids_viz = []
    for i_val, is_opn_viz in enumerate(ind_list_param):

```

```

    if is_opn_viz:
        s_id_viz = TS_IDS_LIST[i_val]
        open_s_ids_viz.append(s_id_viz)

cp_allocs_viz = {cp_id: {} for cp_id in G_AT_C_POINTS}
tmp_ts_loads_tons_viz = {ts_id: [0.0] * N_G_TYPES for ts_id in open_s_ids_viz}

aprx_l2_costs_ton_ts_type_viz = {}
for ts_id_calc_viz in open_s_ids_viz:
    aprx_l2_costs_ton_ts_type_viz[ts_id_calc_viz] = {}
    for g_type_id_calc_viz in range(1, N_G_TYPES + 1):
        v_s_params_viz = V_PARAMS[g_type_id_calc_viz]
        cost_p_km_viz = v_s_params_viz['C']
        q_v_viz = v_s_params_viz['Q']
        v_v_viz = v_s_params_viz['V']
        s_vol_viz_param = v_s_params_viz['specific_volume_m3_ton']

        dist_ts_pl_l2_est_viz = G_DIST_MATRIX[ts_id_calc_viz].get(P_NODE_ID,
float('inf'))
        dist_pl_ts_l2_est_viz = G_DIST_MATRIX[P_NODE_ID].get(ts_id_calc_viz,
float('inf'))

        if dist_ts_pl_l2_est_viz == float('inf') or dist_pl_ts_l2_est_viz == float('inf'):
            aprx_l2_costs_ton_ts_type_viz[ts_id_calc_viz][g_type_id_calc_viz] =
float('inf')
            continue

        rnd_trip_dist_l2_est_viz = dist_ts_pl_l2_est_viz + dist_pl_ts_l2_est_viz
        cost_p_dir_trip_viz = rnd_trip_dist_l2_est_viz * cost_p_km_viz

        eff_tons_p_trip_viz = 0
        if q_v_viz > 0:
            tons_by_q_viz = q_v_viz
            if v_v_viz > 0 and s_vol_viz_param > 0 and s_vol_viz_param !=
float('inf'):
                tons_by_v_viz = v_v_viz / s_vol_viz_param
                eff_tons_p_trip_viz = min(tons_by_q_viz, tons_by_v_viz)
            else:
                eff_tons_p_trip_viz = tons_by_q_viz
        if eff_tons_p_trip_viz > 0:
            aprx_l2_costs_ton_ts_type_viz[ts_id_calc_viz][g_type_id_calc_viz] =
cost_p_dir_trip_viz / eff_tons_p_trip_viz

```

```

        else:
            aprx_l2_costs_ton_ts_type_viz[ts_id_calc_viz][g_type_id_calc_viz] =
float('inf')

    for cp_id_viz, g_amts_type_viz in G_AT_C_POINTS.items():
        for g_type_idx_viz, g_amt_tons_viz in enumerate(g_amts_type_viz):
            act_g_type_id_viz = g_type_idx_viz + 1
            if g_amt_tons_viz == 0: continue

            v_spec_viz = V_PARAMS[act_g_type_id_viz]
            v_ck_l1_viz = v_spec_viz['C']
            dist_to_pl_viz = G_DIST_MATRIX[cp_id_viz][P_NODE_ID]
            cost_dir_pl_l1_aprx_viz = dist_to_pl_viz * v_ck_l1_viz
            best_fac_id_cp_viz = P_NODE_ID
            min_est_path_cost_viz = cost_dir_pl_l1_aprx_viz
            for ts_id_opt_viz in open_s_ids_viz:
                ts_s_data_viz = T_STATIONS_DATA[ts_id_opt_viz]
                ts_max_cap_type_viz = ts_s_data_viz['capacities'][g_type_idx_viz]
                cur_load_ts_type_viz =
tmp_ts_loads_tons_viz[ts_id_opt_viz][g_type_idx_viz]
                if cur_load_ts_type_viz + g_amt_tons_viz <= ts_max_cap_type_viz:
                    dist_to_ts_viz_val = G_DIST_MATRIX[cp_id_viz][ts_id_opt_viz]
                    cost_l1_ts_aprx_viz = dist_to_ts_viz_val * v_ck_l1_viz
                    avg_l2_cost_ton_viz =
aprx_l2_costs_ton_ts_type_viz[ts_id_opt_viz].get(act_g_type_id_viz, float('inf'))
                    est_add_l2_cost_viz = float('inf')
                    if avg_l2_cost_ton_viz != float('inf'):
                        est_add_l2_cost_viz = g_amt_tons_viz * avg_l2_cost_ton_viz
                    cur_ts_path_tot_cost_aprx_viz = cost_l1_ts_aprx_viz +
est_add_l2_cost_viz
                    if cur_ts_path_tot_cost_aprx_viz < min_est_path_cost_viz:
                        min_est_path_cost_viz = cur_ts_path_tot_cost_aprx_viz
                        best_fac_id_cp_viz = ts_id_opt_viz
                    cp_allocs_viz[cp_id_viz][act_g_type_id_viz] = best_fac_id_cp_viz
                    if best_fac_id_cp_viz != P_NODE_ID:
                        tmp_ts_loads_tons_viz[best_fac_id_cp_viz][g_type_idx_viz] +=
g_amt_tons_viz

    l1_det_routes_viz = {}
    for fac_n_id_l1_viz in [P_NODE_ID] + open_s_ids_viz:
        cur_fac_tw_raw_viz = P_OP_WINDOW
        if fac_n_id_l1_viz != P_NODE_ID:

```

```

        cur_fac_tw_raw_viz = T_STATIONS_DATA[fac_n_id_l1_viz]['tw']
        fac_tw_start_sc_viz = cur_fac_tw_raw_viz[0] * T_UNIT_SCALER
        fac_tw_end_sc_viz = cur_fac_tw_raw_viz[1] * T_UNIT_SCALER

    n_tw_fac_vrp_viz = {
        fac_n_id_l1_viz: (fac_tw_start_sc_viz, fac_tw_end_sc_viz)
    }

    for g_type_id_l1_viz in range(1, N_G_TYPES + 1):
        v_spec_l1_viz = V_PARAMS[g_type_id_l1_viz]
        c_for_vrp_l1_viz, dems_for_vrp_l1_viz = [], []
        cur_vrp_n_tw_viz = n_tw_fac_vrp_viz.copy()

        for cp_id_l1_viz_in, allocs_cp_viz_in in cp_allocs_viz.items():
            if allocs_cp_viz_in.get(g_type_id_l1_viz) == fac_n_id_l1_viz:
                dem_this_type_cp_viz =
G_AT_C_POINTS[cp_id_l1_viz_in][g_type_id_l1_viz-1]
                if dem_this_type_cp_viz > 0:
                    c_for_vrp_l1_viz.append(cp_id_l1_viz_in)
                    dems_for_vrp_l1_viz.append(dem_this_type_cp_viz)
                    cur_vrp_n_tw_viz[cp_id_l1_viz_in] = (fac_tw_start_sc_viz,
fac_tw_end_sc_viz)

            if not c_for_vrp_l1_viz: continue

        n_tw_tuple_items_l1_viz = tuple(sorted(cur_vrp_n_tw_viz.items()))

        _dist_viz_val, _cost_viz_val, r_list_viz_val = f_2_s(
            d_node_id=fac_n_id_l1_viz,
            c_node_ids=c_for_vrp_l1_viz,
            d_at_c_tons=dems_for_vrp_l1_viz,
            v_cap_tons=v_spec_l1_viz['Q'],
            v_cap_vol_m3=v_spec_l1_viz['V'],
            v_cost_km=v_spec_l1_viz['C'],
            v_spec_vol=v_spec_l1_viz['specific_volume_m3_ton'],
            n_time_windows=cur_vrp_n_tw_viz,
            n_v_avail=len(c_for_vrp_l1_viz),
            g_dist_mat=G_DIST_MATRIX
        )

        if r_list_viz_val:
            l1_det_routes_viz[(fac_n_id_l1_viz, g_type_id_l1_viz)] = r_list_viz_val

```

```

l2_det_routes_viz = {}
for ts_id_l2_viz in open_s_ids_viz:
    for g_type_idx_l2_viz, tot_load_ts_viz in
enumerate(tmp_ts_loads_tons_viz[ts_id_l2_viz]):
        act_g_type_id_l2_viz = g_type_idx_l2_viz + 1
        if tot_load_ts_viz == 0: continue

        v_spec_l2_viz = V_PARAMS[act_g_type_id_l2_viz]

        dist_ts_pl_g_r = G_DIST_MATRIX[ts_id_l2_viz].get(P_NODE_ID,
float('inf'))
        dist_pl_ts_g_r = G_DIST_MATRIX[P_NODE_ID].get(ts_id_l2_viz,
float('inf'))

        if dist_ts_pl_g_r == float('inf') or dist_pl_ts_g_r == float('inf'):
            pass

        n_trips_l2_w_viz = math.ceil(tot_load_ts_viz / v_spec_l2_viz['Q']) if
v_spec_l2_viz['Q'] > 0 else (float('inf') if tot_load_ts_viz > 0 else 0)
        cur_ts_g_vol_viz = tot_load_ts_viz *
v_spec_l2_viz['specific_volume_m3_ton']
        n_trips_l2_v_viz = 0
        if v_spec_l2_viz['V'] > 0:
            if v_spec_l2_viz['specific_volume_m3_ton'] == float('inf') and
tot_load_ts_viz > 0:
                n_trips_l2_v_viz = 0
            elif cur_ts_g_vol_viz == float('inf') and v_spec_l2_viz['V'] > 0:
                n_trips_l2_v_viz = float('inf')
            elif v_spec_l2_viz['specific_volume_m3_ton'] > 0:
                n_trips_l2_v_viz = math.ceil(cur_ts_g_vol_viz / v_spec_l2_viz['V'])
            else:
                n_trips_l2_v_viz = 0 if cur_ts_g_vol_viz == 0 else float('inf')
        elif cur_ts_g_vol_viz > 0:
            n_trips_l2_v_viz = float('inf')

        n_trips_l2_viz = max(n_trips_l2_w_viz, n_trips_l2_v_viz)

        if n_trips_l2_viz > 0 and n_trips_l2_viz != float('inf'):
            cur_l2_rnd_trip_dist_viz = dist_ts_pl_g_r + dist_pl_ts_g_r
            l2_dist_type_ts_viz = cur_l2_rnd_trip_dist_viz * n_trips_l2_viz

```

```

        l2_det_routes_viz[(ts_id_l2_viz, act_g_type_id_l2_viz)] = {
            'trips': n_trips_l2_viz, 'path': [ts_id_l2_viz, P_NODE_ID] }
    return { 'leg1': l1_det_routes_viz, 'leg2': l2_det_routes_viz, 'open_stations':
open_s_ids_viz, 'allocations_debug': cp_allocs_viz }

def f_8_p(all_res_list, par_front_list, f_name_param="all_sols_vs_par_front.png"):
    feas_costs = [r_item['cost'] for r_item in all_res_list if r_item['cost'] != float('inf') and
r_item['emission'] != float('inf')]
    feas_emis = [r_item['emission'] for r_item in all_res_list if r_item['cost'] != float('inf')
and r_item['emission'] != float('inf')]

    par_costs = [ind_item.fitness.values[0] for ind_item in par_front_list]
    par_emis = [ind_item.fitness.values[1] for ind_item in par_front_list]

    p_2_t.figure(figsize=(12, 8))
    p_2_t.scatter(feas_costs, feas_emis, c='lightblue', marker='o', label='FeasibleSol', s=50,
alpha=0.7, zorder=1)
    if par_costs:
        p_2_t.scatter(par_costs, par_emis, c='red', marker='s', label='ParetoSol', s=70,
zorder=2)

    p_2_t.xlabel('TotalCost (Yuan/Day)', fontsize=12)
    p_2_t.ylabel('TotalCO2Emissions', fontsize=12)
    p_2_t.title('All Feasible Sols & Pareto Front (Exhaustive)', fontsize=14)
    p_2_t.legend(fontsize=10)
    p_2_t.grid(True, linestyle='--', alpha=0.7)
    p_2_t.tight_layout()
    try:
        p_2_t.savefig(f_name_param, dpi=300)
        print(f'All sols vs Pareto plot saved: {f_name_param}", flush=True)
    except Exception as e_exc:
        print(f'Save Pareto plot failed: {e_exc}", flush=True)
    p_2_t.close()

if __name__ == "__main__":
    n_s_cand = N_TS_CANDIDATES
    all_eval_res = []

    tgt_comb_idx = 16
    cur_ind_list = [1, 0, 0, 0, 0]
    print(f'Evaluating specific combo: {cur_ind_list} (Open TS 31)", flush=True)

```

```

print(f"  Eval combo: {cur_ind_list}...", end=" ", flush=True)
cost_val, emis_val, dist_val_res = f_4_e(cur_ind_list)

if cost_val == float('inf') or emis_val == float('inf') or dist_val_res == float('inf'):
    print(f"Result:  Infeasible  (Cost:  {cost_val},  Emis:  {emis_val},  Dist:
{dist_val_res})", flush=True)
else:
    print(f"Result: Cost={cost_val:.2f}, Emis={emis_val:.2f}, Dist={dist_val_res:.2f}
km", flush=True)
    all_eval_res.append({'config': cur_ind_list, 'cost': cost_val, 'emission': emis_val,
'distance': dist_val_res})

print("\nSpecific combo eval done.", flush=True)

print("\nFeasible sols (cost, emis, dist valid):", flush=True)
feas_sols_list = []
for idx_res_item, res_item_val in enumerate(all_eval_res):
    if res_item_val['cost'] != float('inf') and res_item_val['emission'] != float('inf') and
res_item_val['distance'] != float('inf'):
        feas_sols_list.append(res_item_val)
        opened_n_list = [TS_IDS_LIST[j_val] for j_val, bit_v_opn in
enumerate(res_item_val['config']) if bit_v_opn]
        print(f"    Combo  {idx_res_item+1}:  {res_item_val['config']}  (Open:
{opened_n_list if opened_n_list else 'None'}), Cost: {res_item_val['cost']:.2f}, Emis:
{res_item_val['emission']:.2f}, Dist: {res_item_val['distance']:.2f} km", flush=True)
    if not feas_sols_list:
        print("  Specified sol infeasible.", flush=True)
        sys.exit()

par_cand_objs = []
if feas_sols_list:
    res_par_item = feas_sols_list[0]
    ind_obj_val = c_1_r.Individual(res_par_item['config'])
    ind_obj_val.fitness.values = (res_par_item['cost'], res_par_item['emission'])
    ind_obj_val.distance = res_par_item['distance']
    par_cand_objs.append(ind_obj_val)

if not par_cand_objs:
    print("Specified sol infeasible after eval, cannot build Pareto.", flush=True)
else:
    par_front_sols_list = par_cand_objs
    print(f"Found {len(par_front_sols_list)} Pareto sol (current eval sol).", flush=True)

```

```

        plot_fname_par_main =
f'all_sols_vs_par_front_single_cfg_{TS_IDS_LIST[0]}_only.png"
        f_8_p(all_eval_res, par_front_sols_list, f_name_param=plot_fname_par_main)

    print("\nPareto sol details & routes map gen:", flush=True)
    if not par_front_sols_list:
        print("Pareto front empty, no map gen.")
    else:
        rep_best_sol_val = par_front_sols_list[0]

        print("\n--- Specific Combo Details ---", flush=True)
        cost_v_rep, emis_v_rep = rep_best_sol_val.fitness.values
        dist_v_rep = rep_best_sol_val.distance
        opened_s_cfg_list_rep = list(rep_best_sol_val)
        opened_s_ids_n_rep = [TS_IDS_LIST[j_val] for j_val, bit_v_rep in
enumerate(opened_s_cfg_list_rep) if bit_v_rep]
        print(f"  TS Combo: {opened_s_cfg_list_rep} (Open: {opened_s_ids_n_rep if
opened_s_ids_n_rep else 'None'})")
        print(f"  TotalCost: {cost_v_rep:.2f}")
        print(f"  TotalEmis: {emis_v_rep:.2f}")
        print(f"  TotalDist: {dist_v_rep:.2f} km")
        print(f"  NumOpenTS: {sum(opened_s_cfg_list_rep)}")

        try:
            print(f"  Gen specific combo routes map...", end=" ", flush=True)
            r_details_best_val = f_7_g(opened_s_cfg_list_rep)
            f_5_p(opened_s_cfg_list_rep, 0, r_details_best_val, is_rep_best=True,
f_tag=f"cfg_ts{TS_IDS_LIST[0]}_only_asymm")
            print("Done.", flush=True)
        except Exception as e_plot_best_exc:
            print(f"Failed. Error: {e_plot_best_exc}", flush=True)
            traceback.print_exc(file=sys.stderr)
        print("--- Specific Combo Details End ---\n", flush=True)

    print("\nViz results saved to:")
    plot_fname_par = f'all_sols_vs_par_front_single_cfg_{TS_IDS_LIST[0]}_only.png"
    plot_fname_r = f'sol_routes_map_cfg_ts{TS_IDS_LIST[0]}_only_asymm.png"

    print(f"1. {plot_fname_par} - Single eval sol vs (self) Pareto front plot")
    print(f"2. {plot_fname_r} - Specific combo routes map")

```

```
print("\nScript done.")
```
